TOWARDS REALISTIC SIMULATED STUDENTS: ALIGNING LANGUAGE MODELS WITH STUDENT KNOWLEDGE AND BEHAVIOR

A Dissertation Presented

by

ALEXANDER SCARLATOS

Submitted to the Graduate School of the
University of Massachusetts Amherst in partial fulfillment
of the requirements for the degree of

DOCTOR OF PHILOSOPHY

May 2026

Manning College of Information and Computer Sciences

© Copyright by Alexander Scarlatos 2026

All Rights Reserved

TOWARDS REALISTIC SIMULATED STUDENTS: ALIGNING LANGUAGE MODELS WITH STUDENT KNOWLEDGE AND BEHAVIOR

A Dissertation Presented

by

ALEXANDER SCARLATOS

Approved as to style and content by:

Andrew Lan, Chair

Ivon Arroyo, Member

Brendan T. O'Connor, Member

Ryan S. Baker, Outside Member

Ramesh K. Sitaraman, Associate Dean for Educational Programs and Teaching
Manning College of Information and Computer Sciences

DEDICATION

To mom, dad, Jacqueline, and Phoebe.

ACKNOWLEDGMENTS

First, I want to thank my advisor Andrew Lan for his guidance, never ending support and belief in me, and for shaping me into the researcher I am today. He has been generous, patient, and inspiring. He created an environment where my labmates and I could thrive, pursue research we were passionate about, and see the rewards of our hard work. I owe much of my success to Andrew for pushing me to do more than what I thought I was capable of. As he would say, "Good job!"

I want to thank the members of my committee, Ivon Arroyo, Brendan O'Connor, and Ryan Baker, for their support and guidance on making this thesis as impactful as it can be, as well as their support throughout my Ph.D.

I want to thank the members of my lab, who have been loyal, kind, generous, invaluable brainstormers and collaborators, and most importantly, good friends: Nigel Fernandez, Jaewook Lee, Hunter McNichols, Wanyong Feng, Nischal Ashok Kumar, Hasnain Heickal, Zhangqi Duan, Fareya Ikram, Nisarg Parikh, and Shuyan Huang. I also want to thank my mentees, Adriana Caraeni and Farzana Chowdhury, for trusting me to guide them and teaching me how to be a better teacher.

I want to thank my industry collaborators, in particular, Yusong Wu, Anna Huang, Natasha Jaques, Christopher Ormerod, Susan Lottridge, and Simon Woodhead, for their support and determination that made much of my work possible. I would also like to thank the many members of the AI in education community that I have had the pleasure of meeting these past few years; they helped shape my perspective on research, have been incredibly welcoming and kind, and made my conference trips truly memorable.

I want to thank my mentors who believed in me and went out of their way to create opportunities for me to explore my passions, in particular, Donna Siani, Janice Moore, Bob Caputo, Eileen Donlon, and Margaret Schedel.

I want to thank my friends, who were my original collaborators, brought out the creativity in me, and have been a source of endless joy, in particular, Matt Levine, Brandon Dixon, Melissa Knapp, Shea Glasheen, Mike Krukowski, and Rob Morandini.

I want to thank my wife Jacqueline, who has patiently been my rock, my "rubber duck", my source of joy and laughter, my best friend, and my reason for not giving up for the last five years. I thank my daughter Phoebe for the love she brings and for giving my life a new meaning. I thank my parents, who have been my biggest inspiration, for raising me to be curious and hard working but to always pursue what is right. I thank my sister Jasmine for always believing in me and for being my loyal companion and concert buddy. I thank the rest of my family who have been there for me my whole life, helped raise me, taught me kindness and how to be open minded, and continue to be by my side when I need them.

Finally, I thank God for all I have and all I am. This life is a gift.

ABSTRACT

TOWARDS REALISTIC SIMULATED STUDENTS: ALIGNING LANGUAGE MODELS WITH STUDENT KNOWLEDGE AND BEHAVIOR

MAY 2026

ALEXANDER SCARLATOS

B.S., STONY BROOK UNIVERSITY

M.S., STONY BROOK UNIVERSITY

Ph.D., UNIVERSITY OF MASSACHUSETTS AMHERST

Directed by: Professor Andrew Lan

As large language models (LLMs) are increasingly used in education, there is a growing need to quickly verify and steer LLM-generated content for the benefit of students. In particular, it is important to ensure that educational AI systems improve learning outcomes and engagement while minimizing potential harms. While it is possible to test new systems and content on real students, this process can be slow, costly, and insecure. Instead, simulated students, i.e., LLM-based models that mimic student behavior, can be used to quickly and safely verify AI systems. However, LLMs do not typically behave like real students, often exhibiting unrealistic behavioral patterns and knowledge trends, limiting how useful they can be. In this thesis, I address this problem by introducing multiple approaches for aligning LLMs with student behavior and knowledge. I further demonstrate how simulated students can be leveraged to align educational content with student-centered goals.

First, I detail SMART, a method for fine-tuning LLMs with reinforcement learning (RL) that aligns the models with realistic response patterns based on student ability and question difficulty, deriving a reward function from item response theory. I demonstrate how LLMs trained with SMART can be used to reliably estimate question difficulty. Second, I introduce dialogue knowledge tracing (KT), where tutor-student dialogues are viewed as sequences of formative assessments, allowing the use of KT to model student knowledge in dialogues. I also detail LLMKT, an LLM-based method for estimating student knowledge that improves on prior KT approaches. Third, I detail a method for leveraging student models in LLM-based tutor training, where I use an RL pipeline that rewards tutor responses that result in better student learning outcomes, which are estimated using LLMKT. Fourth, I detail an evaluation framework that measures the realism of student simulations along multiple behavioral, cognitive, and linguistic dimensions, facilitating the development of more aligned simulated student methods in the future. Finally, I conclude by detailing future directions for improving and broadening simulated students methods, and for leveraging simulated students to tackle current challenges in education.

TABLE OF CONTENTS

Page

ACKNOWLEDGMENTS . . . . . **v**

ABSTRACT . . . . . **vii**

LIST OF TABLES . . . . . **xv**

LIST OF FIGURES . . . . . **xix**

CHAPTER

1. INTRODUCTION **1**

1.1 Background 3

1.1.1 Intelligent Tutoring Systems 3

1.1.1.1 ITS Efficacy and Self-Regulated Learning 4

1.1.2 Cognitive Modeling 5

1.1.3 Student Modeling 7

1.1.3.1 Item Response Theory 7

1.1.3.2 Knowledge Tracing 8

1.1.3.3 Alternative Student Model Architectures 9

1.1.3.4 Behavioral Student Modeling 9

1.1.3.5 Applications of Student Models 10

1.1.4 Student Simulation 10

1.2 Thesis Overview 12

1.3 Summary of Relevant Publications 14

2. SMART: SIMULATED STUDENTS ALIGNED WITH ITEM RESPONSE THEORY FOR QUESTION DIFFICULTY PREDICTION **16**

2.1 Introduction . . . 16
2.1.1 Contributions . . . 17
2.2 Related Work . . . 18
2.2.1 Direct Item Difficulty Prediction . . . 18
2.2.2 Simulation-Based Difficulty Prediction . . . 19
2.3 Background: Item Response Theory . . . 19
2.4 Problem Formulation . . . 20
2.5 Methodology . . . 21
2.5.1 LLM-based Student Response Simulation . . . 21
2.5.2 SMART: Ability Alignment in Response Simulation via Preference Optimization . . . 22
2.5.2.1 Preference Pair Creation . . . 23
2.5.3 Scoring Model for Open-ended Responses . . . 23
2.5.4 Simulation-Based Difficulty Prediction . . . 24
2.6 Experimental Evaluation . . . 25
2.6.1 Datasets . . . 25
2.6.1.1 Smarter Balanced . . . 25
2.6.1.2 CodeWorkout . . . 26
2.6.2 Metrics . . . 27
2.6.2.1 Item Difficulty Prediction . . . 27
2.6.2.2 Student Simulation . . . 28
2.6.3 Baselines . . . 29
2.6.3.1 Direct Difficulty Prediction Baselines . . . 29
2.6.3.2 Student Simulation-based Difficulty Prediction Baselines . . . 30
2.6.4 Experimental Setup . . . 31
2.6.5 IRT Model Fitting . . . 32
2.7 Results, Analysis, and Discussion . . . 34
2.7.1 Quantitative Results . . . 35

2.7.1.1 SMART outperforms other methods on item difficulty prediction . . . 35
2.7.1.2 Student simulation is key when working with few training items . . . 36
2.7.1.3 Ablation study . . . 36
2.7.1.4 SMART simulated student responses are aligned with abilities and diverse . . . 37
2.7.2 Data Scaling Experiment . . . 37
2.7.3 Qualitative Evaluation . . . 38
2.7.3.1 Models Reflect Ability-Specific Features in Student Responses . . . 39
2.7.3.2 Performance is Item-dependent . . . 39
2.7.3.3 Outlier Generated Responses . . . 41
2.8 Conclusions and Future Work . . . 41
3. EXPLORING KNOWLEDGE TRACING IN TUTOR-STUDENT DIALOGUES USING LLMS . . . 42
3.1 Introduction . . . 42
3.1.1 Contributions . . . 44
3.2 Related Work . . . 45
3.2.1 Knowledge Tracing and Knowledge Component Labeling . . . 45
3.2.2 Dialogue Analysis . . . 46
3.3 Knowledge Tracing in Dialogues . . . 47
3.4 Methodology . . . 49
3.4.1 Automated Dialogue Annotation . . . 49
3.4.2 LLMKT . . . 50
3.4.3 DKT-Sem . . . 52
3.5 Experiments . . . 54
3.5.1 Datasets . . . 54
3.5.2 KT Methods . . . 55
3.5.3 Metrics and Experimental Setup . . . 56
3.6 Experimental Results . . . 57
3.6.1 Quantitative Results . . . 57
3.6.2 Qualitative Analysis: Case Study . . . 60

3.6.3 "Learning Curve" Analysis . . . 62

3.7 Annotation Quality Experiments . . . 64

3.7.1 Human Evaluation . . . 64
3.7.2 Error Analysis . . . 66

3.8 Discussion, Conclusions, and Future Work . . . 67

4. TRAINING LLM-BASED TUTORS TO IMPROVE STUDENT LEARNING OUTCOMES IN DIALOGUES . . . 68

4.1 Introduction . . . 68

4.1.1 Contributions . . . 69

4.2 Related Work . . . 70

4.2.1 AI Tutors in Dialogues . . . 70
4.2.2 Student Outcome Modeling . . . 71
4.2.3 Evaluating Pedagogical Quality of LLMs . . . 71

4.3 Methodology . . . 72

4.3.1 Student Outcome Prediction . . . 72
4.3.2 Following Pedagogical Principles . . . 74
4.3.3 Preference Pair Construction . . . 75
4.3.4 Model Training . . . 76

4.4 Experimental Settings . . . 77

4.4.1 Dataset . . . 77
4.4.2 Baselines . . . 77
4.4.3 Automated Metrics . . . 78
4.4.4 Model Parameters . . . 78

4.5 Experimental Results . . . 79

4.5.1 Quantitative Results . . . 79
4.5.2 Qualitative Analysis . . . 81
4.5.3 Human Evaluation . . . 82

4.6 Conclusions and Future Work . . . 85

5. SIMULATED STUDENTS IN TUTORING DIALOGUES: SUBSTANCE OR ILLUSION? . . . 86

5.1 Introduction . . . 86
5.1.1 Contributions . . . 87
5.2 Related Work . . . 88
5.2.1 Student Simulation in Dialogues . . . 88
5.2.2 Evaluating Simulated Students . . . 89
5.2.3 Student Simulation in Other Settings . . . 90
5.3 Methodology . . . 91
5.3.1 LLM-based Student Simulation . . . 91
5.3.2 Evaluating Simulated Student Turns . . . 92
5.3.2.1 Dialogue Acts . . . 92
5.3.2.2 Correctness . . . 94
5.3.2.3 Errors . . . 94
5.3.2.4 Knowledge Acquisition . . . 95
5.3.2.5 Linguistics . . . 96
5.3.2.6 Inducing Tutor Responses . . . 96
5.3.3 Tuning with RL . . . 96
5.4 Experiments . . . 97
5.4.1 Dataset . . . 97
5.4.2 Simulated Student Benchmarks . . . 99
5.4.2.1 Fine-Tuning . . . 99
5.4.2.2 Prompting . . . 99
5.4.3 Models and Hyperparameters . . . 101
5.4.4 Evaluation Metrics . . . 103
5.4.5 Human Evaluation . . . 103
5.5 Results . . . 106
5.5.1 Quantitative Results . . . 106
5.5.2 Human Evaluation . . . 107
5.5.2.1 Metric and Label Agreement . . . 108
5.5.3 Qualitative Analysis . . . 109
5.5.4 Ablation Study . . . 111
5.5.5 Label Distribution Analysis . . . 112
5.5.6 Results by Turn . . . 114

5.5.7 Examples 115

5.6 Conclusions and Future Work 121

6. CONCLUSIONS 123

6.1 Future Work 124

6.1.1 Data-Driven Personalization and Adaptation 124
6.1.2 Open-Ended and Creative Learning Environments 125
6.1.3 Agentic Educational Assistants 126

BIBLIOGRAPHY 128

LIST OF TABLES

Table **Page**

2.1 Performance on question difficulty prediction. Best performance is in **bold** and second best is underlined. SMART outperforms all direct prediction methods and generally outperforms other simulation-based methods. 31

2.2 Performance on student simulation metrics. Best performance is in **bold** and second best is underlined. SMART significantly outperforms other methods on ability following. 35

2.3 Examples of student responses generated by SFT and SMART for a publicly available Smarter Balanced item, conditioned on abilities in the low, medium, and high ranges. SMART adapts better to the instructed ability than SFT. For low ability, SMART uses very simple language and doesn't discuss how plants make food, while SFT uses more formal language and does mention how plans make food, which would be atypical for a low ability student. For medium ability, both methods write a coherent introduction but don't mention how plants make food; SMART writes a slightly longer response. For high ability, SMART makes sure to motivate the importance of photosynthesis as well as directly answer the question by stating how plants make food, while SFT neglects to mention how plants make food. 40

3.1 Statistics for the CoMTA and MathDial datasets, from left to right: total number of dialogues, total number of turn pairs with correctness labels, percentage of turn pairs where the student response is correct, total number of unique KCs in the dataset, average number of unique KCs per dialogue, average number of KCs per turn pair, and percentage of turn pairs with more than one KC. 54

3.2 Student dialogue turn correctness prediction performance (averaged with standard deviation shown when applicable) for all KT methods, with best numbers in **bold** and second-best underlined. LLMKT outperforms all other methods on both datasets. 58

3.3 An example dialogue with student turn correctness labels, KC labels, and LLMKT's predicted mastery level for each KC. LLMKT accurately predicts student correctness at both turns by leveraging the difficulty of the tutor-posed questions. 60

3.4 Scores and IRR metrics for the human evaluation. Additionally, final turn correctness accuracy for GPT-4o (on the full dataset followed by only on the human evaluation dataset) and the human annotators. 65

4.1 Pedagogical evaluation criteria for tutor utterances in dialogues. 74

4.2 Tutor utterance evaluations for each method on the test set. The best result for each metric is in **bold** while the second best is underlined. DPO significantly outperforms all methods on improving student correctness predictions, and closely matches the performance of GPT-4o on the pedagogical rubric. 79

4.3 Comparing human tutor, GPT-4o, and DPO tutor utterances, on student outcomes (SO) and pedagogical principles (PP). Brown highlights the tutor-posed task. Some irrelevant text in the dialogue context has been removed for brevity. 83

4.4 Human evaluation shows that DPO outperforms human tutors and GPT-4o. 84

5.1 Dialogue acts that students can make at any turn in a dialogue. 93

5.2 Training statistics for all fine-tuned models. Train and test time are reported in minutes. The validation performance (Val. Perf.) metric is accuracy for Acts and Correctness and AUC for Knowledge. 102

5.3 Results on turn-level student utterance prediction. Prompting-based methods perform well on correctness, while fine-tuned methods perform well on acts, knowledge, linguistic similarity, and tutor responses. All methods, other than Oracle, perform poorly on error prediction. Best method is **bolded** and second best is underlined. 103

5.4 Correctness, *Did the student answer correctly?* 105

5.5 Linguistic Similarity, *Ignoring correctness, how linguistically similar is the simulated turn to the actual turn in wording and structure?* 105

5.6 Human evaluation results, where trends roughly match those on automated metrics. 108

5.7 Agreement between human evaluators and automated metrics, human evaluators and LLM-assigned annotations, and inter-rater agreement. 109

5.8 Results of reward function ablation study. Most reward functions result in high performance on the corresponding metric. Best method is **bolded** and second best is underlined. 111

5.9 Qualitative example for a single turn, showing the ground-truth student response (bordered in green), the simulated responses for that turn from all methods (bordered in yellow), and corresponding scores across all metrics. The ground-truth student response is an incorrect math answer (MA) to the previous tutor turn, where the correct answer would divide in the other direction. Most simulated responses are classified as MA, while some are classified as Seek Information (SI) due to having a more questioning tone. All simulated responses except for SFT 8B and DPO 8B are correct, with SFT 3B and DPO 3B inverting 5/8, most prompting methods inverting the full equation, and Reasoning giving a complete solution to the problem. While SFT 8B and DPO 8B are both incorrect, only DPO 8B has the same error as the ground-truth student response. For knowledge, we show the score and the average delta quantile across KCs. In this example, knowledge is highly correlated with task difficulty, reflecting findings in [205]. Specifically, incorrect or partially correct answers are likely to lead to easier followup tutor questions due to scaffolding, and are therefore more likely to be answered correctly by the student. On the other hand, correct answers are likely to be followed by the tutor posing a new task, which is likely to be harder than a scaffolded question, leading to a lower likelihood of the student answering correctly. The knowledge increases for Reasoning because the question is solved, so no new tasks will be posed. Cos. Sim. and ROUGE-L are highly correlated with textual overlap. Finally, Tutor Resp. is lower for prompting methods since they give full solutions, which would be unlikely to lead to the ground-truth tutor response. 117

5.10 Example decomposition of Knowledge Acquisition similarity for a single turn. The simulated turn is from DPO 8B. Early turns in the dialogue with greetings are excluded for brevity. While both answers are correct, the ground-truth simply gives the final answer option, while the simulated turn gives the full fraction associated with the correct answer. The knowledge tracing model estimates that the student's knowledge delta for the ground-truth turn is roughly average for all KCs (2 or 1 between 0 and 4), while the knowledge delta for the simulated turn is above average for all KCs (all 3). This difference may be due to the possibility that answering "c" is simply a lucky guess, whereas answering "6/10" conveys an understanding of the solution, leading to a higher knowledge estimate. This example demonstrates the need for a knowledge-based metric, since two turns can have the same correctness but give different indications of student mastery on concepts. 118

5.11 Example dialogue, question, solution, and associated context provided to OCEAN, Oracle, and ICL prompting methods. Emojis except checkmark are removed. 121

LIST OF FIGURES

Figure **Page**

2.1 Our three-stage pipeline for item difficulty prediction: simulate open-ended responses for students with different abilities, score them, and fit an IRT model on the scores to obtain difficulty estimates. 17

2.2 Our novel preference pair creation method through comparing response likelihood under IRT. 22

2.3 Results of data scaling experiment. Even a small amount of data is sufficient to achieve high performance. SMART begins to outperform SFT more significantly when more data is available. 38

3.1 Overview of the dialogueKT framework and LLMKT. For annotation, we provide a full dialogue to GPT-4o, which generates correctness labels and retrieves Common Core standards as KCs for each turn. For KT, LLMKT takes a dialogue up to the target turn and its KCs to predict mastery on each KC and response correctness. 47

3.2 Knowledge change curves, i.e., estimated KC mastery level changes across dialogue turns, averaged over students for selected KCs in the CoMTA dataset. More common KCs in longer dialogues tend to show increasing mastery levels as the dialogue progresses, while less common KCs tend to show the opposite. KCs shown: "Solve linear equations and inequalities in one variable," "Understand that polynomials form a system analogous to the integers," and "Apply the properties of operations to generate equivalent expressions." 62

4.1 An overview of our methodology for training LLMs to generate tutor utterances with the joint objective of maximizing student learning outcomes and following good pedagogical principles. *Human tutor* refers to tutor utterances in MathDial. 73

4.2 Results of our experiment adjusting the value of λ in DPO training. Left: student correctness prediction changing with λ. Right: pedagogical rubric score changing with λ. We also show values for GPT-4o and the Distill model for comparison. 80

5.1 Overview of the seven evaluation metrics for simulated student turn evaluation, with a real paraphrased tutor–student dialogue serving as the reference. In this example, the ground-truth student turn and the simulated turn have the same dialogue act *Math Answer*; however, the target turn is incorrect while the simulated turn is correct. 92

5.2 Distribution of act labels for real students and simulated student methods. 112

5.3 Distribution of correctness labels for real students and simulated student methods. 113

5.4 Results across metrics and methods broken down for each turn pair index. 114

CHAPTER 1

INTRODUCTION

Recent advances in artificial intelligence (AI), particularly large language models (LLMs), have greatly changed the landscape of educational technology. Using LLMs, it is possible to build highly personalized and adaptive educational systems, where models can process nuanced student inputs in real-time and rapidly generate novel content designed for a particular student. This paradigm stands in contrast to the previous generation of adaptive educational systems, where all content was human-written or generated from carefully designed templates, and only limited modalities of student data could be processed (e.g. if the student answered a question correctly or incorrectly). LLMs therefore have the potential to reshape education broadly, where in the future every student could have access to a personal AI tutor that optimizes content for their personal preferences, adapts to their current knowledge level, and draws on strong pedagogical practices to help the student learn and grow. However, there are several barriers to achieving this future.

A primary challenge is that state-of-the-art LLMs are not *aligned* with real student learning patterns, behaviors, or goals. LLMs are primarily trained to generate outputs that are preferred by data annotators, which may be significantly unaligned with outputs that are best for learning. For instance, "productive struggle" is necessary for learning [243], but LLM outputs that elicit associated emotions like frustration are unlikely to be rated highly, and may be especially hard to achieve with models that are prone to sycophancy [182]. Additionally, providing effective instruction requires a comprehensive understanding of a student's current knowledge and abilities; these are

likely hard for the model to assess without explicit training for knowledge estimation. While several frontier LLMs have begun integrating techniques for improving pedagogical alignment [181, 15, 110], their approaches have several shortcomings: 1) they rely on a one-size-fits-all approach, where a pre-defined pedagogical strategy is used for every student, and 2) they are developed without students-in-the-loop, instead using a top-down approach where the primary steering mechanism is a set of tutoring guidelines, relying on the model's inherent instruction-following ability to implement these guidelines in a way that is optimal for the student.

An ideal solution to these challenges would involve training LLMs with real students in-the-loop. For instance, we can imagine a general reinforcement learning (RL) pipeline: real students interact with LLMs during training, and LLM-generated outputs that result in higher engagement and learning outcomes for those students are given a higher reward. However, this framework is likely unachievable, since 1) a vast amount of data would be needed to train in this way, resulting in prohibitive cost and logistical challenges, and 2) since some LLM outputs will inevitably be low-quality, this risks causing real harm to students involved in the training. Therefore, a promising solution is to use *simulated students*, i.e., AI models that are trained to behave like real students. By swapping out real students for simulated ones in the framework above, we can achieve the best of both worlds: educational models are trained to improve student outcomes while avoiding risks to real students. However, a central challenge remains: *how can we create AI models that behave like real students?* In this thesis, I seek to answer this question and explore how to leverage simulated students to create student-aligned educational AI systems. In the next section, I provide a background on educational systems and prior work related to simulated students. Then in the following section, I provide an overview of this thesis and detail its contributions, specifically towards the alignment of simulated students and their applications for improving educational systems.

1.1 Background

To motivate the importance of simulated students, I first provide an overview of intelligent tutoring systems, which make up the primary digital setting for delivering learning benefits to real students and leverage simulated students. Then, I provide an overview of the models that have historically powered these systems, including cognitive models, student models, and simulated students. I discuss the strengths and limitations of existing approaches, and detail how this thesis builds on prior work.

1.1.1 Intelligent Tutoring Systems

Intelligent tutoring systems (ITSs) are digital learning environments where content dynamically adapts to each particular student. This adaptation can take the form of selecting which questions the student should solve next, recommending content to help the student learn, providing targeted feedback during problem solving, or engaging the student in a dialogue. This adaptation is largely powered by cognitive models and student models, as I will detail in the next subsection.

Many ITSs are built around a cycle of having students learn and practice new topics, where practice questions are drawn from large question banks, and students can watch videos or read explanations to reinforce their understanding. Popular ITSs using this paradigm include Khan Academy [109], Duolingo [236], and Aleks [162]. Other ITSs focus on guided problem solving, where students attempt to solve a problem step-by-step and are given targeted guidance when they make a mistake. Early systems include Andes [77], SQL-Tutor [166], and Carnegie Learning's Cognitive Tutor [12], which preceded the modern MATHia system [124]. Other systems allow students to engage with "teachable agents", such as Betty's Brain [128] and SimStudent [160]. These agents act like students that the real students needed to "teach" concepts to, allowing them to benefit from learning-by-teaching. Another popular form of ITS centers around tutoring dialogues, where students have turn-by-turn conversations with

an automated tutoring agent, allowing them to engage with a problem or concept in an open-ended way. Early automated dialogue tutors include AutoTutor [175] and Why2-Atlas [235]. While these systems can deliver effective and engaging instruction, they rely on effort-intensive hand-crafted rules and scenarios, and offer limited adaptability to open-ended student responses. Recently, LLMs have been used to create automated tutors, facilitating coverage on a wider variety of concepts, more nuanced processing of student responses, and greater adaptivity. LLM-based tutors have been integrated into existing ITSs, including Khan Academy's Khanmigo [110], Duolingo's Video Call [68], and Carnegie Learning's LiveHint [44]. In addition, existing LLM providers have begun integrating tutoring strategies into their models, including ChatGPT's study mode [181], Claude's learning mode [15], and LearnLM, which was later merged into Gemini [83]. LLMs have also been used to power teachable agent systems, like Ruffle & Riley [211], and are increasingly used to generate content in popular ITSs like Duolingo [90].

1.1.1.1 ITS Efficacy and Self-Regulated Learning

Many studies have shown that ITSs are highly effective for improving student learning outcomes and engagement [116, 263, 150, 33, 28, 115]. Additionally, early results examining student outcomes when using LLM-based tutors are promising, showing that pedagogically-aligned prompts can lead to higher test scores [183], and that hybrid human-LLM tutoring is particularly effective for improving learning outcomes [231, 241]. However, a common challenge is that while highly motivated students may benefit from a system, many other students may not engage with the system for enough time to experience learning benefits [254] or in an optimal way, for example, struggling to ask effective questions to LLM tutors [25]. Therefore, an important area of research is teaching students effective self-regulated learning (SRL) skills [247]. Various digital tools have been developed to help students develop

these skills, such as teaching help-seeking via meta-cognitive models [5], encouraging positive learning behaviors and emotions via adaptive feedback [168], and training curiosity and question-asking skills with LLM-generated cues [1]. Unsurprisingly, teacher interventions in the classroom are also highly effective for helping students use AI tools effectively [106].

1.1.2 Cognitive Modeling

ITSs centered around step-by-step problem-solving are largely powered by cognitive models. Cognitive models are structured representations of knowledge and rules, describing what kinds of knowledge are needed when problem-solving and how this knowledge can affect student actions. These models are based on theories of cognition, particularly Adaptive Control of Thought - Rational (ACT-R), where cognition is broken down into declarative knowledge, which is retrieved, and procedural knowledge, which is applied via granular "production rules" [11, 246]. Early ITSs centered on problem-solving, such as Cognitive Tutor [12] and Andes [77], leveraged cognitive models based on ACT-R to power their adaptive tutors (broadly referred to as "cognitive tutors"), specifically model-tracing tutors (MTTs) [12]. These models operated on large hand-crafted sets of production rules, which were encoded as abstract if-then logical programs, and attempted to match student actions to rules. This allowed the model to detect when a student took an incorrect step in problem solving and what particular error they made, enabling the tutor to provide targeted feedback to the student. Example-tracing tutors (ETTs) [7] offer similar functionality, but are built on "behavior graphs" that student actions are matched to, rather than on production rules. Alternatively, constraint-based modeling (CBM) [177], used in SQL-Tutor [166], specifies sets of contextual "constraints" that students need to satisfy when solving a problem correctly, which are easier to design but offer less targeted feedback since they are not grounded in step-by-step solutions.

While cognitive tutors offer high interpretability, they are highly effort intensive to create since each production rule or behavior graph has to be designed by hand. To partly address this challenge, Carnegie Learning introduced Cognitive Tutor Authoring Tools (CTAT), which enables non-programmers to design production rules or behavior graphs for ITSs [6]. Later, the SimStudent system enabled automatic discovery of production rules from worked examples of problem solving by inferring logical rules that lead to correct solutions, thereby reducing the need for hand-crafting rules [133].

Simulated learners (SLs), such as SimStudent [160] and Apprentice Learner [153], are adaptive cognitive models, which simulate student learning in problem-solving settings by updating internal logical rules when given worked examples and feedback. SLs have been shown to follow similar "learning curves" to real students, where error rates gradually decrease with practice [153]. Researchers have therefore used SLs to evaluate the effectiveness of tutoring strategies, using SL outcomes as proxies for real student outcomes [244]. However, SLs are not fit to real student data, and therefore often fail to realistically represent student learning [245]. Additionally, SLs still require significant manual effort to implement, since the space of available rules and the effects of possible actions need to be defined.

While cognitive models have been highly effective in ITSs centered around problem-solving, they are less common in other systems since extensive effort is required to support new problem types. Modern systems tend to prefer machine learning-based student models due to their greater adaptability and lower manual effort. The move from cognitive models to student models reflects a larger trend in AI from symbolic AI to machine learning. In the remainder of this thesis, I focus on machine learning methods for student modeling and simulation, primarily using LLMs. However, exploring the fusion of cognitive models and LLM-based student models could be a promising line of future work, in particular for greater interpretability in learned models.

1.1.3 Student Modeling

A wide range of ITSs are largely powered by student models. Student models are representations of a student's internal cognitive *state*, generally representing their knowledge or ability across various concepts. A student's "knowledge state" is often represented as latent mastery of granular knowledge components (KCs) or latent ability values, representing how well the student understands particular concepts and how likely they are to answer questions related to those concepts correctly. These values are stored numerically and are generally interpretable. In addition to representing student knowledge, the models also represent learning dynamics related to a particular setting, such as the learning impact or difficulty of particular questions or content. Rather than crafted by human experts, as is typically the case with cognitive models, student models are defined by parameters that are fit by observing student responses to questions; typically, student responses are treated as correct or incorrect, where a correct response is treated as evidence of a student having knowledge on associated concepts, and vice versa for incorrect responses. There are many different ways of parameterizing and fitting student models, often informed by the learning environment they are deployed in.

1.1.3.1 Item Response Theory

Item response theory (IRT) [199] models properties of students and questions in a *static* setting, where student knowledge is assumed to not change. IRT models students using latent *abilities*, and models questions using latent *difficulties* and often other properties. There are many variants of IRT, such as ones that model question "discrimination" [35], partial credit scoring [169], and multi-dimensional latent abilities [47]. Model parameters are typically learned using expectation-maximization (EM), leveraging prior estimates of student knowledge distributions.

IRT models are commonly used for computerized adaptive testing (CAT), including in popular standardized test such as the GRE and GMAT, where the goal is to approximate a student's ability by having them solve as few questions as possible, enabling fast and efficient assessment [48]. One strategy for question selection in CAT is maximizing Fisher information, where the difficulty of the selected question should closely match the current student ability estimate [48].

1.1.3.2 Knowledge Tracing

Knowledge tracing (KT) [56] models students and questions in a *dynamic* setting, where student knowledge is assumed to change over time as they practice questions. KT models students using a latent *mastery* for each concept in the system, and models questions using properties of learning dynamics, such as the probability that a student will learn a concept by answering a question. The classic approach for KT, Bayesian KT (BKT) [56], uses a hidden Markov model to connect observed student responses to latent knowledge states, and is learned using EM. Newer variants of BKT additionally model the probabilities of students "slipping" or "guessing" when answering questions for greater interpretability and accuracy [22].

As deep learning became more ubiquitous, neural network-based architectures were applied to KT. While the student learning dynamics in deep learning methods are not easily interpretable, as they are in BKT, they offer greater predictive accuracy by modeling complex interactions between KCs learned by training on large student response datasets; popular datasets for KT training include ones from the ASSISTments system [73]. Such architectures include deep KT (DKT) which leverages LSTMs [194] and context-aware attentive KT (AKT) which leverages attention networks [78]. More recently, architectures like open-ended KT (OKT) [139] combined generative language models with KT architectures, making it possible to model entire textual student responses rather than just the binary correctness of student responses. Another

advantage of using language models for KT is that they can predict student responses to questions that are not in the training data since they are simply represented as text, which earlier KT methods were not able to do since they were represented by learned embeddings. In this thesis, I build on this line of work in Chapter 3, where I detail how to perform KT in the setting of tutoring dialogues and introduce an LLM-based architecture for KT.

1.1.3.3 Alternative Student Model Architectures

Several alternatives to IRT and KT have been proposed in the past. In particular, learning factors analysis (LFA) [46] and performance factors analysis (PFA) [190] make predictions based on student practice counts and success/failure counts, respectively. Another family of methods jointly model student knowledge and question-skill associations, and can therefore be seen as unified cognitive and student models. Such methods include the Q-matrix method [24], Topical HMM [82], sparse factor analysis (SPARFA) [123], and SPARFA-Trace [122].

1.1.3.4 Behavioral Student Modeling

While the methods mentioned so far have focused on modeling knowledge and predicting if students will answer questions correctly, there are many more aspects of student behavior that can be modeled. These aspects play an important role in modeling students wholistically, and are critical to understanding student outcomes. For instance, prior works have modeled motivation, metacognition and SRL behaviors [247, 167, 271], affect and emotion [23, 249], engagement [10, 16], help-seeking [8], response times [192, 27], cheating or "gaming the system" behaviors [21], and likelihood of dropout from online courses [114]. In Chapter 5 of this thesis, I incorporate both cognitive and behavioral aspects into LLM-based student modeling in dialogues.

1.1.3.5 Applications of Student Models

One of the most popular applications of student models is for recommending exercises, hints, or learning content in ITSs [2]. Various heuristic policies have been proposed, such as selecting tutoring actions that maximize short-term "utility" (incorporating estimated knowledge and affect) [171], and selecting practice items that aim to maximize short-term recall of concepts [217]. Some ITSs also leverage IRT by recommending practice problems that are an appropriate difficulty for a student based on their current ability estimate, leading to improved engagement [33]. In addition to heuristic policies, optimal recommendation policies can be learned with the objective of increasing student knowledge or engagement. A popular approach is to form a reward function using a student model, where actions that lead to increased student knowledge are rewarded, and then train a policy using RL [26, 41]; specialized methods in this paradigm have leveraged multi-arm bandits [121, 141], multi-objective rewards [95], and Q-learning for selecting questions with optimal difficulty [267]. I build on this RL paradigm in Chapter 4 of this thesis, where I leverage an LLM-based KT model to form rewards for training an LLM-based dialogue tutoring agent.

1.1.4 Student Simulation

Student simulation is the process by which a student model acts out possible scenarios in an educational setting. Compared to direct predictions with student models, which estimate a student's *current* state, simulated students can estimate a student's possible range of *future* states. We can then observe these anticipated future states to make informed decisions on how to best serve students. In general, student simulations are broadly used for improving the outcome-centric alignment of both human educators and educational AI systems.

Recently, a significant body of work has used LLMs to simulate students for the purpose of training and evaluation across educational settings. In particular,

LLM-based simulated student dialogue agents have been used to create training environments for human tutors [186, 157, 43] or to create training data for LLM-based tutors [64, 238, 137, 223]. It is also common to use them for question evaluation, where a group simulated students respond to questions, and these responses are then analyzed to evaluate question difficulty and quality [142, 88, 148, 172]. Other works have simulated errors or misconceptions that students might have when problem solving, which can leveraged to generate targeted distractor options for multiple-choice questions [76, 74]. Methods for creating these simulated students often involve prompting pre-trained LLMs to act like students by following descriptive "personas" in the prompt, which can include a range of attributes covering knowledge, misconceptions, demographics, and personality traits [152, 137, 237, 238, 88, 143, 172]. Other methods fine-tune LLMs, either by training on student-written text [139, 66, 193, 43] or training the LLMs to be more prone to errors or misconceptions [221, 76, 220].

However, a key limitation in the aforementioned methods is that the LLMs are not trained to reflect the cognitive and behavioral properties of the students they are meant to be simulating; methods relying on pre-trained LLMs suffer from well-known issues in realistically representing linguistic, behavioral, and cognitive properties of students [158, 193], methods fine-tuning on student-written text do not explicitly align with knowledge (with the exception of OKT-based methods), and misconception-based methods do not leverage real student data so also do not follow realistic student learning patterns. This limitation stands in contrast to student models, such as IRT and KT, that explicitly train to align with real student knowledge.

The primary contributions of this thesis work toward resolving this limitation of LLM-based simulated students. In Chapter 2, I introduce a method for training LLM-based simulated students to steer their outputs according IRT-based abilities leveraging real student data; this method is the first to bridge the gap between IRT student models and LLM student simulations. In Chapter 4, I leverage a fine-tuned

student model for training an LLM tutor, rather than relying on pre-trained LLM simulations to provide a training signal. Finally, in Chapter 5, I explicitly measure the behavioral, cognitive, and linguistic properties underlying student utterances in dialogues, and introduce a method for aligning LLM outputs with real student responses by leveraging these measurements.

1.2 Thesis Overview

In the following chapters, I detail my work towards creating realistic simulated students and applying them to improve educational applications. In each chapter, I introduce the respective setting, review relevant literature, detail proposed methodologies, and verify them with experimental results.

In Chapter 2, I study the setting of open-ended question answering, where I propose a method for using simulated students to estimate the difficulty of assessment questions. Using simulated students for question difficulty prediction has various benefits compared to using real students or regression-based AI methods. However, a key challenge is that LLMs do not reliably generate student-like responses, making resulting difficulty estimates poorly calibrated. To address this, I propose SMART, a novel RL pipeline for training LLMs to generate student-like responses that faithfully reflect an expected student ability level relative to a question's difficulty. The key contribution is a reward function derived from item response theory, thereby injecting calibrated ability and difficulty estimates into the LLM. I find that SMART significantly outperforms prior state-of-the-art approaches for question difficulty prediction, and is much more aligned with real student behavior compared to other simulated student methods.

In Chapter 3, I study the tutoring dialogue setting, with the goal of modeling how student knowledge evolves during a conversation with a tutor, enabling reliable automated student assessment in dialogues. To do this, I apply the well-established

knowledge tracing (KT) framework to dialogues by viewing them as sequences of formative assessments. I introduce LLM-based labeling techniques to prepare dialogues for KT, and the labels are rated highly by human experts. Additionally, I introduce LLMKT, a novel LLM-based student model that leverages dialogue context to estimate student knowledge, and find that it significantly outperforms existing KT methods.

In Chapter 4, I study how to improve tutoring in dialogues by aligning LLM-based tutors with student outcomes leveraging student models. Doing so will ensure that LLMs intended for tutoring are designed with student learning as a core goal. I achieve this by training LLM tutors with a novel RL pipeline that encourages both higher pedagogical quality and improved student outcomes. Specifically, tutor dialogue turns are assigned rewards using using 1) an LLM-evaluated rubric determining how well the turn adheres to pedagogical principles, and 2) an estimate of student mastery on relevant knowledge concepts, evaluated using the LLMKT student model. Using automated metrics, I find that this pipeline results in a tutor that exceeds the pedagogical quality and estimated student outcomes of similar sized tutor models, and matches the performance of large proprietary LLMs. When evaluated using humans, I find that this pipeline exceeds the performance of large proprietary LLMs. I also uncover emergent tutoring strategies that arise as a result of the training, showing that student model integration is a powerful tool for driving LLM tutor behavior.

In Chapter 5, I study student simulation in tutoring dialogues, where I focus on systematically *evaluating* the realism of LLM-based simulations. Since simulated students are increasingly used in evaluating and training both human and LLM tutors, it is important to ensure that these simulations resemble real student behavior and learning; otherwise, the tutors may be poorly calibrated and unable to meet students' needs. I derive six high level dimensions from learning sciences research to ground evaluations in: dialogue acts, correctness, errors, knowledge acquisition, linguistics, and inducing tutor responses. For each dimension, I develop automated

reference-based metrics to evaluate simulated student responses on, establishing a data-driven and theory-grounded way to measure alignment with real students. I also show how to train a simulated student model with an RL pipeline by using these metrics to form a reward. I evaluate a wide variety of prompting and fine-tuning simulated student methods using the metrics, and find that supervised fine-tuning (SFT) improves realism significantly over prompting much larger pre-trained LLMs, and that RL improves realism slightly over SFT. I also find that the metrics are highly aligned with human expert evaluations, showing their reliability for evaluating realism. I conclude that even the best methods do not perform well enough to be fully reliable for tutor training, motivating future work to further improve simulated student alignment.

Finally, in Chapter 6, I conclude by summarizing the findings of this thesis and laying out directions for future work. Specifically, I detail how simulated students can be used to improve educational AI systems through personalized experiences, integration with creative learning settings, and agentic assistants to help teachers deliver better outcomes for students. For each of these, I also detail advancements that need to be made in simulated students, particularly related to long-term modeling, multi-modality, and more diverse representation of student populations.

1.3 Summary of Relevant Publications

The content of Chapter 2 appears in:

> [206] Scarlatos, Alexander, Fernandez, Nigel, Ormerod, Christopher, Lottridge, Susan, and Lan, Andrew. SMART: Simulated students aligned with item response theory for question difficulty prediction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (Suzhou, China, Nov. 2025), Association for Computational Linguistics, pp. 25082–25105

The content of Chapter 3 appears in:

> [205] Scarlatos, Alexander, Baker, Ryan S., and Lan, Andrew. Exploring knowledge tracing in tutor-student dialogues using llms. In *Proceedings of the 15th International Learning Analytics and Knowledge Conference* (New York, NY, USA, 2025), LAK '25, Association for Computing Machinery, p. 249–259

The content of Chapter 4 appears in:

> [208] Scarlatos, Alexander, Liu, Naiming, Lee, Jaewook, Baraniuk, Richard, and Lan, Andrew. Training llm-based tutors to improve student learning outcomes in dialogues. In *Artificial Intelligence in Education* (Cham, 2025), Springer Nature Switzerland, pp. 251–266

The content of Chapter 5 appears in:

> [207] Scarlatos, Alexander, Lee, Jaewook, Woodhead, Simon, and Lan, Andrew. Simulated students in tutoring dialogues: Substance or illusion?, 2026

CHAPTER 2

SMART: SIMULATED STUDENTS ALIGNED WITH ITEM RESPONSE THEORY FOR QUESTION DIFFICULTY PREDICTION

2.1 Introduction

Assessing student knowledge and skill levels, an important aspect of education, requires carefully designed assessment items (i.e., questions) with high quality and validity. According to the Standards for Educational and Psychological Testing [70], various factors play a role in the quality and validity of items, with a crucial factor being their difficulty, i.e., how challenging they are to real students. Item difficulties also play a critical role in learning platforms, enabling personalized learning through student-tailored item recommendation [50, 232], item generation [99, 118, 215, 17], and curriculum design [164, 268] to maximize learning outcomes.

Since new items are constantly developed and used in practice, estimating their difficulty before assigning them to students is crucial. Traditionally, estimating item difficulties can be costly and time-consuming [9]: we need (possibly a large number of) real students to respond to them, and then fit an item response theory (IRT) model on their responses to get item difficulty estimates. In contrast, recent work has estimated item difficulties from their (mostly textual) content [230, 81, 29, 94], with some methods leveraging large language models (LLMs) [88, 67]. This approach works well for many items, especially multiple-choice questions (MCQs). Among them, an interesting approach is to use *simulated students* [88, 75, 154] to generate responses to new items at scale, to estimate their difficulties. Initial work in this direction uses

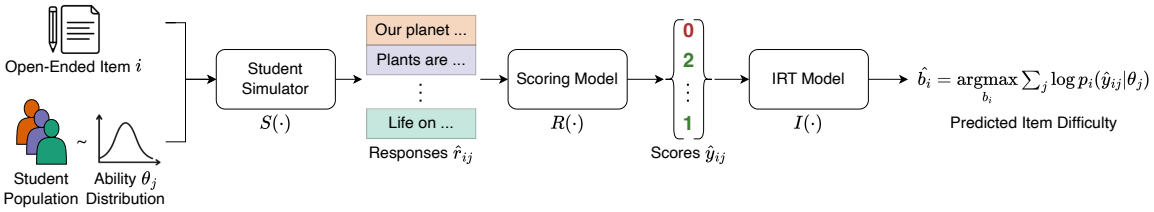


Figure 2.1: Our three-stage pipeline for item difficulty prediction: simulate open-ended responses for students with different abilities, score them, and fit an IRT model on the scores to obtain difficulty estimates.

simulated students to model student responses to MCQs, followed by aggregating their predicted responses and obtaining item difficulty estimates.

Among different item types, open-ended items require students to write short answers, and are common in many domains including math, programming, and particularly in reading comprehension assessment; see Table 2.3 for an example. Since these items require a student to construct an entire answer, they may reveal deeper insights into student knowledge [36, 72, 218, 13] than other item types, such as MCQs. However, estimating the difficulties of open-ended items can be more challenging, since scoring open-ended responses accurately at scale is not as easy as for MCQs. Moreover, there is considerable diversity in terms of both response content and style among real students, which means that it can be difficult to simulate student responses to these items [222].

2.1.1 Contributions

In this paper, we present SMART (**Sim**ulated Students **A**ligned with Item **R**esponse **T**heory), a pipeline for training LLMs as simulated students that generate open-ended responses, aligning them with an instructed ability level and item difficulty.

First, SMART leverages generative LLMs as the student simulator to synthesize *a distribution of* student responses to open-ended items. These responses provide valuable *interpretability* in the difficulty estimation process. We use direct preference

optimization (DPO) to train our student simulator, using a novel method to form preference pairs. We *prefer student responses with higher likelihood under an IRT model*, thus aligning the generated responses with both student ability and item difficulty. Second, we use an LLM-based scoring model to score the generated responses. Finally, we train an IRT model on the scored student responses to obtain item difficulty estimates.

We conduct extensive quantitative experiments on two real-world datasets, one with 49 short-answer reading comprehension items and another with 50 Java coding items. We find that SMART, by simulating IRT-aligned responses to unseen items from a population of students with different abilities, outperforms state-of-the-art item difficulty prediction methods, even in the low-resource setting with few training items. We analyze the simulated student responses and find that SMART responses correlate strongly with their instructed ability, and that the feature distributions of its responses match closely with the ground-truth. We also qualitatively compare simulated student responses across methods, perform a failure pattern analysis, and suggest directions for future work.

2.2 Related Work

2.2.1 Direct Item Difficulty Prediction

Item difficulty prediction has been widely studied [9], particularly in the context of multiple-choice questions (MCQs) [255, 75, 201]. It is common to train regression models using handcrafted features [40, 230, 105, 81], although finetuning language models such as BERT has proven to be highly effective [81, 275, 147, 30]. Custom architectures, such as TACNN, have also been proposed for this task [94].

2.2.2 Simulation-Based Difficulty Prediction

An increasing body of research uses simulated students to estimate item difficulty. They follow a similar pipeline to our work, where they overgenerate a large set of responses using AI models, and then fit an IRT model to these responses to recover the item difficulty. Such methods have prompted ensembles of pre-trained language models to answer the items [142, 233], finetuned language models to predict student responses given prior responses [264], and trained LSTMs with varying proportions of incorrect labels [120]. [88] experiment with LLM ensembles, finetuning, and persona-based prompting, finding that the most effective method depends on the dataset.

Our approach builds on this line of work but introduces several key innovations. First, we explicitly train simulated students to follow a prompted ability level, allowing the simulated students to align with the underlying IRT distribution. Second, we focus on open-ended responses, which are not handled by most finetuned or feature-based methods. Third, our system is designed to operate effectively on small item sets, where feature-based and BERT-based methods typically underperform.

2.3 Background: Item Response Theory

Item response theory (IRT) [199] is a framework for jointly estimating scalar-valued abilities of students and parameters of "items" (or questions), mainly difficulty and discrimination, from student responses to items. In the simplest case, each student is represented by a single scalar ability $\theta_j \in \mathbb{R}$, each item is represented by a single scalar difficulty $b_i \in \mathbb{R}$, and each score y_{ij} is binary, i.e, 0 (incorrect) or 1 (correct). This simple model is known as the 1-parameter logistic (1PL) model [199], and uses the following formulation:

$$P_i^{\text{1PL}}(y_{ij}|\theta_j) = \sigma(\theta_j - b_i). \tag{2.1}$$

In practice, θ_j and b_i are assumed to be drawn from a standard normal, with implementations using expectation maximization or variational inference [120] to enforce standard normal priors on these variables.

In our setting, we study *short-answer* questions where the response is open-ended. We assume that students can receive partial credit on responses, leading to ordinal or continuous scores. For ordinal response data, we use the generalized partial credit model (GPCM) [169], which adds a "step" parameter for each of the C total score categories:

$$P_i^{\text{GPCM}}(y_{ij}|\theta_j) = \frac{e^{\sum_{y=0}^{y_{ij}} a_i(\theta_j - b_i + d_{iy})}}{\sum_{c=0}^{C-1} e^{\sum_{y=0}^{c} a_i(\theta_j - b_i + d_{iy})}}, \tag{2.2}$$

where the response $y_{ij} \in \{0, \ldots, C-1\}$, d_{iy} is the step parameter for item i for score y, and a_i is the discrimination parameter for item i. GPCM smoothly adjusts the probability of different ordinal scores according to student ability and item difficulty, with step parameters determining where the probability mass shifts from one score to the next.

We also handle continuous scores, i.e., $y_{ij} \in [0, 1]$, where the student's response lies on a spectrum between incorrect and correct. In this setting, we model correctness probability using a continuous Bernoulli distribution [146]:

$$\begin{aligned} P_i^{\text{Bern.}}(y_{ij}|\theta_j) = & P_i^{\text{1PL}}(Y = 1|\theta_j)^{y_{ij}} \cdot \\ & P_i^{\text{1PL}}(Y = 0|\theta_j)^{(1-y_{ij})}. \end{aligned} \tag{2.3}$$

2.4 Problem Formulation

We assume that we have access to a real-world dataset of human-scored student responses to open-ended short-answer items. Each item, indexed by i, is associated with 1) a short question text q_i, 2) (optional) meta data m_i, e.g., a passage text

for reading comprehension items, 3) (optional) a scoring rubric w_i, and 4) a set of human-scored student responses denoted by $D_i = \{r_{ij}, y_{ij}\}$. An IRT model is fit to this data to get ground-truth estimates of each student ability θ_j and item difficulty b_i. Our goal is to predict the item difficulty parameters of previously *unseen* test items before students respond to them.

2.5 Methodology

Our approach, SMART, illustrated in Figure 2.1, uses a novel, three-stage pipeline, which simulates student responses to short-answer, open-ended items, to help estimate their difficulty. First, SMART leverages human-scored responses by real students to *open-ended items* to learn a student response simulator, $S(\cdot)$, parameterized by a generative LLM. The simulator $S(\cdot)$ generates a *distribution of student responses* to unseen open-ended items. Second, an LLM-based scoring model, $R(\cdot)$, scores these generated responses. Third, an IRT model, $I(\cdot)$, is trained on the scored student responses to estimate item difficulties.

2.5.1 LLM-based Student Response Simulation

We first detail our LLM finetuning method for student simulation, resulting in a model we denote as $S(\cdot)$. We combine the question text q_i, the meta data text m_i, and the student ability θ_j in our input prompt, and train the LLM using supervised finetuning (SFT) to maximize the likelihood of the ground-truth student response r_{ij} as output. We convert θ_j to a string after rounding to 4 decimal places, which we find works better than binning values into descriptive buckets; we leave an exploration of more advanced methods for numeric inputs to LLMs, such as weighting model parameters [239], for future work.

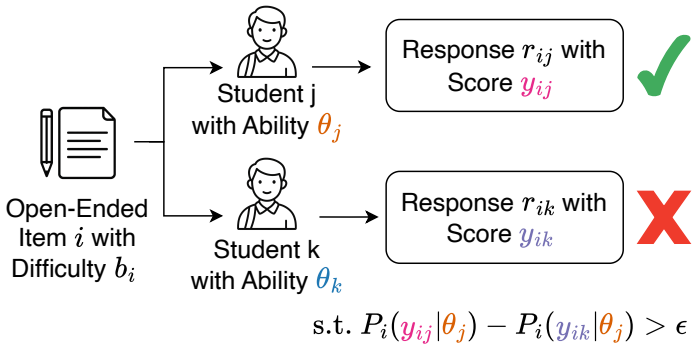


Figure 2.2: Our novel preference pair creation method through comparing response likelihood under IRT.

2.5.2 SMART: Ability Alignment in Response Simulation via Preference Optimization

We now motivate and detail SMART, our novel method aligning simulated student responses with their given ability and the expected difficulty of the given item. Even though the LLM-based student simulator $S(\cdot)$ is finetuned to maximize the likelihood of the textual content of student responses, we observe that the responses it generates do not always align well with the prompted ability value, likely because the SFT objective treats all output sequences equally without any special attention paid to the prompted ability.

To address this limitation, we propose the following idea: to incorporate information on the IRT likelihoods of student responses, in addition to their textual content. The intuition is that generating responses with high likelihoods under the IRT model should lead to better alignment between the IRT parameters and response content, which should benefit the downstream task of item difficulty prediction. To achieve this alignment, we use DPO [197] to *prefer responses with scores having a higher IRT likelihood* with the input student ability and item difficulty.

2.5.2.1 Preference Pair Creation

Our novel preference pair creation method is illustrated in Figure 2.2. Given an item's question text q_i, meta data m_i, and a student's ability θ_j, we compare two possible responses: one with a higher IRT likelihood, r_{ij}^{w}, which is preferred to another with a lower IRT likelihood r_{ij}^{ℓ}, thereby forming a preference pair. We find such preference pairs among the human-scored real student responses. For each real student response r_{ij}, we construct a set of possible *negative candidates*, $\mathcal{R}_{ij}$, which are other responses to item i that a student with ability θ_j would be less likely to write, according to the IRT model. Formally:

$$\mathcal{R}_{ij} = \{r_{ik} : P_i(y_{ij}|\theta_j) - P_i(y_{ik}|\theta_j) > \epsilon\}\,, \tag{2.4}$$

where P_i is either P_i^{GPCM} (Eq. 2.2) if the scores are ordinal or $P_i^{\text{Bern.}}$ (Eq. 2.3) if the scores are continuous. The hyperparameter $\epsilon \in [0, 1)$ sets the threshold for how much *more* likely a response must be than another to form a preference pair, which is needed to reduce noisy preference pairs. In summary, for some student that responds with r_{ij}, for every other possible response, we ask the question: *how likely is it for the same student to write response* r_{ik}? We randomly select m negative candidates from $\mathcal{R}_{ij}$ for training, creating m preference pairs where the ground-truth response, $r_{ij}^{w} = r_{ij}$, is preferred over each sampled response, $r_{ij}^{\ell} = r_{ik}$. We note that $\mathcal{R}_{ij}$ could be empty in some cases, for instance, if the ground-truth student response is unlikely for the corresponding θ_j.

2.5.3 Scoring Model for Open-ended Responses

Given a distribution of varied generated student responses to items, we need to score them before fitting an IRT model to estimate item difficulties. This scoring task is challenging since it requires scoring synthetic responses to items not seen during training. We parameterize our scoring model $R(\cdot)$ by finetuning a generative LLM,

given its ability to handle long-context inputs, to discern complex (partial) scoring criteria, optionally from a rubric, and to generalize to scoring responses to new items in the test set. We combine the question text q_i, the meta data m_i, the scoring rubric w_i, and the student response text r_{ij} into a prompt x_{ij}, and instruct the LLM to only output a single integer for the score label $y_{ij} \in \{0, \ldots, C-1\}$. For ordinal scores, we produce an estimate score $\hat{y}_{ij}$ via greedy decoding from $R(\cdot|x_{ij})$. For continuous scores, following [205], we smoothly interpolate between the token logits corresponding to fully correct and incorrect scores:

$$\hat{y}_{ij} = \frac{e^{R(1|x_{ij})}}{e^{R(0|x_{ij})} + e^{R(1|x_{ij})}}, \tag{2.5}$$

where $R(y|x_{ij})$ is the output logit value for token y returned by $R(\cdot|x_{ij})$, and we train using binary cross-entropy loss.

2.5.4 Simulation-Based Difficulty Prediction

We now detail how to use our ability-controlled student simulator and automated scoring model to predict item difficulties. First, we create a population of n simulated students such that their abilities match the training distribution: we create a 50-bucket histogram of abilities in the train set, proportionally draw n samples from the resulting distribution, and convert each sample to a random uniform value in the corresponding bin. Second, we perform the student simulation: for each drawn ability and item in the test set, we prompt the simulated student model $S(\cdot)$ to generate a corresponding response using nucleus sampling. Third, we score each generated response using the automated scoring model $R(\cdot)$. Finally, we re-fit the IRT model $I(\cdot)$ using the new set of scored responses. For each generated response, we assign a student ID by rounding the ability to the nearest decimal place, which we find to be crucial in preventing overfitting by reducing the number of ability parameters significantly. We

take the difficulty estimates under this IRT model for the new, unseen items as our final predictions.

2.6 Experimental Evaluation

We now detail experiments that we conduct to validate our approach on real student response datasets.

2.6.1 Datasets

We conduct our experiments on two real-world open-ended student response datasets, each containing around 50 items and thousands of student responses.

2.6.1.1 Smarter Balanced

We first use a private real-world dataset, provided to us by Smarter Balanced[1]. The dataset contains student responses to open-ended short answer items administered to students in the United States, containing around 85K responses to 49 items from around 63K students. These open-ended items assess reading comprehension skills including argumentation, explanation, and narration, at the grade 6 level with the average length of student responses being 92 tokens. Several questions include long reading passages, up to $4,000$ tokens in length, leading to increased memory usage and training time; therefore, we prompt `Llama-3.1-70B-Instruct`, loaded with NF4 quantization, to summarize these passages while retaining the main elements and original writing style. Each response is manually scored by a trained human scorer, using a rubric with detailed scoring criteria, with scores in the set $\{0, 1, 2\}$. The meta data for each item includes a passage that the student must read before answering the question. Our trained 3B parameter scoring model achieves a quadratic weighted kappa (QWK) of 0.616 on this dataset, indicating moderate-to-high agreement with

[1] `https://smarterbalanced.org/`

ground-truth scores, which is competitive with unseen item open-ended response scoring on other datasets[2].

2.6.1.2 CodeWorkout

We additionally use the publicly available CodeWorkout dataset [60], originally used for the 2nd CSEDM Data Challenge [184]. The dataset, commonly used for student modeling in open-ended programming tasks [139, 66], contains 50 Java coding problems in a first year college programming course, with student solutions across 2 semesters. Students can attempt a problem multiple times after receiving compiler feedback, so we only consider the first attempt for a problem for each student, resulting in 10,834 responses from 246 students. Each response is scored with a continuous value between 0 and 1, representing the portion of test cases that passed; the test cases themselves are not made available. To ensure that models understand how to correctly solve the problems, we include 2 LLM-generated solutions in each item's meta data, using the method described in [66]. Our trained 3B parameter scoring model achieves a Pearson's correlation coefficient (PCC) of 0.849 on this dataset, indicating high agreement with ground-truth scores.

We split each dataset into 5 cross-validation folds, with each fold split 60/20/20 item-wise for train/validation/test, respectively. We create these splits such that 1) each item appears in train, validation, and test at least once, and 2) the distribution of item difficulties should be roughly the same across train, validation, and test over all folds. We achieve these requirements using the following algorithm. First, we sort all items according to their difficulty. Second, we partition all items into 10 difficulty buckets of up to 5 items each. Third, we sequentially construct a list of items by adding one random item at a time from each bucket in a round-robin fashion; the

[2]Best QWK on unseen items in NAEP Reading Automated Scoring Challenge was 0.528: `https://github.com/NAEP-AS-Challenge/reading-prediction/blob/main/results.md`

resulting list is striped by difficulty bucket. Finally, we evenly rotate this list 5 times to form the 5 cross-validation folds, partitioning each rotation at 60% and 80% to create a train/validation/test split.

We fit IRT models (P^{GPCM} for Smarter Balanced and $P^{\text{Bern.}}$ for CodeWorkout) on each fold to estimate ground-truth student ability and item parameters. For each fold, to ensure no information from unseen items is leaked at train time, we fit three separate IRT models, one on train data, one on train plus validation data, and one on all data. We use the estimated parameters from the three models to train our student simulator, and to perform validation and test-time evaluation, respectively.

2.6.2 Metrics

2.6.2.1 Item Difficulty Prediction

Following prior work in item difficulty prediction [94, 88], we use the widely adopted *Pearson Correlation Coefficient* (**PCC**) to measure the linear correlation between predicted and ground-truth item difficulties, *Spearman Rank Correlation Coefficient* (**SCC**) to measure the correlation between ranks of predicted and ground-truth item difficulties, and *Root Mean Squared Error* (**RMSE**) for a precision comparison between predicted and ground-truth item difficulties.

For all methods, including baselines, we transform predicted difficulties to have the same mean and standard deviation as the ground-truth difficulties on the train set; doing so improves RMSE when predicted difficulties are correlated but distributionally shifted from the ground-truth. We perform the transformation using the following formula:

$$\hat{b}_i \leftarrow \frac{\sigma_2}{\sigma_1}(\hat{b}_i - \mu_1) + \mu_2, \tag{2.6}$$

where $\hat{b}_i$ is the predicted difficulty for item i, μ_1 and σ_1 are the mean and standard deviation of the predicted item difficulties, respectively, and μ_2 and σ_2 are the mean and standard deviation of the item difficulties in the train set, respectively.

2.6.2.2 Student Simulation

We additionally propose metrics to investigate how faithful the generated responses are to real student ones. Since we model students at a population level, reference-based metrics such as ROUGE-L [135] and BERTScore [272] are not suitable. Therefore, following prior work in population simulation [39, 260], we report distribution-level metrics comparing the generated and ground-truth response distributions in a latent embedding space. We first extract embeddings for all responses using a sentence encoder [200], specifically `multi-qa-mpnet-base-dot-v1`. We then compute the *Fréchet Inception Distance* (**FID**) [91], which is common for evaluating generated images but has recently been adapted to text [262, 39], and **MAUVE** [195], which measures the similarity of cluster distributions in the embedding space, to evaluate the alignment between the predicted and ground-truth student response distributions. For computing MAUVE, we use the official implementation[3]. For computing FID, we adapt the implementation from [38]. Following [39], we also measure the similarity in diversity distribution by computing **Div. KL**, the KL divergence between the histograms of pairwise response cosine similarity across all ground-truth responses and across all simulated responses. Finally, we compute **θ-Align**, the Spearman correlation between input student ability and the predicted score of the corresponding generated response, measuring how well simulated students align with their prompted abilities.

[3] `https://github.com/krishnap25/mauve`

2.6.3 Baselines

We compare our approach, SMART, to state-of-the-art item difficulty prediction approaches as well as to several strong novel baselines adapted to our setting. These approaches fall into two categories: predicting difficulties 1) directly from item text, and 2) via student simulation.

2.6.3.1 Direct Difficulty Prediction Baselines

Following prior work [9, 29, 32, 105, 174], we adopt a strong finetuning baseline, **Diff SFT**, which finetunes a recent and capable encoder-based language model, specifically ModernBERT-base [242]. We use a regression setup, minimizing mean squared error loss, to directly predict item difficulties given the item's question text and meta data. We also adopt a **Random Forest** baseline, which is common for difficulty prediction [9, 81], where we encode the item's question text and meta data using the `multi-qa-mpnet-base-dot-v1` SBERT model [200], and use the resulting embeddings as input features to fit a random forest regressor to item difficulty. We use `multi-qa-mpnet-base-dot-v1` for our SBERT model since it is the highest performing model with a 512 token limit, according to the sentence transformers model page[4]. We also found that it worked better than `all-distilroberta-v1` in preliminary experiments. We train our random forest baseline using the RandomForestRegressor class from the sklearn[5] library. We additionally restrict max_features to the size of train set to avoid overfitting, which we found improves performance.

Since our set of items is small, we also introduce several baselines that do not require training. We use a simple yet strong baseline, **kNN Mean**, that retrieves the k items in the train set with the highest SBERT embedding cosine similarity, and

[4] `https://sbert.net/docs/sentence_transformer/pretrained_models.html#original-models`

[5] `https://scikit-learn.org/`

takes their mean difficulty as the predicted test item difficulty. Similar methods have been used in automated scoring [34, 80]. We search over $k \in [1, 5]$ and select the best performing value on the validation set, selecting $k = 1$ for Smarter Balanced and $k = 5$ for CodeWorkout. Finally, we adopt two LLM prompting baselines leveraging in-context learning (ICL) [37], **ICL Random** and **ICL kNN**, where we prompt `Llama-3.1-70B-Instruct` with k in-context examples of items retrieved from the train set at random or using k-Nearest Neighbors (kNN) [74], respectively. We ask the model to first reason about the question's difficulty in a chain-of-thought manner before generating a numeric difficulty prediction. For kNN retrieval, we use the same SBERT model as above, and perform a similar hyperparameter search, selecting $k = 4$ for Smarter Balanced and $k = 2$ for CodeWorkout.

2.6.3.2 Student Simulation-based Difficulty Prediction Baselines

We compare SMART to other student simulation-based approaches for item difficulty prediction. For a fair comparison, we use the same base model and prompt template as SMART for the following methods, allowing these baselines to also serve as ablations. First, we employ **Response Gen ZS**, where we prompt an LLM in a zero-shot (ZS) manner to generate a student response given the item and ability. Since the model has not been trained on the distribution of numeric student abilities, we convert abilities to corresponding labels from 10 descriptive buckets, ranging from "minimal" to "mastery", with descriptions obtained by prompting Claude 3.5 Sonnet [14]. We first normalize all train abilities to $[-3, 3]$ before sampling, and then map abilities to 10 labeled buckets with the following ranges: "Minimal": $\theta < -2.5$, "Emerging": $\theta \in [-2.5, -2)$, "Developing": $\theta \in [-2, -1.5)$, "Approaching Proficiency": $\theta \in [-1.5, -0.8)$, "Proficient": $\theta \in [-0.8, 0)$, "Competent": $\theta \in [0, 0.8)$, "Skilled": $\theta \in [0.8, 1.5)$, "Advanced": $\theta \in [1.5, 2.0)$, "Exceptional": $\theta \in [2, 2.5)$, "Mastery":

$\theta \geq 2.5$. This method is similar to the persona-based prompting method in [88], where we use ability instead of student demographics.

Second, we employ **Response Gen SFT**, as explained in Section 2.5.1, which finetunes the LLM on ground-truth student responses given the item and ability. Prior works that finetune simulated students generally include prior student responses in the prompt [88, 264]; we use the student's ability as a proxy for this information.

Model	Smarter Balanced			CodeWorkout		
	PCC ↑	SCC ↑	RMSE ↓	PCC ↑	SCC ↑	RMSE ↓
Direct Item Difficulty Prediction Methods						
Diff SFT	0.4937	0.3188	0.7627	−0.0425	−0.0036	0.7714
Random Forest	0.4021	0.3115	0.8623	0.1846	0.1418	0.6783
kNN Mean	0.5959	0.5219	0.6806	0.1949	0.2415	0.6701
ICL Random	0.1215	0.0348	1.0426	0.2487	0.1806	0.6462
ICL kNN	0.5273	0.4717	0.7634	0.0209	0.0821	0.7397
Student Simulation-based Item Difficulty Prediction Methods (Llama-3.2-1B)						
Response Gen ZS	0.2651	0.2436	0.9088	0.0733	0.0812	0.7226
Response Gen SFT	0.5621	0.4254	0.7174	0.2475	0.2388	0.6487
SMART (ours)	<u>0.6490</u>	0.4861	<u>0.6699</u>	0.3082	0.3188	0.6225
Student Simulation-based Item Difficulty Prediction Methods (Llama-3.2-3B)						
Response Gen ZS	0.3446	0.3963	0.8568	0.1487	0.1636	0.6912
Response Gen SFT	0.6069	**0.5830**	0.6905	**0.4256**	<u>0.3648</u>	**0.5627**
SMART (ours)	**0.6737**	<u>0.5661</u>	**0.6197**	<u>0.3929</u>	**0.4230**	<u>0.5890</u>

Table 2.1: Performance on question difficulty prediction. Best performance is in **bold** and second best is <u>underlined</u>. SMART outperforms all direct prediction methods and generally outperforms other simulation-based methods.

2.6.4 Experimental Setup

We use `Llama-3.2-Instruct` [85] from HuggingFace [248] as the base model for our simulated student and automated scoring models. We perform all experiments with both the 1B and 3B parameter versions to examine how our methods scale across model sizes. We use the 1B automated scoring model to score responses from the 1B

simulated student model, and do the same for 3B. We load base models with NF4 quantization [63] and finetune using LoRA [93].

We perform a preliminary hyperparameter search on our validation sets. We set LoRA's rank = 128, $\alpha = 64$, and dropout = 0.05. For Smarter Balanced, for both the scoring model and student model, we perform SFT for 1 epoch, with learning rate = 1e-4 and linear warmup for 10% of steps, weight decay = 1e-2, gradient clipping norm = 1.0, and effective batch size = 64 using gradient accumulation. For DPO, we set learning rate = 1e-5, $\beta = 0.5$, the probability threshold $\epsilon = 0.1$ (Eq. 2.4), the number of negative candidates $m = 3$, and train on a random 20% subset of the training set to reduce costs. For CodeWorkout, we use the same hyperparamters except for the following: for the scoring model we perform SFT for 5 epochs with learning rate = 5e-5, for the student model we perform SFT for 3 epochs with learning rate = 1e-5, and perform DPO for 1 epoch with learning rate = 1e-6 on the full training set. We conduct all experiments on the Smarter Balanced dataset on NVIDIA A10G GPUs and all experiments on the CodeWorkout dataset on NVIDIA A40 GPUs.

At inference time, we sample $n = 1,000$ student abilities for simulation and generate a single response for each item/ability pair. We decode up to 500 tokens with nucleus sampling, setting temperature = 0.7 and p = 0.95. We use an inference-time batch size of 64 for the simulated student model and 2 for the scoring model, optimizing for inference speed.

2.6.5 IRT Model Fitting

We perform IRT fitting at 2 distinct stages in our pipeline: 1) when fitting to real student responses to create the ground-truth ability and item parameters, and 2) when fitting to simulated student responses to produce difficulty estimates.

The Smarter Balanced dataset does not contain student identifiers, but rather contains prior ability estimates based on each student's responses to a larger set of

questions, including other types such as multiple choice questions; the 49 short answer questions we use comprise all short answer items in the full set. To ensure that other item types and test set responses do not influence our ability estimates, we fit an IRT model from scratch only on the 49 items used in this study. Since we do not have access to student identifiers, we use the prior ability as a proxy. However, the student responses are very sparse on our set of questions, with around 63K unique ability values for around 85K responses, leading to a poor fit for the IRT model. As a solution, we round each prior ability to a single decimal place (we found this precision to work best out of several tried) and use the resulting value as the student identifier. This transformation leads to a significantly better fit for the ground-truth IRT model. We note that this process assumes access to prior estimates of student ability, which may or may not be available in real-world settings. However, we note that this a limitation of the data, rather than our method. Additionally, we note that this transformation does not unfairly benefit our method over the baselines, so the comparison between methods in this work remains fair.

When we fit the IRT model with the simulated responses, we perform a similar operation for assigning identifiers. We round the prompted ability for each simulated response to one decimal place, and use this value to form a new identifier for all simulated responses that result in that rounded ability. When re-training the IRT model, we include the ground-truth student responses from the train set only. While these ground-truth student responses do not directly interact with the estimated parameters for non-train items or simulated students, we find their inclusion to improve performance, perhaps due to their effect on global variables such as AdamW's adaptive learning rate.

We implement our IRT models using PyTorch, using the formulations in Equations 2.2 and 2.3 to compute the probability of a student's score for a particular item. For GPCM, following prior work [170], we set the first step parameter to

$d_{i0} = 0$, since only $C - 1$ step parameters are required for C score classes. We also impose the location constraint [170], where all step parameters for an item sum to 0, i.e., $\sum_{y=0}^{C-1} d_{iy} = 0$, ensuring that difficulty parameters across items are on the same scale and are identifiable. We also fix all discrimination parameters to 1, which improves model fit. We do not enforce normal priors on any parameters. We train the model using gradient descent, using the cross-entropy loss between the predicted score probabilities and the ground-truth scores. We train using the AdamW optimizer for 50 epochs with learning rate = 1e-3, weight decay = 0, and batch size = 256; for CodeWorkout, we train for 200 epochs when fitting the *ground-truth* data, which is necessary due to the smaller number of ground-truth student responses. We reserve a random validation set of 20% of all responses to determine how predictive our ground-truth IRT parameters are, comparing the most likely student scores predicted by the IRT model and the ground-truth scores in the validation set. For Smarter Balanced, we measure performance using QWK since scores are ordinal, with the IRT model achieving a QWK of 0.57 on the validation set. For CodeWorkout, we measure performance using PCC since scores are continuous, with the IRT model achieving a PCC of 0.49. These values show moderate predictability, indicating that while the student scores are difficult to predict, the parameters are well-calibrated.

2.7 Results, Analysis, and Discussion

We now detail our quantitative results on item difficulty prediction and student response simulation, and qualitatively analyze the simulated student responses and SMART's strengths and weaknesses.

2.7.1 Quantitative Results

2.7.1.1 SMART outperforms other methods on item difficulty prediction

We report the average performance across all test items and cross-validation folds for both datasets in Table 2.1. Our approach, SMART, outperforms all direct prediction methods on all metrics, even largely maintaining this advantage when using the small 1B base LLM. It also outperforms all other student simulation methods, with the exception of SFT when using 3B models; in these cases, SMART outperforms SFT by wide margins on some metrics while it is closely behind on others. These results indicate that both methods are powerful predictors of item difficulty, although there is high variability due to the small number of items in each dataset; larger test sets may be required to more clearly show SMART's advantage. We also observe that CodeWorkout is a more challenging dataset, with most methods performing worse than they do on Smarter Balance in terms of PCC and SCC. Still, the fact that SMART consistently outperforms baselines on both datasets shows that it can generalize across educational domains and student populations.

Model	Smarter Balanced				CodeWorkout			
	MAUVE ↑	FID ↓	Div. KL ↓	θ-Align ↑	MAUVE ↑	FID ↓	Div. KL ↓	θ-Align ↑
Student Simulation-based Item Difficulty Prediction Methods (Llama-3.2-1B)								
ZS	0.0058	0.2713	0.2535	0.0399	0.0196	0.0747	0.0714	−0.0014
SFT	0.1483	0.0847	<u>0.0215</u>	0.4481	<u>0.1777</u>	<u>0.0269</u>	<u>0.0315</u>	0.1730
SMART	0.1315	0.0918	0.0482	<u>0.7338</u>	**0.1931**	**0.0267**	**0.0284**	**0.5895**
Student Simulation-based Item Difficulty Prediction Methods (Llama-3.2-3B)								
ZS	0.0052	0.3386	0.4032	0.0211	0.0042	0.1284	0.2779	−0.0101
SFT	**0.1858**	**0.0757**	**0.0137**	0.5158	0.1426	0.0333	0.0515	0.2226
SMART	<u>0.1713</u>	<u>0.0785</u>	0.0279	**0.7655**	0.1479	0.0303	0.0335	<u>0.5582</u>

Table 2.2: Performance on student simulation metrics. Best performance is in **bold** and second best is <u>underlined</u>. SMART significantly outperforms other methods on ability following.

2.7.1.2 Student simulation is key when working with few training items

We observe that direct prediction baselines significantly underperform student simulation methods on both datasets. Diff SFT, which follows the approach of existing state-of-the-art methods [9, 29, 32], performs moderately well on Smarter Balanced but poorly on CodeWorkout. This result is perhaps not surprising, since Diff SFT's training signal completely comes from given item difficulties, and thus can only work when a large number of training items are available. Direct difficulty prediction baselines that require no training also follow a similar pattern: kNN Mean and ICL kNN perform moderately on Smarter Balanced, likely because reading comprehension questions with similar semantic features are likely to pose similar tasks to the student and thus have similar difficulties. However, since this heuristic does not apply to coding questions, these baselines do not generalize across datasets. The poor performance of ICL Random also shows that powerful LLMs do not perform well on the difficulty prediction task without careful calibration. In our low resource setting with few items to train with, student simulation is key: student ability and item difficulty are only related via the response score when training an IRT model, but our student simulator can effectively couple IRT training with textual student responses and a scoring model. As a result, this extra information helps us to calibrate the IRT model better, making it easier to generalize to previously unseen items.

2.7.1.3 Ablation study

We compare the performance of Response Gen ZS, Response Gen SFT, and SMART, with all models parameterized with the same base LLM and prompts, serving as an ablation study. Response Gen SFT significantly outperforms Response Gen ZS, which suggests that finetuning is required for the LLM to adapt to the response distribution of real student responses. With the exception of some metrics when using 3B models, SMART outperforms Response Gen SFT, which validates our hypothesis

that preferring responses with high IRT likelihoods in DPO training helps in the difficulty prediction task by calibrating the model in the context of student ability and implicit item difficulty.

2.7.1.4 SMART simulated student responses are aligned with abilities and diverse

Table 2.2 compares the student simulation methods on metrics regarding the quality and faithfulness of simulated responses. We see that SMART outperforms other methods by a wide margin on the θ-Align metric, which measures the correlation between the desired input ability and inferred ability by scoring the generated responses. This observation shows that DPO training leads to a strong preference for the model to follow the input ability, resulting in a faithful student simulator. We also compare the simulated student methods on distribution similarity metrics, measuring how closely their generated responses align with ground-truth ones. Response Gen ZS performs poorly, which emphasizes the need for finetuning or DPO training, over simple prompting, to align the LLM with diverse responses by real students. Interestingly, Response Gen SFT is marginally better than SMART on these metrics on Smarter Balance, while SMART is better on CodeWorkout. This observation highlights the fact that DPO training prioritizes the IRT likelihoods, which may or may not align with how realistic the generated responses appear. We also observe that 1B models outperform 3B models on these metrics on CodeWorkout, perhaps due to the lower coding ability of the 1B model, therefore aligning better with code written by novice coders (students).

2.7.2 Data Scaling Experiment

We conduct an experiment where we vary the amount of data available for training by using random subsets of the student responses in the training set at 0%, 25%, 50%, 75%, and 100%. We re-train both SFT and SMART using the 1B model on

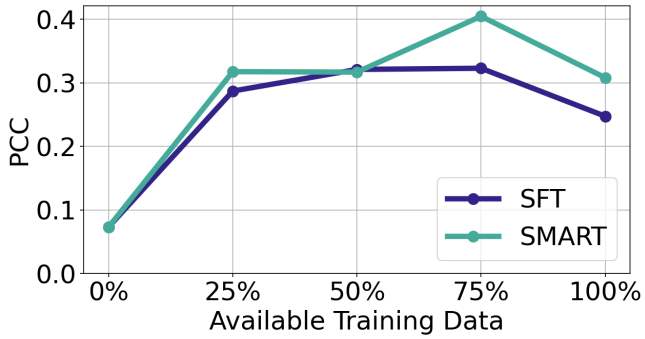


Figure 2.3: Results of data scaling experiment. Even a small amount of data is sufficient to achieve high performance. SMART begins to outperform SFT more significantly when more data is available.

these subsets, and report the PCC averaged over all cross-validation test splits on CodeWorkout. To only examine the impact of training data on the student model, we use the scoring model trained on all data, since in reality a stronger scoring model may be available (such as one that runs test cases on generated code).

We show the results of this data scaling experiment in Figure 2.3. We observe that even a small amount of training data is sufficient to drastically increase performance from zero-shot (25% of the training data contains 1,636 responses and 3,027 preference pairs in total). SFT and SMART perform similarly until 75%, when SMART begins to outperform SFT. Surprisingly, performance drops for both methods from 75% to 100%, indicating that some training responses may negatively impact performance, such as ones that represent item types that are not present in the test set. This result further reflects the challenges of testing on a small number of items.

2.7.3 Qualitative Evaluation

To further understand the strengths and weaknesses of our method, we perform a qualitative analysis, where we examine patterns in items and generated responses that affect difficulty prediction accuracy.

2.7.3.1 Models Reflect Ability-Specific Features in Student Responses

Table 2.3 shows examples of generated responses to Smarter Balanced items for students with varying ability levels. Compared to SFT, SMART more clearly reflects realistic patterns based on ability. Broadly speaking, students with low ability have short one-sentence responses, spelling mistakes, poor grammar, and incorrect punctuation. High-ability students, on the other hand, have longer responses with complex sentence structures and strong vocabulary, and make an effort to answer all aspects of the question to receive full credit. SMART also captures student response diversity, especially for medium-ability students, whose responses usually contain simple words, but differ in length, coherence, and score obtained by the trained scoring model. Our preference pair creation encourages this diversity since varied responses can be preferred as long as they are sufficiently likely under the IRT model.

2.7.3.2 Performance is Item-dependent

For Smarter Balanced items, we find that SMART's performance is highly dependent on the type of reading comprehension task (argumentation, narration, explanation, among others) in the item. In particular, difficulty estimates are accurate for items that require information recall or those that add narrative elements to a story. On the other hand, estimates are less accurate for items requiring higher order thinking [129], such as summarizing the main idea of a passage, or writing an introduction to a paragraph that includes a "thesis" or a "main controlling idea". For these items, we find that SMART tends to *overestimate* their difficulty. We see that even for high input ability values, SMART tends to generate incomplete responses that only state facts from the passage, rather than summarizing it. Such responses receive only partial credit and thus lead to high difficulty values under IRT. Future work can attempt to augment the training data with synthetic responses with higher-order thinking to aid simulation for high-ability students.

Passage	Plants need food to survive, just as animals do; but, unlike animals, they do not eat solid foods. Instead, plants get most of the food they need from the sun through a process called photosynthesis. Using this process, plants can absorb the sun's light energy to make their own food. Although they cannot move around to find food, plants can make their food anywhere as long as they have three things: carbon dioxide, water, and light...
Question	Write an introduction to the report that establishes and introduces a clear claim about how plants make their own food.
Ability	Response
	SFT
Low	Photosynthesis is the process by which plants make their own food from the sun.
Medium	Plants are very important to us because they give us oxygen, that we need to survive. Plants are very important because they give us oxygen.
High	Plants are very important to us. We rely on them for food and for oxygen. Plants make all of this possible. They take the carbon dioxide, the water, and the light, and turn them into sugar and oxygen. That is why photosynthesis is so important to animals.
	SMART (ours)
Low	that the plants are good for us and the world
Medium	Plants are one of the most important things on earth, without them we would not be alive today. Plants are very important to us humans because without them we wouldn't be able to survive, they also help us in many ways like we can make food out of them and plants help us by providing oxygen.
High	Photosynthesis is the process by which plants create food. The process of photosynthesis is very important to all living things, especially because without it, the air we breathe would be filled with only carbon dioxide, and without oxygen, animals and humans would not be able to survive. In this report, you will learn more about how plants make their own food.

Table 2.3: Examples of student responses generated by SFT and SMART for a publicly available Smarter Balanced item, conditioned on abilities in the low, medium, and high ranges. SMART adapts better to the instructed ability than SFT. For low ability, SMART uses very simple language and doesn't discuss how plants make food, while SFT uses more formal language and does mention how plans make food, which would be atypical for a low ability student. For medium ability, both methods write a coherent introduction but don't mention how plants make food; SMART writes a slightly longer response. For high ability, SMART makes sure to motivate the importance of photosynthesis as well as directly answer the question by stating how plants make food, while SFT neglects to mention how plants make food.

2.7.3.3 Outlier Generated Responses

We observe that SMART occasionally generates outlier responses with fidelity issues, with failure patterns including 1) repeating the passage, 2) not being relevant to the item, and 3) being highly repetitive. These outlier responses are often inaccurately scored by the scoring model, affecting downstream item difficulty prediction. A possible solution is to use a fidelity checking model to filter out such outlier responses and re-generate when it happens.

2.8 Conclusions and Future Work

In this paper, we presented SMART, a novel method for aligning LLM-based simulated students with a prompted ability and implicit item difficulty via DPO training. The key is to form preference pairs where preferred responses are more likely under an IRT model. We perform a large-scale simulation with the resulting student model, automatically scoring generated responses with a trained LLM, and fit the resulting data with IRT to estimate the difficulties of unseen items. Through extensive experiments on a real-world student response dataset, we show that SMART outperforms other item difficulty prediction methods by capturing response patterns in the data that reflect the diverse abilities of the students and the difficulties of the items.

There are many avenues for future work. First, we can include generated student responses in the pool of candidates for DPO [188], which may help to explicitly disprefer outlier responses that are not aligned with real student data. Second, we can explore the applicability of SMART in other domains such as programming, math, and in non question-response settings such as dialogues [208]. Finally, we can extend SMART's student representation to go beyond simple scalar-valued student abilities and into more descriptive personas [250].

CHAPTER 3

EXPLORING KNOWLEDGE TRACING IN TUTOR-STUDENT DIALOGUES USING LLMS

3.1 Introduction

Tutoring, often in the form of natural-language dialogues between tutors and students, has proven to effectively improve student learning outcomes, in both human tutoring [173] and dialogue-based intelligent tutoring systems [175]. Recent advances in generative artificial intelligence (AI), especially large language models (LLMs), have led to the development of generative AI-powered tutoring chatbots, such as Khan Academy's Khanmigo [110] and Carnegie Learning's LiveHint [44]. By wrapping LLMs around learning content such as textbooks and practice questions, these automated chatbots can interact with students in real time, potentially reaching a larger number of students than human tutors, while greatly reducing authoring efforts relative to previous-generation dialogue tutors [101].

To use LLMs as tutors, it is important to i) ensure they use effective pedagogical strategies and ii) control the safety of their output. Most existing LLM-powered tutoring chatbots are developed by either fine-tuning on actual tutor utterances in tutoring dialogue transcripts or prompt engineering. Along these lines, plenty of works over the last two years have studied how to make LLMs follow effective tutoring principles. The work in [240] studies the latent decision-making process behind human tutor utterances and shows that adopting decisions by human experts can benefit LLMs in tutoring. The work in [152] identifies tutor moves when they interact with an LLM-powered simulated student agent and shows that move annotation data

can make LLMs become better tutors. The work in [83] generates large amounts of synthetic tutoring data via AI roleplay and shows that fine-tuning on this data, in addition to data collected from human tutors and students, makes LLMs more effective in pedagogy. The work in [62] proposes a way to measure tutor uptake, i.e., acknowledging what students have said, which is another beneficial strategy for LLMs to adopt. The work in [196] attempted to summarize tutor utterances into actionable feedback for students using LLMs, but found that the resulting messages were unable to outperform those written by real human teachers.

One important observation is that existing works involving LLMs have almost exclusively studied the *tutor turns* in dialogues rather than the *student turns*. A possible reason is that student turns can be noisy in nature, partially due to students not fully mastering the required knowledge, resulting in utterances that contain errors, which are harder to analyze than tutor utterances. Take Google's LearnLM [83] for example: student dialogue turns in their AI roleplay-based synthetic training data are generated by prompting an LLM to make an error that a real student would make. This error information is then fed to the tutor LLM to help the tutor model identify the error and provide corresponding feedback. However, there is a key limitation to this approach: assuming LLMs can behave like real students may not be reliable. As recent work [76] shows, LLMs are ineffective at anticipating, parsing, and following flawed reasoning. More importantly, real open-ended student discourse is highly valuable since it may indicate flaws in student knowledge and even reveal specific misconceptions. Therefore, analyzing student dialogue turns can be a form of formative assessment to help tutors and AI-powered chatbots understand student progress and provide more targeted feedback. It can also potentially improve simulated student agents [107] to make them better at reflecting real student behavior, which can, in turn, improve the training of tutor chatbots through reinforcement learning [198, 89].

3.1.1 Contributions

In this work, we propose dialogue knowledge tracing (dialogueKT), a novel task, which analyzes student discourse within the knowledge tracing (KT) framework [191]. We enable KT on dialogues via a two-step process: we i) use proprietary LLMs, specifically GPT-4o [178], to annotate dialogues with student response correctness and knowledge component (KC) tags using Common Core standards, and ii) train KT models on the annotated data to estimate student knowledge at the dialogue-turn level. We introduce a novel LLM-based KT method, LLMKT, that leverages the textual content in dialogues, by fine-tuning the open-source Llama 3 LLM [85] on the KT objective. Using two existing tutor-student math dialogue datasets, we show that i) our new methods significantly outperform existing KT methods, especially when little training data is available, and ii) all KT methods can accurately predict student correctness given enough training data. We also use qualitative analysis and learning curve visualizations to show that LLMKT learns meaningful knowledge state estimates. Finally, we show that GPT-4o's dialogue annotations are accurate, according to human experts. Overall, by assessing student performance in more detail, we can make it possible to apply KT to LLM-human tutoring dialogues, enabling mastery learning and various other pedagogies and forms of detection afforded by KT [191].

We acknowledge many limitations of our work upfront since it is, to the best of our knowledge, the first at attempting to perform KT in open-ended tutoring dialogues. First, a "time step" in our KT setup corresponds to a single student turn in the dialogue, which is highly granular; we cannot explore setting an entire dialogue as a time step since no existing open-source large-scale tutoring dialogue dataset links students across dialogues. Second, our work is restricted to math since analyzing the KCs in math problems is challenging yet well-studied; it remains to be seen whether our approach of Common Core standard tagging generalizes to other subjects. Third,

due to the smaller scale of tutoring dialogue datasets compared to those typically used in standard KT works, our methods need to be tested on larger-scale data. One dataset we use in our experiments contains partial dialogues that are cut short for LLM evaluation purposes [165], which further adds to this issue. Therefore, we publicly release our code[1] so that researchers who own a large amount of dialogue data can experiment on their proprietary data.

3.2 Related Work

3.2.1 Knowledge Tracing and Knowledge Component Labeling

Knowledge tracing (KT) [191] is a well-studied task in the student modeling literature. It breaks down student learning into a series of time steps practicing certain KCs and uses the correctness of each response step to track student knowledge. Classic Bayesian KT methods use a latent binary-valued variable to represent whether a student masters a KC or not. Factor analysis-based methods [52, 190] use features in addition to latent ability parameters in a logistic regression setting to predict student response correctness. Since neural networks became popular, plenty of deep learning-based KT methods were developed. These models have limited interpretability since student knowledge is represented as hidden states in complex, deep neural networks. Most of these methods use long short-term memory networks [194] or variants [216], coupled with with memory augmentation [270], graph neural networks [259], attention networks [78, 187], or pre-trained word embeddings in question text [140]. Many recent works have attempted to use LLMs in some ways for KT. Some use the neural network architecture of LLMs and train on student response data [266]. Others use LLMs to represent questions and responses in certain domains such as math [102], language learning [225, 58], and computer science education [126, 139]. More recently,

[1] `https://github.com/umass-ml4ed/dialogue-kt`

[132] attempted prompting-based KT that directly asks LLMs to represent questions and student knowledge states as text. Despite some success in question response modeling, it remains to be seen whether these models can be useful in text-heavy KT situations such as dialogues.

There exist many prior works on automated KC tagging and discovery, including the Q-matrix method [24], which assigns KCs to questions using student response data. More recently, methods such as [214] also learn latent KCs by training deep learning models on KT data and enforcing priors from pedagogical theory. While these methods effectively learn new KCs, they do not produce textual descriptions of KCs, making them difficult to interpret. Several recent works have used LLMs to assign expert-written KCs to math problems via prompting [149, 258], fine-tuning [213], and reinforcement learning [130]. However, none have attempted assigning KCs in open-ended dialogues. A prior work has attempted to tag KCs in dialogue turns [57], although using a manual approach that cannot be scaled.

3.2.2 Dialogue Analysis

Existing works on dialogue analysis using LLMs have primarily focused on analyzing tutor behavior. The works in [61, 228] automatically classify the "talk moves" that tutors and students make via fine-tuned LLMs, and the work in [3] uses talk moves in dialogues to predict student performance in post-tests. In pre-LLM dialogue tutors, AutoTutor used latent semantic analysis to infer whether a student response was correct, and when incorrect, whether misconceptions were involved [42]. Why2-Atlas used deep syntactic analysis to convert a student response to a proof, analyzed the proof for errors represented as lack of knowledge [235], and later used Bayesian networks [156]. The Watson Tutor used ensembles of different classifiers on a range of textual features for correctness evaluation [49] and Glicko-2 [79] for KT.

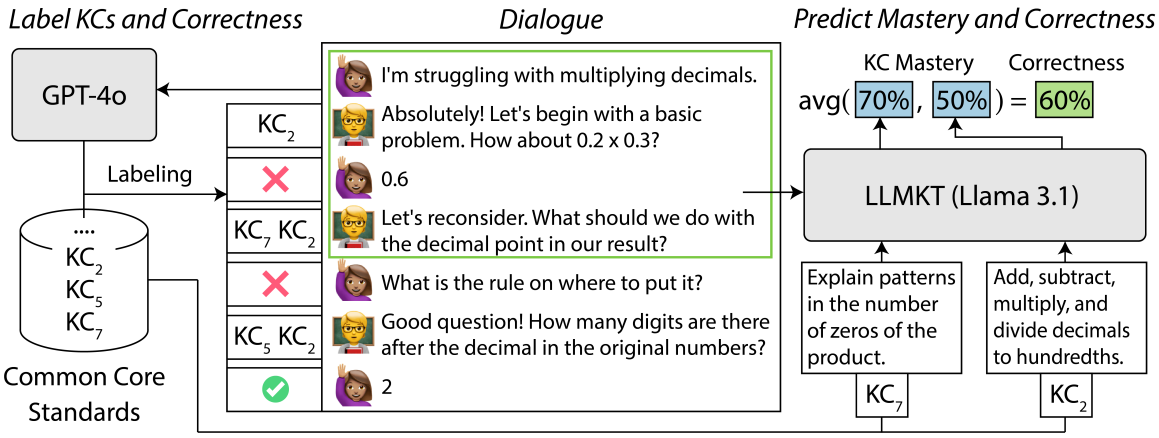


Figure 3.1: Overview of the dialogueKT framework and LLMKT. For annotation, we provide a full dialogue to GPT-4o, which generates correctness labels and retrieves Common Core standards as KCs for each turn. For KT, LLMKT takes a dialogue up to the target turn and its KCs to predict mastery on each KC and response correctness.

3.3 Knowledge Tracing in Dialogues

We now formulate the dialogueKT framework. Our goal is to develop methods that can i) estimate student knowledge on KCs involved in each dialogue turn and ii) use these estimates to predict the correctness of student dialogue turns. Intuitively, we can view tutor-student dialogues as sequences of (formative) assessments. Visualized in Figure 3.1, some tutor turns can be viewed as posing a task that is related to some learning objectives and one or more KCs. They can reflect different tutor moves: asking a clarification question, identifying a student error, providing actionable feedback, prompting the student to think in a Socratic way, or offering encouragement [229, 62]. Student turns, on the other hand, contain correct or incorrect responses to tasks posed by the tutor, which reflect their thinking process and offer insights into their knowledge state. A pair of tutor-student dialogue turns corresponds to a time step in KT. Therefore, we split the dialogueKT framework into three stages: i) identify the KCs involved in each student dialogue turn, ii) classify whether the student response is correct or incorrect, and iii) apply new and existing KT methods on the resulting data.

Formally, we define each dialogue d to be a sequence of alternating tutor turns, t, and student turns, s, i.e., $d = (s_0, t_1, s_1, \ldots, t_M, s_M)$, where M is the total number of turn pairs for that dialogue and s_0 is present if the student initiates the dialogue. Each relevant turn pair is assigned a correctness label $y_j \in \{0, 1\}$, where $j \in \{1, \ldots, M\}$ is the turn pair index; $y_j = 1$ means that s_j is a correct response to t_j, and $y_j = 0$ means that it is incorrect. In practice, we also use $y_j = \text{na}$ in cases where correctness is not well-defined for t_j, e.g., the turn is off-topic or the tutor is offering emotional support. Each tutor turn is also associated with a set of KCs, $\mathcal{C}_j = \{c_{j1}, \ldots, c_{jK}\}$, where K is the number of KCs for that turn. These KCs are related to learning objectives outlined by the tutor, which lists knowledge elements that the student must posses to respond correctly. In this work, we use the Common Core math standards [55] as the KCs since they are i) standard and widely adopted and ii) textual in nature and are interpretable to both humans and LLMs. For irrelevant dialogue turns that we do not consider, $\mathcal{C}_j = \emptyset$.

In a dialogue turn, we use $z_{jk} \in \{0, 1\}$ to denote the student's unobserved, binary-valued latent mastery status of the KC, c_{jk}, that is involved in the current step; $z_{jk} = 1$ means that the student masters KC k in dialogue turn j. In our framework, the student's mastery of all KCs involved in a turn determines whether they will respond correctly to the task posed by the tutor. Formally, the probability of a correct student response at turn j can be written as $P(y_j = 1 | z_{j1}, \ldots, z_{jK})$. The latent KC mastery depends on the entire dialogue history, i.e., $P(z_{jk} = 1) = \phi(c_{jk}, t_{\leq j}, s_{<j}, y_{<j}, \mathcal{C}_{<j})$, where ϕ returns the probability of the student mastering KC c_{jk}, and subscripts $< j$ and $\leq j$ correspond to all past student and (including the current) tutor dialogue turns, respectively. In the next section, we detail how we parameterize and train this function in practice.

3.4 Methodology

We now detail our two primary technical contributions. First, we detail how to prompt pre-trained LLMs to automatically annotate dialogue turns to collect the KC and response correctness labels needed for dialogue knowledge tracing. Second, we propose new KT methods for the dialogueKT task that incorporate the rich textual information in tutor-student dialogues. We visualize the dialogueKT framework and the LLMKT method in Figure 3.1.

3.4.1 Automated Dialogue Annotation

In order to perform knowledge tracing on dialogues, each student conversation turn must first be annotated with two labels: i) the correctness of the student response, and ii) a list of KCs involved in the turn. While it is possible for human experts to provide these annotations, doing so would be incredibly time consuming and difficult to scale. Following recent works that have had success using LLMs to annotate data in educational tasks [240, 209], we use a state-of-the-art LLM, GPT-4o [178], to perform the annotation via simple, zero-shot chain-of-thought (CoT) prompting[2]. We conduct the two annotation tasks separately since we find that doing so leads to improved accuracy compared to combining them. In Section 3.7, we conduct a human evaluation with math teachers to show that the GPT-provided annotations are mostly accurate and aligned with human judgment.

For correctness annotation, we provide the full dialogue as input to the LLM and instruct it to first summarize each turn in the dialogue in a CoT way, before labeling the correctness of each turn. We specifically instruct the LLM to "summarize" to leverage the fact it is pre-trained extensively on summarization tasks; this approach is

[2]We release our prompts in an online supplementary material: `https://osf.io/873ms?view_only=f32472a6560649e692a4e2ba603ef52c`

inspired by recent work [151] that has shown that such an approach is effective for tasks such as code editing.

For KC annotation, we adapt the recursive tagging algorithm from [149], which tags math word problems with Common Core math standards. The algorithm uses hierarchically organized standards from the Achieve the Core[3] (ATC) coherence map, where standards are grouped into mid-level "clusters", which are further grouped into high-level "domains". To assign standards to a dialogue, we first provide the LLM with the full dialogue and the list of all 11 domains, and ask it to summarize the dialogue before selecting all domains that are relevant to the learning objectives. We then take the union of all clusters that are children of the selected domains and prompt the LLM again, this time asking it to summarize at the turn level but still select relevant clusters at the dialogue level. Finally, we take the union of all standards that are children of the selected clusters and prompt the LLM a final time, having it summarize each turn and assign standards at the turn level. We treat each Common Core math standard as a KC.

We note that by annotating a full dialogue with a single prompt, KT in real-time becomes impossible since information from future turns is available when annotating any given turn. This limitation could be alleviated in practice by annotating dialogues one turn at a time; however, such an approach is costly and we leave that for future work.

3.4.2 LLMKT

Another feature that makes dialogueKT different than the regular KT task is that most dialogue turns (at least 58%) are associated with more than one KC; see Table 3.1 for details. In contrast, most existing KT methods are developed for questions on a single KC; solutions that can deal with the occasional multi-KC question include

[3] `https://achievethecore.org/`

i) converting a single question-response pair with multiple KCs to multiple pairs for different "questions", each covering a single KC, and ii) treating a unique combination of multiple KCs as a new KC. However, due to the prevalence of dialogue turns tagged with multiple KCs, we found that these solutions are not applicable to our setting. Therefore, we consider the student's likelihood of responding correctly to a tutor-posed task to be their average mastery of all KCs involved in the turn. Empirically, we found that averaging over KC masteries performed better than taking a product over them, consistent with findings in prior work [155]. Formally, we model the probability of a correct student response at a turn as

$$P(y_j = 1) = \frac{1}{K} \sum_{k=1}^{K} P(z_{jk} = 1). \tag{3.1}$$

In other words, we use a compensatory model rather than a conjunctive model [155]. We now detail our proposed LLM-based KT method, which we refer to as LLMKT. One can, in theory, estimate a student knowledge state, z_{jk}, by simply providing the text of the dialogue up to t_j to a pre-trained LLM and prompting it to predict whether the student masters the KC, c_{jk}, for each $k \in [1, K]$. However, since LLMs are not pre-trained on the specific KT task, we cannot expect them to do well in this "zero-shot" setting. Therefore, we adopt a fine-tuning approach using Llama-3.1-8B-Instruct, an open-source, decoder-only LLM, for the KT task. We model the estimated probability of a student mastery $\hat{z}_{jk}$ as

$$\hat{z}_{jk} = P_\theta(z_{jk} = 1) = \frac{\exp(v_\theta^{\mathcal{T}}(t_{\leq j}, s_{<j}, c_{jk}))}{\sum_{\text{tok} \in \{\mathcal{T}, \mathcal{F}\}} \exp(v_\theta^{\text{tok}}(t_{\leq j}, s_{<j}, c_{jk}))}, \tag{3.2}$$

where θ represents the model parameters, and $v_\theta^{\mathcal{T}}(\cdot)$ and $v_\theta^{\mathcal{F}}(\cdot)$ return the output logits of the LLM for the "True" and "False" vocabulary tokens, respectively. In other words, with i) the entire conversation history and ii) a textual prompt instructing the LLM to estimate the student's current KC mastery level as input, we use an

LLM to parameterize this estimate through two token generation probabilities. In practice, to improve model efficiency, we pack all KCs in C_j into a single prompt; we use customized attention masks and position embeddings to ensure that KCs do not attend to each other and their mastery levels are estimated independently. We do not explicitly provide KCs or correctness labels for previous turns due to memory constraints, but observe that LLMs can infer them from the context.

We fine-tune the LLM by maximizing the likelihood of all observed student response correctness labels, or equivalently, minimizing the binary cross entropy loss between the ground truth and predicted correctness values, y_j and $\hat{y}_j$, as

$$\hat{y}_j = \frac{1}{K} \sum_{k=1}^{K} \hat{z}_{jk} \tag{3.3}$$

$$\mathcal{L}(d;\theta) = -\sum_{j=1}^{M} y_j \log \hat{y}_j + (1 - y_j) \log(1 - \hat{y}_j). \tag{3.4}$$

While this method is relatively simple, it has many advantages. First, because we start with a powerful pre-trained LLM, LLMKT is able to leverage existing textual analysis capabilities that are not available in other KT architectures. Second, we do not add any new parameters to the pre-trained model, likely reducing the need for large amounts of training data. Third, since LLMKT operates on textual content rather than question/KC embeddings that need to be learned from scratch, it should be more generalizable to new KCs that are not seen during training. We note that LLMKT is similar to CLST [102], which trains an LLM to predict student correctness using a softmax over tokens (Eq. 3.2). However, unlike LLMKT, their method does not handle multiple KCs per item, which is critical to the dialogueKT task.

3.4.3 DKT-Sem

In addition to LLMKT, we experiment with an alternative, simpler KT method for dialogues, which can be viewed as a strong baseline. We slightly modify the deep KT (DKT) model [194] to use semantic embeddings of the textual content in

dialogues, resulting in a method we term DKT-Sem. We map the text embeddings of every dialogue turn to the latent knowledge space in DKT as input, and map the text embeddings of KCs to the latent knowledge space to predict KC masteries. We believe that this simple method can result in better generalization across KCs and dialogues compared to DKT, which learns question and KC representations from scratch. Formally, we model KC mastery probability as

$$\mathbf{t}_j = \text{S-BERT}(t_j), \quad \mathbf{s}_j = \text{S-BERT}(s_j), \quad \mathbf{c}_j = \tfrac{1}{K}\textstyle\sum_{k=1}^{K} \text{S-BERT}(c_{jk}) \tag{3.5}$$

$$\mathbf{y}_j = \text{emb}(y_j), \quad \mathbf{x}_j = \tanh([\mathbf{t}_j; \mathbf{s}_j; \mathbf{c}_j]\mathbf{W}) \oplus \mathbf{y}_j \tag{3.6}$$

$$\mathbf{h}_j = \text{LSTM}(\mathbf{x}_{\leq j}), \quad \hat{\mathbf{z}}_{j+1} = \sigma(\mathbf{h}_j \mathbf{B} \mathbf{C}^T). \tag{3.7}$$

The input at each time step is created by first computing the sentence BERT (S-BERT) [200] embeddings of the teacher turn, $\mathbf{t}_j$, student turn, $\mathbf{s}_j$, and element-wise average embeddings of the KCs, $\mathbf{c}_j$, respectively. Following DKT, response correctness is mapped to a learnable embedding, $\mathbf{y}_j$, which is then added element-wise to a transformed version (through $\mathbf{W}$ and tanh) of the concatenated tutor turn/student turn/KC embeddings, resulting in the input vector $\mathbf{x}_j$. Finally, the vector of estimated KC masteries at the next turn, $\hat{\mathbf{z}}_{j+1}$, are computed via a bilinear projection, $\mathbf{B}$, between the LSTM hidden state, $\mathbf{h}_j$, and the matrix of all KC S-BERT embeddings, $\mathbf{C}$, followed by the sigmoid function. We train DKT-Sem using the same objective as LLMKT (Eq. 3.4). Due to the smaller model size, DKT-Sem is easier to train than LLMKT, especially when access to computational resources to support LLM fine-tuning is limited.

Dataset	Num. Dia.	Num. Labels	Correct	Num. KCs	Avg. KCs/Dia.	Avg. KCs/Turn	Turns w/ >1 KC
CoMTA	153	623	57.78%	164	3.20	1.94	58.75%
MathDial	2,823	13,200	49.86%	145	4.16	2.32	82.89%

Table 3.1: Statistics for the CoMTA and MathDial datasets, from left to right: total number of dialogues, total number of turn pairs with correctness labels, percentage of turn pairs where the student response is correct, total number of unique KCs in the dataset, average number of unique KCs per dialogue, average number of KCs per turn pair, and percentage of turn pairs with more than one KC.

3.5 Experiments

3.5.1 Datasets

We experiment with two datasets, CoMTA and MathDial, to study the dialogueKT task and validate the proposed KT methods. We select CoMTA since it is recent and highly reflective of AI-student dialogues that are becoming prevalent, and we select MathDial since it is relatively large-scale. We acknowledge that there are many other datasets available, such as the ones in [61, 229, 240] and ones not focusing on math education; our overall framework should be applicable to these as well. We show statistics for CoMTA and MathDial in Table 3.1.

The CoMTA dataset [165] contains 188 dialogues between human students and Khanmigo, Khan Academy's GPT-4-powered tutor. In most dialogues, students ask for help with a specific topic or problem(s) that they do not know how to solve, and work through one (or more) problems with the tutor throughout the dialogue. The tutor is primed with a series of customized prompts developed by Khan Academy to behave like real human tutors: to use the Socratic method, frequently asking questions at almost every dialogue turn, and to not directly reveal correct answers to students. Each dialogue centers around a math subject among Elementary, Algebra, Trigonometry, Geometry, and Calculus. However, since Common Core does not contain Calculus

standards, we remove Calculus dialogues, resulting in 153 remaining dialogues. Since no train/test split is defined, we perform a 5-fold cross-validation in experiments.

The MathDial dataset [152] contains 2,848 dialogues between students simulated by GPT-3.5 and crowd workers role-playing as human tutors. Each dialogue is in the context of an incorrect "student" solution to a math problem, and the tutor's goal is to help the student arrive at the correct solution by helping them identify and correct their errors. We use the original train/test split, containing 2,235/588 dialogues, respectively, after removing a total of 25 dialogues where GPT-4o's annotation of student response correctness and turn KCs failed.

3.5.2 KT Methods

In addition to LLMKT and DKT-Sem, we also evaluate the performance of a suite of widely-used KT methods on the dialogueKT task. Our goal is to test whether existing KT methods, developed for a slightly different use case, are applicable to our setting and how they compare to methods designed for dialogueKT.

We evaluate multiple popular deep learning-based KT methods: **DKT** [194], **DKVMN** [270], **AKT** [78], **SAINT** [53], and **simpleKT** [144], all sourced from the pyKT code repository [145]. There are two key differences between these existing KT methods and our proposed KT methods for dialogues: i) they learn new embeddings for questions, KCs, and response correctness without leveraging any textual information, and ii) they are designed to analyze questions that cover a single KC. Since most of the dialogue turns contain at least two KCs in the CoMTA and MathDial datasets, we convert each turn into K "pseudo-turns" that are each associated with one of the KCs, c_{jk}; all such pseudo-turns are associated with the same correctness label, y_j, of that turn. This modification follows common practice in the KT literature [78, 144]. We predict a single $\hat{\mathbf{z}}_j$ vector for each turn pair, using the output of the most recent time step that does not take y_j as input, to avoid leaking the correctness label across

pseudo-turns. We note that predicting a vector instead of a single probability requires altering the size of the output projection for all methods except DKT. We train all methods using the same objective as LLMKT and DKT-Sem (Eq. 3.4). We also compare against Bayesian KT (**BKT**) [56], a classical KT method that uses a hidden Markov model to learn interpretable, per-KC learning acquisition parameters and use them to predict changing mastery levels for a student. Since BKT is designed to model data where each question is only assigned a single KC, we similarly expand turns into "pseudo-turns" as described above. Due to the rigidity of BKT, we use a slightly different setup compared to other methods. At train-time, we train on all pseudo-turn labels, rather than aggregating at the true turn level. At test-time, we take the average pseudo-turn prediction to be the true turn prediction. In contrast to the other methods, this pseudo-turn level prediction setup does not leak correctness labels because information does not interact across KCs in BKT. We use pyBKT [20] to train and generate predictions.

3.5.3 Metrics and Experimental Setup

To quantitatively evaluate the effectiveness of KT methods in the dialogueKT task, we employ 3 widely used metrics: i) accuracy (**Acc.**), the portion of predicted labels that match the ground truth after rounding $\hat{y}_j$ to 0 or 1, ii) the area under the receiver operating characteristic curve (**AUC**), the most widely used metric in the KT literature as it accounts for class imbalance and all prediction thresholds, and iii) **F1**, the harmonic mean between binary precision and recall. We compute these metrics on all correctness labels except for the first label in each dialogue, since existing KT methods are unable to make educated estimates without prior labels in context.

We implement LLMKT using the Huggingface Transformers library [248] and perform fine-tuning with LoRA [93] on NVIDIA RTX A6000 GPUs. We implement DKT-Sem using PyTorch and use the `all-mpnet-base-v2` S-BERT model to generate

text embeddings. During training, we randomly take 20% of the dialogues in the train set to use as a validation set and perform early stopping. We then evaluate all metrics on the test set, and repeat for all 5 folds on CoMTA.

Before computing final results, we perform a grid search to find optimal hyperparameters for each method, keeping the checkpoint with the highest AUC on the validation set. For LLMKT, we perform the search on the CoMTA dataset and use the same parameters on MathDial to reduce training time, and for other methods we use the best hyperparameter values found for each dataset. For LLMKT, we search over learning rate $\in \{5 \cdot 10^{-5}, 1 \cdot 10^{-4}, 2 \cdot 10^{-4}, 3 \cdot 10^{-4}\}$ and LoRA's rank $r \in \{4, 8, 16, 32\}$, and fix LoRA's α to 16, batch size to 64 using gradient accumulation, gradient norm clipping to 1.0, and train for 5 epochs. For other KT methods, we search over learning rate $\in \{1 \cdot 10^{-4}, 2 \cdot 10^{-4}, 5 \cdot 10^{-4}, 1 \cdot 10^{-3}, 2 \cdot 10^{-3}, 5 \cdot 10^{-3}\}$ and embedding size $\in \{8, 16, 32, 64, 128, 256, 512\}$, and fix batch size to 64, train for 100 epochs, and set all other model-specific hyperparameters to the defaults in pyKT. For all methods, we train with the AdamW optimizer with a weight decay of $1 \cdot 10^{-2}$. We found the optimal learning rate and r of LLMKT to be $2 \cdot 10^{-4}$ and 16, respectively; the optimal hyperparameters for other KT methods varied across methods and datasets, but tended to have smaller embedding sizes on CoMTA and larger ones on MathDial since the former is much smaller in scale compared to the latter.

3.6 Experimental Results

3.6.1 Quantitative Results

Table 3.2 shows the quantitative performance of all KT methods on both datasets. We see that LLMKT significantly outperforms all existing KT methods and generally outperforms DKT-Sem on both datasets, confirming its effectiveness in the dialogue setting. This advantage over other KT methods is likely due to the use of a pre-trained LLM to capture the rich textual content in dialogues. As a result, the LLM can

Method	CoMTA			MathDial		
	Acc.	AUC	F1	Acc.	AUC	F1
BKT	$51.02_{\pm 11.16}$	$52.50_{\pm 10.61}$	$53.41_{\pm 13.53}$	60.71	64.19	56.71
DKT	$52.59_{\pm 7.37}$	$53.20_{\pm 10.50}$	$49.56_{\pm 4.73}$	59.90	63.22	55.18
DKVMN	$47.22_{\pm 9.12}$	$46.81_{\pm 8.04}$	$48.80_{\pm 12.37}$	57.53	60.42	52.45
AKT	$49.95_{\pm 4.74}$	$51.40_{\pm 7.34}$	$50.78_{\pm 11.04}$	62.07	63.31	57.14
SAINT	$49.17_{\pm 4.88}$	$47.81_{\pm 3.89}$	$50.38_{\pm 6.68}$	55.97	60.10	54.24
simpleKT	$51.25_{\pm 4.05}$	$51.25_{\pm 5.60}$	$49.82_{\pm 10.45}$	59.24	63.83	53.85
DKT-Sem	$\underline{56.83}_{\pm 9.75}$	$\underline{61.82}_{\pm 6.56}$	$\mathbf{63.27}_{\pm 9.57}$	<u>62.17</u>	<u>66.18</u>	<u>59.30</u>
LLMKT	$\mathbf{58.01}_{\pm 5.40}$	$\mathbf{65.79}_{\pm 4.44}$	$\underline{60.72}_{\pm 10.77}$	**68.41**	**76.71**	**62.21**

Table 3.2: Student dialogue turn correctness prediction performance (averaged with standard deviation shown when applicable) for all KT methods, with best numbers in **bold** and second-best <u>underlined</u>. LLMKT outperforms all other methods on both datasets.

estimate student knowledge after fine-tuning on student turn response correctness labels. We also see that DKT-Sem performs better than all existing KT methods, even though it largely uses the same model architecture as DKT. This result further emphasizes the benefit of utilizing the textual dialogue context in some way, even if the original model architecture of existing KT methods remains largely unchanged.

On CoMTA, we see that all existing KT methods almost completely fail, unable to outperform the simple baseline of predicting the majority class label (which corresponds to an Acc. of $57.83_{\pm 5.08}$, AUC of $50.0_{\pm 0.00}$, and F1 of $58.87_{\pm 29.70}$). This result is expected since the size of the CoMTA dataset is small and neural network-based KT methods are data-hungry. Another important factor is that there are 164 unique KCs across only 623 labeled dialogue turn pairs, which is not enough for these methods to properly learn the KC embeddings or for BKT to learn the per-KC parameters. In contrast, LLMKT needs only minimal training data to reach acceptable accuracy, due to the underlying LLM being highly effective at understanding previous student dialogue turns. It can also leverage the textual content of the KC definitions to generalize across KCs, which is especially beneficial for ones that do not occur frequently.

On MathDial, which has 21x more data than CoMTA, we see that existing KT methods perform significantly better (for reference, the majority baseline only gets an Acc. of 52.64 since class labels are balanced). This result is expected and shows that they can perform the dialogueKT task when given enough training data. While DKT-Sem still outperforms existing KT methods, it does so by a smaller margin than on CoMTA, indicating DKT-Sem has its greatest advantage when little training data is given. However, LLMKT still maintains a large advantage over all other KT methods. This result suggests that even on large-scale datasets, LLM inference over prior dialogue history and textual content of KCs can still significantly improve KT performance, even when labeled student turn correctness data is abundant.

Finally, we see that performance of all KT methods is relatively low on the dialogueKT task, with a maximum of around 76% AUC for student dialogue turn correctness prediction. This result is in stark contrast to results on the standard KT task, where state-of-the-art KT methods can surpass 80% AUC for question response correctness prediction on some datasets. This observation suggests that dialogueKT is a challenging task, due to various reasons: First, dialogues can be relatively short, giving very little context for KT methods to leverage. For example, dialogues in CoMTA are cut short on purpose, with only 4-5 labeled turn pairs on average, to serve the goal in [165] on evaluating how accurate LLMs are at math. Therefore, we call upon researchers who have access to dialogues with more turns to study these methods on their (possibly proprietary) data using our publicly available implementation. Second, student behavior in dialogues is more unpredictable compared to traditional KT, since there can be many diverse types of student responses to tutor turns. We believe that these challenges make dialogueKT a task worth studying in future work; it is likely that new KT methods need to be developed to improve performance on this task.

Speaker	Turn Text	KCs and Predicted Masteries
Tutor	Point X is at (-4, 6) and point Z is at (6, -2). Determine the coordinates of point Y on line segment XZ such that XY is 3 times as long as YZ. What do you think is the important information here?	1) Use a pair of perpendicular number lines to define a coordinate system. 0.4688 2) Apply the Pythagorean Theorem to find the distance between two points. 0.4688 3) Recognize and represent proportional relationships between quantities. 0.5622 4) Find the point on a line segment between two points that partitions the segment in a given ratio. 0.6225
Student	Firstly, I need to determine the distance between point X and Z. That would be the square root of 164, which is roughly 12.8 ✓	
Tutor	Excellent beginning! You've calculated the total length of line segment XZ. Now, how can we use this information to determine the coordinates of point Y? Keep in mind, XY is 3 times as long as YZ.	1) Recognize and represent proportional relationships between quantities. 0.3208 2) Find the point on a line segment between two points that partitions the segment in a given ratio. 0.3486
Student	I would divide 12.8 by 3 total parts. ✗	

Table 3.3: An example dialogue with student turn correctness labels, KC labels, and LLMKT's predicted mastery level for each KC. LLMKT accurately predicts student correctness at both turns by leveraging the difficulty of the tutor-posed questions.

3.6.2 Qualitative Analysis: Case Study

We now detail findings from our qualitative analysis, where we seek to i) identify challenges in the dialogueKT task, ii) understand why LLMKT has an advantage over existing KT methods, and most importantly, iii) show how LLMKT can be useful to tutors in practice. Therefore, we perform a case study using a dialogue in the test set of CoMTA.

We identify two primary challenges in the data that are unique to the dialogueKT task and are difficult for prior KT methods. First, the open-ended nature of dialogues means that student behavior is much less structured than in traditional assessment or

practice settings. For example, when the tutor poses a question to the student, the student often i) does not answer directly but instead asks a clarifying question, ii) skips ahead to the final solution in the case of step-by-step problem solving, or iii) asks the tutor to give them a different problem. These student actions are inherently difficult to anticipate ahead of time since they define behavior beyond what can be captured by the KCs. As a result, there are many turns in these dialogues that are "irrelevant" to the KT task.

Second, since dialogues can be short (around 4 turn pairs on average in CoMTA), there can be little historical context to estimate student knowledge from. In many dialogues, students are proficient in one aspect of problem solving, responding to several questions correctly at first, but then struggle when the tutor moves on to a new KC. Without sufficiently long student history, it is difficult to estimate KC mastery levels for ones that are seen for the first time. This problem is partially due to existing datasets not linking students over multiple dialogues.

We find that, unsurprisingly, LLMKT's advantage over existing KT methods comes from leveraging nuanced details in the dialogue, rather than solely relying on previous correctness and KC labels. For the most part, changes in predicted mastery levels correlate with previous turn correctness; as a result, dialogues with many correct or incorrect turns in a row are easy to predict for all KT methods. LLMKT excels in cases when it relies on textual information, rather than correctness labels, to adjust its KC mastery estimates. Table 3.3 shows one such example: At the first turn, the tutor asks the student for relevant information regarding the problem, and the student responds correctly by applying the Pythagorean Theorem, which the model accurately predicts. However, after the tutor asks the student to find the coordinates of Y, the student incorrectly wants to divide the segment by 3 rather than 4, which the model also accurately predicts. The trained model seems to have estimated that the second tutor question is more difficult than the first, using the exact textual content of the

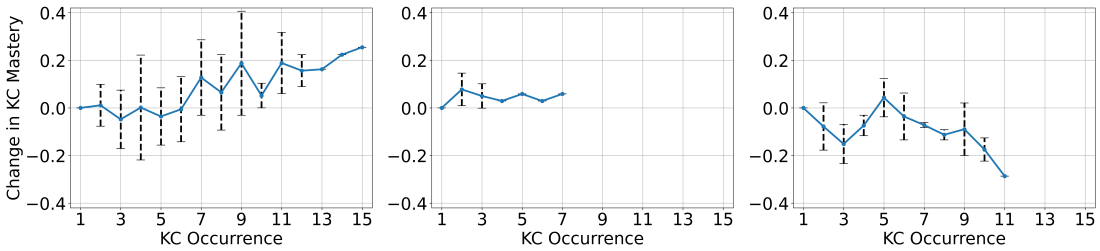


Figure 3.2: Knowledge change curves, i.e., estimated KC mastery level changes across dialogue turns, averaged over students for selected KCs in the CoMTA dataset. More common KCs in longer dialogues tend to show increasing mastery levels as the dialogue progresses, while less common KCs tend to show the opposite. KCs shown: "Solve linear equations and inequalities in one variable," "Understand that polynomials form a system analogous to the integers," and "Apply the properties of operations to generate equivalent expressions."

tutor question, even though they have overlapping KC labels. This correct prediction is not possible without the question parsing ability enabled by using an LLM as the underlying KT model.

We also found that LLMKT tends to predict low mastery when a KC is seen for the first time in a dialogue, possibly through observing that students do not react well to the tutor shifting to new KCs mid-way through the dialogue. This observation also suggests that LLMKT can effectively use dialogue context to detect when new KCs are introduced. Overall, these observations suggest that there are numerous advantages in exploiting the textual content in dialogues for the dialogueKT task, rather than just relying the correctness and KC labels; LLMKT seems to be an encouraging step in this direction for future KT methods.

3.6.3 "Learning Curve" Analysis

A common method for assessing the quality of KCs and KT methods is examining how well they match pedagogical or cognitive theory, and in particular, the power law of practice [219]: as students practice a particular skill, their mastery will increase rapidly at first and then more slowly over time. Following prior work [214], we visualize

changes in predicted mastery levels at the KC-level, examine trends in the curves, and investigate whether there is evidence of the power law of practice. We use LLMKT's predicted masteries on CoMTA for all dialogues when they are in the test split of the cross-fold experiment. We use CoMTA instead of MathDial for this analysis since the latter uses LLMs to simulate students; it is unlikely for simulated students to behave like real ones, which we found in a preliminary analysis.

For a KC c, we first find each time that KC occurs in the data, use $c_j^{(i)}$ to denote its jth occurrence in the ith dialogue, and use $\hat{z}_j^{(i)}$ to denote the corresponding predicted KC mastery level. We then subtract $\hat{z}_1^{(i)}$ from each $\hat{z}_j^{(i)}$ to track the *change* in predicted mastery from the first prediction, since different students may have different initial knowledge levels. We plot the average KC mastery level, i.e., $\frac{1}{N}\sum_{i=1}^{N} \hat{z}_j^{(i)} - \hat{z}_1^{(i)}$, where N is the number of dialogues that have at least j turns that have c, and show the standard deviation using error bars.

We examine plots of the 15 most frequent KCs since less common KCs do not have enough data to show clear trends. Out of these, the results are mixed: 5 out of 15 KCs trend towards increasing mastery as the dialogues go on, 5 KCs trend towards decreasing mastery, and 5 KCs do not change significantly in mastery. Figure 3.2 shows a representative example of each case. KCs with an upward trend typically have few changes in their masteries for the first few turns, as well as relatively high variance. Additionally, these KCs are more prevalent in longer dialogues and correspond to the more commonly occurring KCs in the data. These patterns can be explained by two concurrent trends in the data: First, there are several long dialogues where a student struggles with a math topic at the beginning, getting some questions correct and others incorrect, but after some time becomes more proficient and consistently responds correctly; these dialogues contribute to the steady increase in KC mastery. Second, there are several short dialogues where the student either simply knows the topic and consistently responds correctly, or struggles with the topic and responds

incorrectly; these dialogues contribute to the high variance and the minimal change in early turns.

On the contrary, KCs with a downward trend or no trend at all tend to be associated with shorter dialogues and occur in fewer dialogues. These dialogues tend to feature a student repeatedly trying to solve a problem without any success, with the dialogue often ending immediately after a correct solution is reached. KC mastery can also drop significantly at the beginning of the dialogue once the student starts to respond incorrectly. Since there are few dialogues to average over, these downward trends can dominate the knowledge change curves; the most common KC with a downward trend only appears in 8 dialogues, which is less than most KCs with upward trends.

Overall, when we take into account the small amount of data and how dialogues in the CoMTA dataset are truncated, the knowledge change curve trends seem to mostly resemble the power law of practice; when there are sufficient turns in a dialogue, the student's predicted mastery increases before leveling out eventually. The main reason these trends are not more prevalent is likely due to these dialogues typically ending after only a few turns. This setting is different than the problem-solving setup mostly used in past KT research, where students practice a KC repeatedly by solving different problems, even after mostly mastering it. Again, we call upon researchers who have access to data with longer dialogues to conduct more extensive analyses.

3.7 Annotation Quality Experiments

3.7.1 Human Evaluation

To quantify GPT-4o's accuracy on the correctness and KC labeling tasks, we conduct an IRB-approved human evaluation using expert human annotators. We hired 3 former math teachers at $40 US/hour, each rating the same set of 30 randomly selected dialogues from CoMTA with GPT-4o's labels, totaling 166 turn pairs. For each turn in each dialogue, we ask annotators to score GPT-4o's label i) on correctness,

Label	Score	Overlap	α	GPT-4o Final Turn Acc.	Human Final Turn Acc.
Correctness	0.9317 / 1	0.8434	0.1809	75.82 (83.33)	85.56
KCs	3.2831 / 4	0.3494	0.4383	–	–

Table 3.4: Scores and IRR metrics for the human evaluation. Additionally, final turn correctness accuracy for GPT-4o (on the full dataset followed by only on the human evaluation dataset) and the human annotators.

as 1 if they think the label is accurate and 0 otherwise, and ii) on the standards selected by GPT-4o on a 1-to-4 scale with 4 as the highest. We provide the exact instructions given to annotators in the online Supplementary Material[4]. For both correctness and KCs, we report the average human-assigned **Score** and two inter-rater reliability (IRR) metrics: i) the portion of times that the labels of all 3 annotators are exactly the same (**Overlap**) and ii) the Krippendorff's $\boldsymbol{\alpha}$ coefficient [45]. We also compute the accuracy of correctness predictions at the final turn using CoMTA's ground truth labels for both GPT-4o and our annotators.

Table 3.4 shows the human evaluation results. We see that our annotators give GPT-4o very high scores for correctness and moderate-to-high scores for KCs, indicating that GPT-4o is able to competently perform these tasks. IRR is volatile, with high overlap and low α on correctness, and low overlap but moderate α on KCs. However, the low α on correctness is expected given the high class imbalance of the data (more than 90% of labels are 1), and the low overlap on KCs is expected since there are 4 classes and exact ratings can be subjective. While GPT-4o's accuracy on final turn correctness is lower than the correctness score might suggest, this is expected since the final turns are much harder to compute correctness on without subsequent tutor turns to help indicate the label. As reference, the CoMTA tech report [165] reports final turn correctness evaluation with GPT-4o at similar accuracy (78.3). Finally, given that

[4] `https://osf.io/wrkej?view_only=f32472a6560649e692a4e2ba603ef52c`

the average human annotator's final turn correctness accuracy is only slightly higher than GPT-4o's, we can conclude that the correctness labeling task is a challenging one where GPT-4o achieves close to human-level performance.

3.7.2 Error Analysis

To understand when GPT-4o makes errors in labeling correctness and KCs, we qualitatively analyze its annotations. When labeling correctness, most of GPT-4o's errors are on the final turn, especially in questions that require some numerical calculations to verify correctness, such as when the student provides the numerical result of a sum or product calculation. This is a known limitation in GPT-4/4o [209] and can possibly be improved through external tools such as calculators. The other common error tends to be when GPT-4o should label a turn as "na" but does not, e.g., when the tutor asks the student whether they know how to use a function on their calculator. GPT-4o also sometimes labels a turn as correct when the tutor asks a student if they know a concept and the student indicates they do not, showing that GPT-4o can be confused by the tutor's intentions.

When labeling KCs, GPT-4o's primary error is not assigning sufficient standards to a turn to describe all the required KCs. This error is mostly due to not selecting enough relevant domains and clusters early in the annotation process, especially for ones with broad or abstract descriptions. As a result, there are not enough related standards to select from to use as KCs. On the other hand, GPT-4o occasionally assigns too many KCs to a turn, selecting all standards that are related to the topic but not necessarily only ones relevant to the current question. These errors can likely be resolved with more sophisticated prompting, potentially running the annotation process multiple times through sampling and taking a majority vote. We leave the exploration of such methods that require more API calls for future work. Finally, there are several dialogues not covered by Common Core standards; there is little

that GPT-4o can do other than select a somewhat related standard. This observation suggests that it may be necessary to augment the Common Core standards with other sources to have a more complete set of candidate KCs, or defer to humans for labeling in these cases.

3.8 Discussion, Conclusions, and Future Work

In this paper, we introduced the task of dialogueKT with the goal of estimating student knowledge from open-ended tutor-student dialogues. We used GPT-4o to automatically annotate student response correctness and relevant KCs in the dialogue turns, and showed via an expert human evaluation that these annotations are largely accurate. We introduced a new LLM-based KT method that leverages the textual content in dialogues, and showed that LLMKT i) significantly outperforms existing methods on dialogueKT and ii) works with limited training data. We also showed that our models use the textual content in dialogues to capture student knowledge in meaningful ways. There are numerous avenues for future work. First, we can jointly model tutor moves and student knowledge states to analyze what types of tutor moves are effective at improving student knowledge. Second, in addition to student knowledge, we also need to account for student affect dynamics in dialogues, possibly with the help of other data modalities such as audio and video. Third, the scope of dialogueKT can be expanded upon: in this work, we only focus on math dialogues between a single student and tutor, but future work can extend to other subjects and dialogues involving multiple students. Fourth, there are many potential applications of dialogueKT that can be explored. Future work should investigate whether estimated knowledge states are useful for personalization, providing teacher support, or can even be used to train AI-based tutors by powering simulated student agents. Overall, we hope that this initial investigation can broaden the scope of student modeling to conversational settings, which have become of significant interest lately.

CHAPTER 4

TRAINING LLM-BASED TUTORS TO IMPROVE STUDENT LEARNING OUTCOMES IN DIALOGUES

4.1 Introduction

Recent advances in generative artificial intelligence (AI), including large language models (LLMs), have opened new possibilities in education and in particular on scaling up personalization. One form of personalization that generative AI powers is interactive learning via *tutoring dialogues* between AI-powered tutors and students. These interactions have the potential to tailor instruction to each student's needs and progress, while offering personalized feedback, all in real time, in a scalable way. Given the widespread success of human tutors for improving student outcomes [173], many recent works have developed LLM-based tutors, showing promise across various educational domains [86, 134, 176, 185, 189, 211, 226, 256]. Many LLM-based tutors are even deployed in practice, such as Khan Academy's Khanmigo [110] and Carnegie Learning's LiveHint [44]. Several preliminary studies have shown that interacting with LLMs can increase student learning [269], although some have shown that students can develop an over-reliance on LLMs which negatively impacts their learning [117].

Many prior works have focused on improving LLMs' ability to follow effective tutoring principles, adapting them for the tutoring task that they are not pre-trained for. One approach, explored in [240], analyzes the decision-making process underlying human tutor utterances, showing that integrating expert decisions enhances LLM-based tutoring. Another study, [152], examines tutor moves in interactions with an LLM-powered simulated student agent, demonstrating that move annotation data

contributes to better tutoring performance. Similarly, [83] investigates the role of AI roleplay in generating synthetic tutoring data and finds that fine-tuning LLMs on this data, along with human tutor-student interactions, significantly improves their pedagogical effectiveness. Moreover, [62] introduces the concept of tutor uptake—acknowledging student responses—as a valuable strategy for LLMs to adopt.

While these efforts offer valuable insights into how LLMs can emulate effective human tutoring strategies, the question remains whether such approaches truly maximize student learning outcomes. Rather, a data-driven approach, where student outcomes form a reward signal, could potentially lead to AI tutors that are more aligned with educational goals. Additionally, current approaches often rely on large proprietary LLMs, coming with many downsides; they are not fully controllable and cannot be easily customized, can be costly to use, and relinquish control of private student data. Therefore, developing effective AI tutors with smaller, open-source LLMs remains an important goal.

4.1.1 Contributions

In this paper, we propose a novel approach to train a small, open-source LLM to not only follow effective pedagogical principles, but directly optimize for student learning outcomes. Our method performs three key steps. First, at each dialogue turn, we gather multiple candidate tutor utterances from a variety of sources, including human tutors and LLMs with varying styles and sizes. Second, we evaluate each candidate utterance on two aspects: (1) whether it elicits a correct response in the next student turn, using a trained student model for dialogues to predict student behavior, and (2) whether it adheres to a set of effective pedagogical principles, using GPT-4o in a LLM-as-a-judge evaluation setup. Third, we contrast good candidate utterances with poor ones, and fine-tune Llama 3.1 8B [85] with offline Reinforcement Learning (RL), specifically Direct Preference Optimization (DPO) [197]. We demonstrate that

our optimized LLM tutor significantly increases the likelihood of next turn student response correctness, while reaching comparable pedagogical quality to that of a much larger, proprietary LLM, GPT-4o [178]. Through qualitative analysis and human evaluation, we confirm that our approach produces high-quality tutor utterances and reveal emergent tutoring strategies that arise from our training approach.

We acknowledge up front that the most significant limitation of our work is that we do not experiment with real students. Since access to students at the scale necessary for our work is beyond our capability, we use a simulated student model instead. While we believe that our work is a reasonable starting point to train LLM-based tutors to maximize student outcomes, future work with real students in the loop is highly important. Therefore, to facilitate further research, we publicly release our code[1] and encourage researchers and practitioners with access to real-world tutoring dialogue settings to build on our work.

4.2 Related Work

4.2.1 AI Tutors in Dialogues

There is a long history of AI-based tutors in education that interact with students through dialogues. Early systems, such as Cognitive Tutors, construct cognitive models of students to provide targeted feedback [12]. AutoTutor engages students by asking targeted questions, and assesses student correctness using latent semantic analysis [84]. Why2-Atlas converts a student response to a proof, which it uses to identify misconceptions and guide a dialogue [235]. While these systems were often effective for improving student learning, they required significant engineering and had limited flexibility. In contrast, recent LLM tutors can more easily adapt to new contexts, interpret student responses, and cater personalized content towards the

[1] `https://github.com/umass-ml4ed/tutorbot-dpo`

student. Several LLM tutors are implemented through refined prompt engineering [110], with some taking on specialized roles such as teachable agents [211] or "co-pilots" for human tutors [241]. Other works fine-tune LLM tutors to enhance the pedagogical capabilities over the base models. A common approach is to generate simulated dialogues, where the tutor utterances are constructed to follow good pedagogical practice, and fine-tune on those [223, 83]. Several works also generate examples of low quality tutor utterances and use them as negative samples in DPO training to improve over fine-tuning [18, 224].

4.2.2 Student Outcome Modeling

While there are many ways of measuring student outcomes, in this work we focus on the well-studied setting of *next item correctness*. Student modeling in this setting is typically handled by knowledge tracing (KT), where a binary correctness is predicted for the next item a student attempts based on the student's history so far [56]. KT models have used recurrent neural networks [194], self-attention networks [78], and, more recently, LLMs [58, 139]. A recent work introduces LLMKT [205], an LLM-based model that predicts *next turn* student correctness in dialogues. Therefore, we leverage LLMKT to predict student outcomes in this work. Similar to our work, prior works have used RL to discover teaching policies with rewards derived from student models [89, 198], including KT-based student models [41]. Another recent work used LLM estimates of student post-test scores to refine math worksheets [87]. However, to the best of our knowledge, ours is the first to do so in the context of tutoring dialogues.

4.2.3 Evaluating Pedagogical Quality of LLMs

To validate LLM tutors, we need to be able to evaluate them along pedagogical measures. Typically, researchers construct a pedagogical *rubric*, which defines multiple properties that tutor utterances should follow. Rubric-based evaluation of generated utterances is then performed by human experts [98, 83, 240] or by LLMs [103, 209].

In this work, we design a pedagogical rubric and primarily use LLMs to evaluate tutor utterances, but also humans at a smaller scale to ensure our results are reliable. Similar to [209], we also use LLM-assigned rubric scores to form DPO preference pairs.

4.3 Methodology

We now detail our methodology to generate tutor dialogue utterances to maximize student learning outcomes. Figure 4.1 shows an overview of our approach with an example, where the problem is from the MathDial [152] dataset. In this scenario, a student has a misconception, missing the "twice" part of the problem, and provides an incorrect answer. We generate multiple, diverse candidate versions of the next tutor utterance to follow the student's response, using a collection of LLMs with different sizes and styles. Then, among the generated utterances and the human tutor utterance, we construct preference pairs to fine-tune an LLM using DPO. We consider an utterance to be preferred if it (1) likely elicits a correct student response and (2) follows good pedagogical principles. The former criterion employs a student model, LLMKT [205], to predict whether the student will correctly respond to the tutor in their next turn. The latter criterion employs a set of rubric items to evaluate whether a tutor utterance follows good pedagogical principles. Before detailing each component, we define some key notations: a dialogue d consists of an alternating sequence of tutor and student turns, $d = (t_1, s_1, \ldots, t_M, s_M)$, where M is the number of turn pairs, indexed by m, and the textual content of each turn is the "utterance".

4.3.1 Student Outcome Prediction

We now detail the model we use for predicting student outcomes in dialogues. We leverage recent work on knowledge tracing (KT) in dialogues, LLMKT [205], an LLM-based model that predicts if a student will respond correctly to a tutor-posed task in their next turn, given dialogue history and the knowledge components

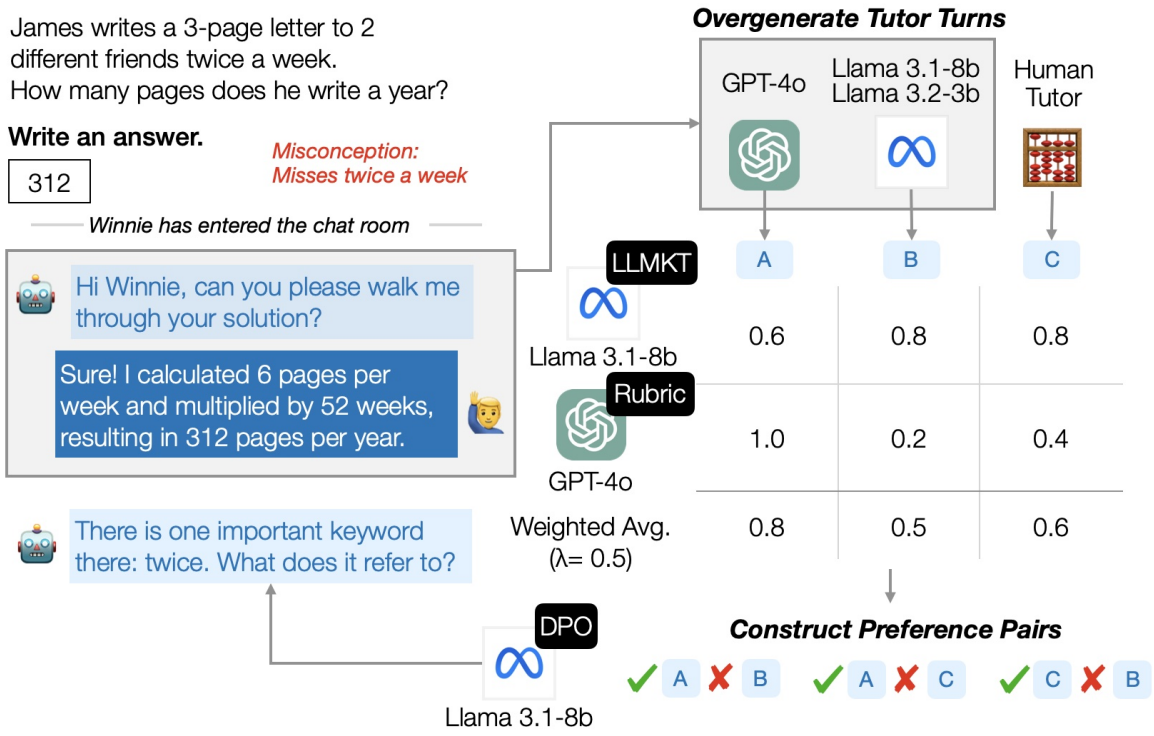


Figure 4.1: An overview of our methodology for training LLMs to generate tutor utterances with the joint objective of maximizing student learning outcomes and following good pedagogical principles. *Human tutor* refers to tutor utterances in MathDial.

(KCs) [55] embedded in a tutor turn. The model is highly accurate at predicting student correctness, achieving 0.76 AUC on the MathDial test set, making it a reliable automated source for estimating student outcomes.

In LLMKT, at the m-th dialogue turn for the student, the model analyzes the conversation history $t_{\leq m}, s_{<m}, \mathcal{C}_m$, which denote the tutor utterances up to and including this current turn, prior student turns, and the set of KCs involved in the tutor-posed task in the current turn, respectively. It estimates student knowledge levels on each KC, and combines these estimates to predict whether the student will respond to the tutor-posed task in the current turn correctly. Therefore, we leverage LLMKT as a *student simulator* and use it to evaluate whether a generated tutor utterance at a dialogue turn will elicit a correct response from the student.

Criteria	Explanation
Accuracy	Ensuring the response does not contain false or misleading statements.
Progress	Determining whether the response helps the student move forward.
Guidance	1. Error identification: Correctly pinpoints the student's mistake. 2. Strategic Hinting: New information or guidance for help. 3. Withholding: Refrains from directly providing the final answer. 4. Encouraging: Motivates the student to persist in their attempt.

Table 4.1: Pedagogical evaluation criteria for tutor utterances in dialogues.

4.3.2 Following Pedagogical Principles

In addition to promoting correct student responses, we also hand-craft a set of rubric items for effective pedagogical principles, listed Table 4.1. We then employ GPT-4o to assess how well the generated tutor utterances align with the rubric. By incorporating pedagogical evaluation aspects rather than focusing solely on correctness prediction, our approach discourages oversimplification of tutor utterances and instead encourages utterances that provide meaningful guidance.

Our evaluation draws inspiration from feedback assessment studies [98, 103, 227] and focuses on common errors made by LLMs when generating feedback for math problems [209]. The rubric evaluates generated tutor utterances on six granular items, each assigned a binary label, across three core aspects: *Accuracy*, *Progress*, and *Guidance*. Considering all aspects, we also have GPT-4o provide an *overall score* for the utterance on a 1-10 scale. Our prompt leverages chain-of-thought so that GPT-4o provides reasoning about the utterance before assigning its scores.

We include the human tutor utterance from MathDial in our prompt as a point of comparison, mainly to assist with evaluating *Accuracy*, which can be challenging [209]. We find the 1-10 scale works slightly better than simply averaging all the binary rubric items, possibly because it enables GPT-4o to decide which rubric items are more relevant given the context of the dialogue.

4.3.3 Preference Pair Construction

To create a dataset of preference pairs for DPO training, we collect candidate tutor utterances from four sources: (1) the human tutor utterances from the MathDial dataset, (2) utterances generated by GPT-4o, where the evaluation criteria (rubric and intending to elicit correct student responses) are included in the prompt, (3) utterances generated by Llama 3.1 8B using a generic prompt about behaving like a math tutor (without the rubric), and (4) utterances generated by Llama 3.2 3B using the same generic prompt. In general, method (2) contains high-quality tutor utterances that serve as positive examples that score high under our rubric. However, we found that in many cases (1) performs better for eliciting correct student responses due to the concise nature of the utterances. Utterances generated by (3) are a mix of high and low quality, while utterances generated by (4) are typically lower quality and serve as a source of negative examples for preference optimization. Prior work has shown diverse candidate quality, particularly negative examples, to improve DPO performance [209, 252].

After evaluating each candidate using LLMKT and the rubric, we create a combined weighted score for a candidate tutor utterance at turn m:

$$s_m = \lambda \cdot y_m + (1 - \lambda) \cdot r_m, \tag{4.1}$$

where y_m is the probability of a correct student response predicted by LLMKT at this turn, and r_m is the overall rubric score at this turn normalized in $[0, 1]$. We adjust λ to balance the tradeoff between how much the generated utterances elicit correct student responses compared to how much they follow pedagogical practice. By default, we set $\lambda = 0.5$ to achieve a balance between these objectives; in our experiments, we show how varying λ affects the balance between both.

We then use the score s_m to construct preference pairs between candidate tutor utterances. We consider a candidate with score s_m^1 to be preferred over a candidate

with score s_m^2 if the former is greater by some threshold, i.e., $s_m^1 - s_m^2 > \epsilon$. If the scores are within ϵ, we do not form a preference pair. In practice, we set $\epsilon = 0.1$ to achieve a balance where noisy preference pairs are excluded, but we retain enough data to sufficiently train the model, both of which have been shown to be important considerations for DPO [112].

4.3.4 Model Training

We train our model in a two-stage process: (1) distillation and (2) DPO. Distillation is a common way to enhance the capabilities of small LLMs by mimicking the behavior of much larger LLMs [257]. In our case, we fine-tune Llama 3.1 8B on candidate tutor utterances generated by GPT-4o. Through this distillation stage, we gain access to a *local* model that scores well on our pedagogical rubric.

We then use DPO and our preference pairs to further steer the distilled model towards effective tutoring. DPO trains an LLM by contrasting outputs in a preference pair given the same input prompt, using the following objective:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,t_m^w,t_m^l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(t_m^w|x)}{\pi_{\text{ref}}(t_m^w|x)} - \beta\log\frac{\pi_\theta(t_m^l|x)}{\pi_{\text{ref}}(t_m^l|x)}\right)\right], \tag{4.2}$$

where t_m^w and t_m^l represent the preferred and unpreferred tutor utterances, respectively. x represents the input prompt, comprising an instruction, the context of the dialogue, and the dialogue history $t_{<m}, s_{<m}$. π_θ denotes the model being trained, π_{ref} represents a frozen reference model, and β is a hyperparameter that controls the divergence between the learned and reference policies.

In this work, we use the distilled model as the reference model π_{ref} and for initializing the weights of π_θ. We find the distilled model works much better than using the base Llama model, since the distilled model already performs better on the rubric. Additionally, we set $\beta = 0.1$, a relatively low value for DPO compared to a common value of $\beta = 0.5$. The low β value allows π_θ to diverge more from π_{ref}, which we find

necessary to increase LLMKT's predictions of eliciting correct student responses in the next dialogue turn.

4.4 Experimental Settings

4.4.1 Dataset

We evaluate our framework using the MathDial dataset [152], which consists of tutoring dialogues focused on mathematics problems from GSM8K [54]. Each dialogue is centered around an incorrect student solution to the math problem, and the goal of the dialogue is for the tutor to guide the student to the correct solution by addressing their misconceptions. Tutors are role-played by crowd workers, while students are simulated by GPT-3.5. Despite only being half-authentic, MathDial is the largest publicly available one-on-one tutor-student math dialogue dataset to the best of our knowledge. To estimate correctness with LLMKT, which requires knowledge component labels at the current turn, we use the annotated knowledge components from the Dialogue KT version of the dataset [205] and filter out unlabeled turns.

We follow the original MathDial train/test split. After filtering, our test set has 588 dialogues with 3,101 tutor turns. We split the train set into a 80/20 train/validation split at the dialogue-level, resulting in 1,809/453 dialogues with 11,058/2,811 tutor turns. When creating our overgenerated tutor turn dataset for distillation and DPO training, we take a subset of the train/validation split to reduce labeling costs, resulting in 483/135 dialogues with 3,080/920 GPT-4o-generated tutor utterances for distillation and 9,662/3,095 preference pairs, with our default parameters of $\lambda = 0.5$ and $\epsilon = 0.1$, for DPO.

4.4.2 Baselines

We compare our preference-optimized LLM, which we refer to as **DPO**, with Llama 3.1 8B as the base model, against the following baselines: the **Base model** of

Llama 3.1 8B prompted with our evaluation criteria (rubric and intending to elicit correct student responses); its supervised fine-tuned version on human tutor utterances in the original dataset, **SFT**; a fine-tuned version distilled from GPT-4o-generated utterances detailed above in Section 4.3.4, **Distill**; **GPT-4o**, the large, proprietary LLM, which is prompted with our evaluation criteria; and finally, the **Human Tutor** utterances from the original MathDial dataset.

4.4.3 Automated Metrics

We evaluate tutor utterances on both **student outcomes** and **pedagogical principles**. Student outcome prediction uses LLMKT to estimate the probability of a correct student response, averaged across turns. Evaluating how well the utterance follows the pedagogical principles will be reported in the same order as in Table 4.1, reporting scores for **Acc.** (Accuracy), **Prog.** (Progress), **Err.** (Error Identification), **Hint** (Strategic Hinting), **Wth.** (Withholding), and **Enc.** (Encouraging), along with an **Overall** score.

4.4.4 Model Parameters

We use the `meta-llama/Meta-Llama-3.1-8B-Instruct` model from Hugging Face [71] for all local experiments. To adapt the model, we use Low-Rank Adaptation (LoRA) [93] with a rank parameter of $r = 64$, scaling factor $\alpha = 32$, and a dropout rate of 0.05. We train using the AdamW optimizer, with a learning rate of 1×10^{-4} for distillation and 3×10^{-5} for DPO, with a linear warmup phase for the first 10% of training steps. We use an effective batch size of 64 with gradient accumulation, set weight decay to 1×10^{-2}, and set a gradient clipping maximum norm of 1.0. We evaluate the loss on the validation set after each epoch, and achieve the minimum validation loss after three epochs for distillation and after one epoch for DPO. At test time, we generate tutor utterances using greedy decoding. We conduct all experiments on NVIDIA RTX A6000 GPUs.

Method	Student Outcomes	Pedagogical Principles						
		Acc.	**Prog.**	**Err.**	**Hint**	**Wth.**	**Enc.**	**Overall**
Human Tutor	0.45	**0.99**	0.66	0.58	0.49	<u>0.97</u>	0.53	6.97
GPT-4o - Proprietary LLM								
Base Model	<u>0.49</u>	**0.99**	**0.97**	**0.92**	**0.96**	**0.99**	<u>0.89</u>	**9.40**
Llama 3.1 8B Instruct - Open-source LLM								
Base Model	0.43	0.82	0.69	0.70	0.64	0.94	0.62	7.20
SFT	0.47	0.86	0.32	0.23	0.21	0.90	0.31	4.73
Distill	0.47	0.95	0.92	<u>0.90</u>	<u>0.91</u>	**0.99**	0.82	8.93
DPO ($\lambda = 0.5$)	**0.65**	<u>0.97</u>	<u>0.96</u>	**0.92**	**0.96**	**0.99**	**0.92**	<u>9.37</u>

Table 4.2: Tutor utterance evaluations for each method on the test set. The best result for each metric is in **bold** while the second best is <u>underlined</u>. DPO significantly outperforms all methods on improving student correctness predictions, and closely matches the performance of GPT-4o on the pedagogical rubric.

4.5 Experimental Results

4.5.1 Quantitative Results

Table 4.2 shows the results of tutor utterance generation on the MathDial test set, across all methods on the automated metrics. We observe that DPO significantly outperforms all other methods on student correctness prediction, improving over the next best method, GPT-4o, by 33%. This result shows that DPO training is necessary to generate tutor utterances that are more likely to achieve correct student responses, which cannot be achieved simply through prompting. Additionally, DPO achieves similar scores to GPT-4o on the pedagogical rubric, with both methods scoring very high on all rubric items and the overall score. This result shows that our DPO training pipeline can effectively bring the pedagogical ability of small, open-source LLMs with only 8B parameters to the level of very large, proprietary LLMs. Moreover, we see that DPO improves over distillation on almost all metrics, showing that DPO is necessary to achieve both objectives.

A notable observation is that human-written tutor utterances, while almost always accurate, score relatively low on most other rubric items. This result is not surprising

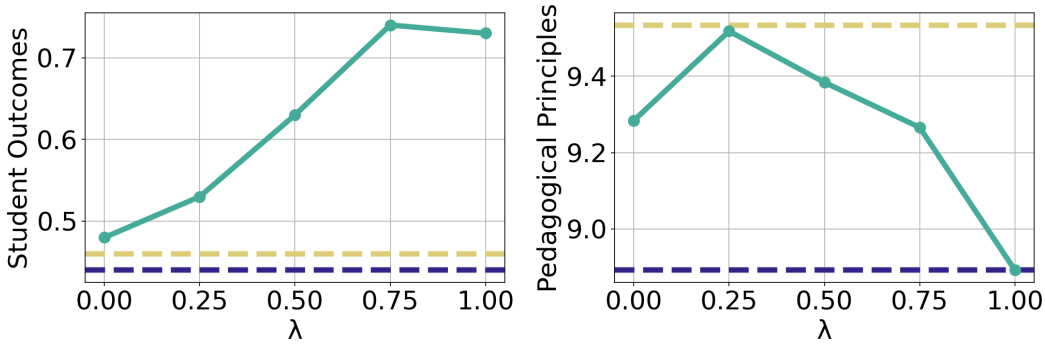


Figure 4.2: Results of our experiment adjusting the value of λ in DPO training. Left: student correctness prediction changing with λ. Right: pedagogical rubric score changing with λ. We also show values for GPT-4o and the Distill model for comparison.

since human tutors do not act according to our evaluation rubric; many of the human utterances are relatively short and simply ask the student to retry without giving guidance, and occasionally feature the tutor getting frustrated with the student. As a result, the SFT model, trained on human tutor utterances, does not score high on our rubric; the distilled model and even the base Llama model significantly outperform it on the rubric items.

Student Outcomes vs. Pedagogical Principles We investigate how adjusting the value of λ can balance the two goals of maximizing student outcomes under LLMKT and following pedagogical principles. We vary the value of $\lambda \in \{0.00, 0.25, 0.50, 0.75, 1.00\}$, where larger values attribute more weight to the student correctness prediction objective, and evaluate on a subset of the data with 500 turn pairs. Figure 4.2 shows the result of this experiment. We see that, as expected, increasing λ from 0 to 1 generally increases student correctness prediction performance while generally decreasing the rubric score. Practitioners can get their desired balance by changing λ; for example, a value of around 0.5 to 0.75 can maintain a relatively high score on the pedagogical rubric while significantly improving performance on soliciting correct responses from students. Perhaps surprisingly, changing λ from 0.75 to 1.0 does not increase student outcome performance, while decreasing λ from 0.25 to 0.0 *decreases* pedagogical per-

formance. This result implies that a small reward signal from correctness prediction may implicitly help pedagogical effectiveness, and vice versa.

4.5.2 Qualitative Analysis

We conduct a qualitative analysis of generated tutor utterances to investigate the strengths and weaknesses of our method, how it compares to baselines, and what types of patterns emerge in the model-generated text.

Table 4.3 shows an example dialogue context and tutor utterances from the human tutor, GPT-4o, and DPO. We see that the human tutor states that the student is not correct and prompts the student to try again, providing a subtle hint to focus on the cost of the dozen. However, this utterance neither gives LLMKT, the student model, any confidence in the student being able to answer correctly, nor follows strong pedagogical principles. GPT-4o guides the student toward correctly calculating the profit, but directly provides half of the solution in its hint, limiting deep student engagement. It also poses the challenging task of computing the total profit, which requires multiple calculation steps. Since the student is still struggling with unit conversion, LLMKT is not confident that the student will respond correctly. In contrast, our DPO-trained model provides an actionable hint for the student, following the rubric. It also poses the simpler task of asking the student to first find the cost of a half dozen, which is necessary for later steps. As a result, LLMKT is confident the student can respond correctly.

Overall, we find that our DPO-trained model excels at posing *concrete* tasks that are nontrivial, but more feasible for students compared to GPT-4o. We observe that the behavior of posing tasks this way is the primary way of achieving higher predictions for student correctness, and appears to be an emergent behavior of training on the LLMKT-derived scores. While the tasks sometimes only ask the student to perform simple arithmetic, they are typically more involved. For example, in another dialogue,

the model asks "How long does it take to download 200 GB at a rate of 2 GB/minute?" This task requires the student to set up a single variable equation and understand the roles of the given values. We also find that the DPO-trained model is more likely to ask questions, whereas other methods often directly tell the student to perform a task.

However, both LLMs still have some limitations, particularly when students give partially correct responses. In these cases, the models may ignore the progress the student has made and focus on errors that are irrelevant to the recent student response. Correctly identifying errors in math with LLMs has been shown to be challenging in prior work as well [76, 138], indicating additional techniques may be needed to fix this behavior.

4.5.3 Human Evaluation

We also conduct a human evaluation to further assess different tutor models and how our automated metrics align with human judgment.

We recruit two independent volunteer university students to annotate tutor utterances according to our evaluation metrics. We randomly sample 10 dialogues from the test set after filtering based on toxicity and low quality ground-truth tutor utterances. The annotators evaluate 5 consecutive tutor turns from each dialogue, resulting in a total of 50 evaluation instances. At each instance, we show participants the dialogue so far and next tutor utterance candidates generated by (1) human tutors, (2) GPT-4o, and (3) our DPO-trained model, in random order without revealing the method.

For student outcome prediction, annotators rank the three candidate utterances based on how likely they think each one will lead to a correct student response. We ask annotators to take into account both the task posed in the turn and the student's knowledge demonstrated in prior turns. We use rankings instead of absolute values since the latter may be harder for humans to calibrate. For pedagogical principles, we

Problem			
Chloe bought chocolate-dipped strawberries at $50 a dozen. She then sold them for $30 for half a dozen. How much is Chloe's profit if she sold 50 dozens?			
Dialogue History			
t_1	Hey Alejandra. If Chloe buys the strawberries at $50 for 12 and sells them for $30 for 6, how much profit would that be for one dozen?		
s_1	For one dozen, Chloe would make a profit of $20, which is $50 - $30 = $20.		
t_2	Remember that she's selling half a dozen for $30. How many half dozen can she sell from each $50 dozen she buys?		
s_2	She can sell two half dozen for each $50 dozen she buys, so she can make a profit of $60 from each $50 dozen.		
Next Tutor Turn (t_3) Candidates			
Method	**Tutor Utterance**	**SO**	**PP**
Human Tutor	Remember it's not $60 profit, because they cost her to buy them. Try again.	0.33	4
GPT-4o	It looks like there's a misunderstanding in your calculations. Remember, Chloe's cost for half a dozen is $25 (since $50 per dozen divided by 2). She sells half a dozen for $30, so her profit per half dozen is $30 - $25 = $5. Try recalculating her total profit using this correct profit per half dozen.	0.59	**10**
DPO	It looks like there's a misunderstanding. If Chloe buys a dozen for $50 and sells half a dozen for $30, let's first find out how much she paid for half a dozen. If $50 is for 12, how much is $50 divided by 2?	**0.84**	**10**

Table 4.3: Comparing human tutor, GPT-4o, and DPO tutor utterances, on student outcomes (SO) and pedagogical principles (PP). Brown highlights the tutor-posed task. Some irrelevant text in the dialogue context has been removed for brevity.

simply ask annotators to follow the same evaluation rubric outlined in Table 4.1 and provide an overall score on a 1-10 scale.

Results Table 4.4 shows average student correctness ranks (rank 1 means top-ranked) and rubric scores from human annotators. We see that DPO outperforms both human tutors and GPT-4o on both metrics, with all differences statistically significant ($p < 0.05$) according to a paired t-test. Higher correctness ranks show that DPO learns how to pose tasks that are manageable to students, while the higher rubric score shows that it does so while maintaining pedagogical principles. Additionally, the higher rubric score may imply that the correctness objective also indirectly improves

Method	Correctness Rank$^{\downarrow}$	Rubric Score$^{\uparrow}$
Human Tutor	2.12	7.36
GPT-4o	2.13	8.07
DPO	**1.75**	**8.55**

Table 4.4: Human evaluation shows that DPO outperforms human tutors and GPT-4o.

pedagogical quality. While DPO outperforming GPT-4o on the rubric is contrary to the automated results in Table 4.2, the automated results may reflect self-bias from GPT-4o as the evaluator [253].

Since these evaluation tasks can be highly subjective, we also investigate the inter-rater agreement between the two human annotators and the automated metrics. We use Kendall's τ [108] for correctness ranks and Pearson's correlation coefficient ρ [127] for rubric scores. For correctness ranks, Kendall's τ is only 0.06 between the two annotators and averages 0.08 between LLMKT and both annotators. This low agreement is not surprising due to the subjective nature of the task; we find that there are many near-ties between different candidate utterances according to LLMKT's output probabilities. For rubric scores, the inter-rater agreement is notably higher: Pearson's ρ is 0.15 between the two annotators and averages 0.27 between GPT-4o and both annotators, the latter being statistically significant. This result suggests that evaluating pedagogical principles is much easier than predicting student correctness, though the former is still subjective, since annotators may value certain rubric items differently.

Overall, despite showing some promise through preliminary evaluation, it is important to deploy our trained tutor LLMs to real students and test whether they actually lead to good student learning outcomes.

4.6 Conclusions and Future Work

In this paper, we introduced a methodology to train LLM tutors to maximize student outcomes in dialogues while maintaining high pedagogical quality. We use student simulation to predict how likely a tutor utterance will yield a correct student response, and use GPT-4o to evaluate the pedagogical quality of tutor utterances using a rubric. We use both sets of predictions to score overgenerated tutor utterances and fine-tune Llama 3.1 8B with direct preference optimization. Our resulting model significantly outperforms other methods for increasing predicted student outcomes, and matches the performance of GPT-4o, a much larger closed model, on pedagogical aspects. There are many avenues for future work. First, an evaluation with real students should be carried out to determine if our methods are still effective in real-world scenarios. Second, future work should investigate how to optimize for longer-term learning outcomes, such as concept mastery or performance on post-dialogue assessments. Third, future work should include estimates of student affect and engagement as part of the reward. Finally, future work should apply our method to dialogues in non-math domains, such as language learning or computer science.

CHAPTER 5

SIMULATED STUDENTS IN TUTORING DIALOGUES: SUBSTANCE OR ILLUSION?

5.1 Introduction

Private tutoring has been shown to be highly effective for helping students learn. Naturally, with the emergence of powerful large language models (LLMs), significant attention has been put on how to effectively use LLMs as tutors. Many LLM-based tutors have already been deployed by learning platforms and companies [110, 44, 181, 83, 15], aiming at scaling up the impact of on-demand, real-time tutoring.

Efforts in developing LLM-based tutors can be broadly categorized into two types: alignment-based and student-based. The first type relies on aligning LLMs with well-established pedagogical principles, especially on how to respond to student utterances during tutoring dialogues to maximize learning and engagement [110, 208, 224]. The second type resorts to training, often via reinforcement learning (RL), with students in-the-loop; since doing RL training with real human students at large scale is difficult, existing works either ask experts to role-play students [83] or prompt LLMs to simulate students [64, 87, 131, 152, 274].

However, both of these student simulation approaches have obvious limitations: the former is still not scalable, while the latter requires LLM-based students to behave realistically. Many recent works have shown that LLMs cannot reliably simulate student behavior simply through prompting [158, 223]. Moreover, it is challenging to make LLM-based simulated students follow cognitive characteristics of real human students, like the power law of practice [205, 244], when learning. In addition to these

well-known shortcomings, there do not exist many evaluation metrics for realistically simulating students in dialogues, particularly at the turn-level; recent works focus on linguistic features at the population-level [193], or consistency with synthetic profiles at the dialogue-level [143]. However, it is difficult to determine if systems developed using simulated students are reliable without thoroughly verifying the simulated students themselves.

5.1.1 Contributions

In this paper, we explore the task of creating more realistic LLM-based simulated students using a two-stage approach. First, we identify six high-level dimensions, identified by the learning sciences research community, that define student behavior in dialogues: 1) dialogue acts, 2) correctness, 3) error-making, 4) knowledge acquisition, 5) language use, and 6) tutors' responses. For each dimension, we develop reference-based automated evaluations to measure the realism and faithfulness of a simulated student response with respect to a ground-truth response. Second, we benchmark performance on a set of strong simulated student methods, including several prompting-based methods, supervised fine-tuning (SFT), and one using multi-objective RL. To the best of our knowledge, our work is the first to 1) develop reference-based metrics for realistic student simulation in dialogues, and 2) examine the ability of LLMs to replicate real student turns in dialogues. We conduct extensive experiments on a real-world dataset with 2,000 dialogues between human tutors and students. Using both automated metrics and human evaluations, we find that: (various styles of) prompting often fail to capture most dimensions of student behavior, SFT significantly improves the alignment of generated student utterances with these dimensions, and RL leads to further improvements. We further show that our automated metrics

show strong agreement with human experts. We make our code and data annotations publicly available[1] to facilitate the further development of simulated student models.

5.2 Related Work

5.2.1 Student Simulation in Dialogues

Many recent works have explored LLM-based student simulation in educational dialogues, mainly using prompting and conditioning on student personas. MathDial introduces a dialogue dataset with LLM-simulated students exhibiting specific misconceptions [152], while other work uses zero-shot prompting to generate student responses for the purpose of downstream tutor training [64]. Several works simulate students by conditioning LLMs on cognitive states, misconceptions, and prior knowledge, enabling diverse simulated interactions with tutoring agents [137, 237, 238]. Other works use LLMs to simulate student learning curves and realistic misconceptions [100, 211], behavior that corresponds to different demographic, behavioral, or personality traits [157, 143, 131], and goals and learning trajectories [212]. One recent work leverages a multi-agent LLM pipeline to refine cognitive and linguistic properties of student responses, and performs DPO on resulting data [43]. Another work leverages machine unlearning to remove knowledge of specific concepts from LLMs in order to study how they relearn these concepts through dialogue tutoring [220]. Finally, recent works have also used SFT to create simulated students, finding that this approach significantly improves authenticity [193, 43]. In general, dialogue-based simulated students are typically developed for the purpose of generating data for training and evaluating LLM tutoring agents [64, 208, 238], or creating training environments for human tutors [186, 157].

[1] `https://github.com/umass-ml4ed/sim-student-eval`

However, these works tend to focus on simulating entire dialogues that are not grounded in real student data. In contrast, our work focuses on predicting student utterances at the turn-level, which we validate by comparing against real student data. While some prior works explicitly predict turn-level knowledge and correctness of student turns [205, 96], they do not predict the full text of student utterances, therefore missing many other aspects of student behavior.

5.2.2 Evaluating Simulated Students

Despite growing interest in student simulation, hardly any existing work thoroughly evaluate the quality of simulated student methods and how closely do they resemble real human students. A recent survey identifies a lack of validation in many works, and proposes a "Turing-like test" to capture realism [107]. A recent work uses an LLM-conducted Turing test to measure simulated student realism, although it remains to be seen if LLMs can identify simulated students as well as humans, or if the results of the test are correlated with more reliable measures of simulation realism [65]. A concurrent work identifies the "competence paradox", where LLMs are tasked with simulating learners that are intrinsically less knowledgeable, and argues that simulated students should be constrained by definitions of student behavior and knowledge [261]. Another recent work conducts a comprehensive analysis of simulated student realism, where expert teachers interact with simulated students, and identifies a wide range of behavioral dimensions that indicate realism and reveal weaknesses in current approaches [158]. The identified realism indicators include error patterns, consistency in knowledge acquisition, question-asking behavior, and linguistic patterns. However, all of these indicators are evaluated using human experts, leaving the question of how to *automatically* evaluate simulated students unanswered.

A common automated evaluation approach is measuring the consistency between simulated outputs and the personas they were generated on, typically via LLM-as-a-

judge, used both in education [143, 250, 131] and other domains [4, 119, 111]. However, these evaluations assume access to a ground-truth persona and are often conducted in fully synthetic settings. Alternative approaches use human-evaluated realism scales [152, 158, 43], alignment between simulated performance and conditioned cognitive states [206, 237, 31], accurately predicting correctness [205], and textual similarity between real and simulated student solutions [204, 66]. Finally, several works measure population-level statistics of simulated student properties, such as temporal error rates [244] or length and frequency of student turns [193, 274], and compare against statistics measured on real student interactions.

Our evaluation metrics build on many of the ideas used in these prior works. By comparing simulated turns to ground-truth ones, we effectively measure the consistency of the simulated student. We explicitly model many of the aspects identified as important in prior works, including correctness, errors, knowledge, behavior (through dialogue acts), and similarity to real student utterances (through cosine similarity and ROUGE-L). Additionally, our work is the first to establish a framework for reference-based evaluation of simulated students in dialogues, complementing prior works that focus on fully synthetic dialogue generation and others that focus on reference-based evaluations in non-dialogue settings.

5.2.3 Student Simulation in Other Settings

Beyond dialogues, student simulation has been studied in problem-solving and assessment settings. A large body of work has studied student "misconceptions", particularly in math, typically fine-tuning LLMs to reliably predict errors that students will make when solving problems [203, 76, 221, 202, 59]. Other works have fine-tuned or prompted LLMs to simulate student responses to exam questions, often for the purpose of evaluating question properties such as difficulty [31, 206, 172, 97, 88, 87, 148]. Another recent area of interest is studying student behavior in programming settings,

typically by fine-tuning LLMs on real or simulated student traces [204] and conditioning on student knowledge states [66]. Earlier non-LLM approaches modeled student problem-solving behavior and adaptation to tutor feedback in math learning platforms [159], with a focus on modeling the cognitive states of students [12].

5.3 Methodology

We now detail our notations, methodology for simulating students in dialogues, and evaluating the faithfulness of these simulations. A dialogue $d = (s_0, t_1, s_1, \ldots, t_M, s_M)$ is defined as an alternating sequence of student turns, s_i, and tutor turns, t_i, where M is the number of *turn pairs* in the dialogue; s_0 is present if the student initiates the dialogue, and s_M is present if the student ends the dialogue. For a dataset of N real tutor-student dialogues, $\mathcal{D}$, d^n denotes the n-th dialogue. We refer to the textual content of a turn as an "utterance". Dialogues are often grounded in a question, q, that the student is attempting and the tutor is guiding them through. In these cases, the question is associated with a set of *knowledge components (KCs)*, C, i.e., skills that the student must possess to answer the question correctly.

5.3.1 LLM-based Student Simulation

We simulate realistic student behavior by using an LLM, $\mathcal{M}_S$, to generate a predicted student utterance, $\hat{s}_i$, at each turn i in the dialogue, conditioned on the current dialogue history and any other relevant context. Formally, this process is summarized as $\hat{s}_i \sim \mathcal{M}_S(s|s_{<i}, t_{\leq i}, q, P)$. P represents a prompt, which includes instructions on simulating student behavior, but can also include additional context on the student, such as a *persona*, if available. $\mathcal{M}_S$ can be a pre-trained LLM, but can also be fine-tuned using SFT on a dataset of existing student utterances, minimizing the negative log likelihood of student utterances, i.e., $\mathcal{L} = -\sum_{n=0}^{N}\sum_{i=0}^{M} \log \mathcal{M}_S(s_i^n|s_{<i}^n, t_{\leq i}^n, q^n, P^n)$.

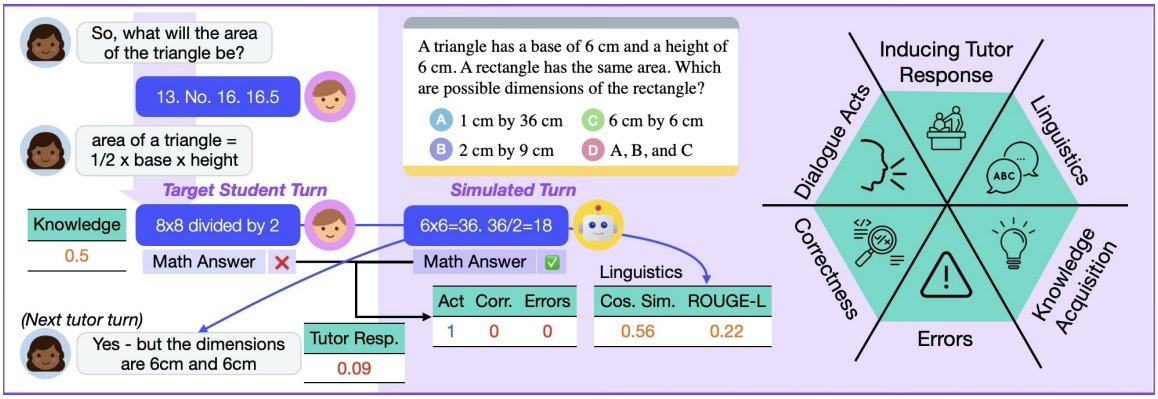


Figure 5.1: Overview of the seven evaluation metrics for simulated student turn evaluation, with a real paraphrased tutor–student dialogue serving as the reference. In this example, the ground-truth student turn and the simulated turn have the same dialogue act *Math Answer*; however, the target turn is incorrect while the simulated turn is correct.

We can further refine $\mathcal{M}_S$ using RL in order to optimize for specific properties of student behavior, which we detail later.

5.3.2 Evaluating Simulated Student Turns

We ground our evaluation metrics in several high-level measurable properties of student utterances across six dimensions. These metrics cover important aspects of students, such as behavior, knowledge, and language use, that enable us to make meaningful comparisons between real and simulated students. Several of the metrics rely on ground-truth labels for real student turns; we use LLM annotation to provide these labels, via GPT-4.1 [179]. We show the agreement between the labels and human experts in Table 5.7. Finally, several metrics require fine-tuned LMs, as detailed below.

5.3.2.1 Dialogue Acts

Our first student behavior measure is dialogue acts, a widely studied approach for classifying actions taken in dialogues. We develop a list of five student dialogue acts in Table 5.1, rooted in prior work [234, 92, 163]. We define a_i as the act that the student takes in turn s_i and $\hat{a}_i$ as the act in the simulated student turn $\hat{s}_i$. We define

the similarity to be binary-valued, i.e., 1 if $a_i = \hat{a}_i$ and 0 otherwise. We prompt an LLM to provide the ground-truth act labels. To reduce the costs of our evaluation metric, we then fine-tune a local LLM, $\mathcal{M}_{\text{act}}$, to classify dialogue acts, trained on the ground-truth act labels. At evaluation time, we use this model to classify the acts of simulated student turns, i.e., $\hat{a}_i \sim \mathcal{M}_{\text{act}}(a|\hat{s}_i, s_{<i}, t_{\leq i})$.

Dialogue Act	Description
Math Answer	When the tutor asks a math content-related question, the student attempts to answer that question.
Not Understanding	The student simply indicates that they do not know the answer to a question or do not understand a concept.
Seek Information	The student seeks more information regarding the math problem or topic, for example, by asking a clarifying or conceptual question.
Off-Topic	The student utterance is unrelated to the problem or math topic, including greetings, goodbyes, and other casual conversation.
Acknowledge	The student simply acknowledges what the tutor said in the previous turn.

Table 5.1: Dialogue acts that students can make at any turn in a dialogue.

We show our dialogue act definitions in Table 5.1. To derive our set of acts, we started with definitions from prior work [234, 92, 163] and then refined them to fit our setting of one-on-one math tutoring and to retain a level of granularity that balanced low ambiguity and high behavioral descriptiveness. We then internally labeled acts and compared with LLM labels. Afterward, we further refined our definitions to address areas of disagreement and confusion. For example, we originally included an "Other" act, but found that we could achieve better labeling accuracy by explicitly labeling each utterance's act. We also originally included distinct acts for correct and incorrect math answers, but collapsed those to avoid overlapping with the correctness metric.

5.3.2.2 Correctness

Another important aspect of student behavior is whether they are responding correctly to questions (or tasks) raised by the tutor during the dialogue, who often "scaffold" problem solving by asking small sub-questions [19]. Concretely, we define y_i as the correctness of s_i; $y_i = 1$ if s_i is a correct response to the question in the tutor turn t_i, and $y_i = 0$ if it is an incorrect response. Additionally, $y_i = na$ if the tutor does not ask a question in t_i; these turns include off-topic ones or simple utterances that keep the dialogue flowing. Similarly, $\hat{y}_i$ indicates the correctness of the simulated student turn, $\hat{s}_i$. When $y_i \neq na$, we consider the correctness similarity to be 1 if $y_i = \hat{y}_i$ and 0 otherwise. When $y_i = na$, we do not measure correctness similarity. We use an LLM to provide ground-truth correctness labels. We find that classifying correctness by fine-tuning an LLM is much more challenging than doing so for dialogue acts. Instead, we prompt a reasoning-based LLM, GPT-5 mini [180], with low reasoning effort, to evaluate the correctness of simulated student turns, including the ground-truth turn as reference.

5.3.2.3 Errors

In addition to correctness, it is also important to understand the *errors* that students make when they give incorrect answers [113]. We define e_i as the error in the ground-truth student turn and $\hat{e}_i$ as the error in the simulated student turn. If the ground-truth turn has an error, we set the error similarity to 1 if $e_i = \hat{e}_i$ and 0 otherwise. We do not evaluate similarity if the ground-truth turn is not incorrect. In practice, we only need to test error equivalence, avoiding the challenge of classifying errors. Therefore, we prompt GPT-5 mini to determine if two turns have the same error. We perform correctness and error evaluation in a single prompt to reduce costs.

5.3.2.4 Knowledge Acquisition

In order for simulated students to be useful for modeling learning outcomes, they need to follow *knowledge acquisition* patterns that are similar to real students [244]. We formalize knowledge acquisition using knowledge tracing (KT) [56], where knowledge is represented as mastery of KCs. Following [205], a framework for KT in tutoring dialogues, we use an LLM-based model to predict student correctness in subsequent turns. Formally, we define $C_i \subseteq C$ as the set of KCs relevant to the tutor turn t_i, which we identify using LLM prompting.

Using these KC labels and the correctness labels, as detailed above, we train a model, $\mathcal{M}_{\text{KT}}$, to estimate a student's current *knowledge state*, which represents the student's current mastery over all KCs in C. We achieve this by training the model to predict if the student will correctly answer the *next* tutor-posed task correctly, conditioned on the dialogue so far. Formally, the estimated probability of a correct answer is $\frac{1}{|C_{i+1}|}\sum_{k=1}^{|C_{i+1}|} z_{i+1,k}$, where $z_{i+1,k} = \mathcal{M}_{\text{KT}}(c_{i+1,k}|s_{\leq i}, t_{\leq i})$ represents the student's estimated mastery of the KC $c_{i+1,k}$.

To measure the *acquisition* of knowledge, we define $\nabla Z_i = \{z_{i,1} - z_{i-1,1}, \ldots, z_{i,|C|} - z_{i-1,|C|}\}$, i.e., how much the student's mastery changes for each KC since the last turn. We similarly compute the change in knowledge for the simulated student turn, $\nabla \hat{Z}_i = \{\hat{z}_{i,1} - z_{i-1,1}, \ldots, \hat{z}_{i,|C|} - z_{i-1,|C|}\}$, where $\hat{z}_{i+1,k} = \mathcal{M}_{\text{KT}}(c_{i+1,k}|\hat{s}_i, s_{<i}, t_{\leq i})$. Since the range of these values can be highly dependent on $\mathcal{M}_{\text{KT}}$ and the data it is trained on, we compare the quantile buckets of mastery deltas rather than the raw values themselves. Specifically, we compute 5 equal sized mass-based quantile bins for all ∇Z_i, where $\text{quant}(z) \in \{0, \ldots, 4\}$ returns the quantile index. We then define the similarity between ∇Z_i and $\nabla \hat{Z}_i$ as $1 - \sum_{k=1}^{|C|} \frac{|\,\text{quant}(\nabla Z_{i,k}) - \text{quant}(\nabla \hat{Z}_{i,k})|}{4|C|} \in [0, 1]$, i.e., the inverted average distance between the delta quantiles. We provide a step-by-step computation of knowledge acquisition similarity in Table 5.10 to illustrate the intuition for this metric.

5.3.2.5 Linguistics

Prior works have found that LLM-based simulated students often fail to reproduce basic linguistic patterns of real students [158]. Therefore, we employ existing text-based similarity metrics, specifically the cosine similarity between the text embeddings of the simulated and ground-truth utterances, which measures their semantic similarity, and ROUGE-L [135], which measures word-level recall. Both of these measures capture many aspects of linguistic patterns in student utterances, such as specific word use, utterance length, and sentiment. We use the `Qwen3-Embedding-8B` model [273] in the cosine similarity calculation.

5.3.2.6 Inducing Tutor Responses

Finally, we measure how well student turns fit into the natural progression of a dialogue. We can examine if the simulated student response is likely to induce the actual response from the tutor, assuming that the tutor would respond similarly if the predicted student utterance is similar to the actual one. Therefore, we leverage the likelihood of the *ground-truth, next tutor turn* conditioned on the simulated student utterance in this turn. We fine-tune an LLM tutor model, $\mathcal{M}_T$, with SFT on the ground-truth tutor utterances and calculate the inverse perplexity of the tutor turn. Formally, this metric is defined as $\exp\big(-\frac{1}{|t_{i+1}|}\log P_{\mathcal{M}_T}(t_{i+1}|\hat{s}_i, s_{<i}, t_{\leq i})\big)$.

5.3.3 Tuning with RL

Given these automated evaluation metrics, we can create a *reward function* to provide real-time feedback on simulated student responses to improve them via RL. Specifically, we take the average of all metrics to form our final reward, which is possible, since all metrics are in the $[0, 1]$ range; this simple aggregation can form a Pareto-optimal policy on convex Pareto fronts [136], and we leave more complex aggregations for future work. In this work, we use offline RL rather than online algorithms to reduce computational burden.

We use a simple four-stage pipeline for offline RL training: we 1) generate n candidate student responses for each dialogue turn in the train set using an SFT-ed student model, 2) evaluate each candidate student response on each of our evaluation metrics, 3) form preference pairs between all possible pairs of the n candidate responses for each turn, where a response is preferred if it has a higher average score across metrics, and 4) train the student model on the resulting preference pairs using direct preference optimization (DPO) [197]. In practice, we employ two techniques for eliminating noisy preference pairs. First, following [206], we only form a pair if the score difference is greater than a threshold, ϵ. Second, we do not train on any of the first 5 turns in a dialogue, since we find that student behavior is highly uncertain early in dialogues, resulting in noisy reward signals.

We note that the purpose of our RL pipeline is to explore the potential of leveraging our metrics to improve simulated student realism. However, we caution that training on LLM-evaluated metrics can lead to reward hacking, where the trained model can learn to overfit to errors in the evaluations, potentially causing them to become unreliable. For this reason, we encourage future works to carry out expert human evaluation in situations when evaluation metrics are leveraged during training.

5.4 Experiments

We now detail our experimental setup to 1) benchmark a variety of student simulation methods on a real-world dataset and 2) validate our automated evaluation metrics using human evaluation.

5.4.1 Dataset

We conduct our experiments on Question-Anchored Tutoring Dialogues 2k [265], the largest publicly-available dataset of real student-tutor dialogues from the [69] learning platform. Each dialogue centers around a middle school student solving a

math multiple-choice problem, where a trained tutor guides the student through the process via online chat. Problems span a wide range of topics, including Algebra, Geometry, and Number Sense. After processing, the dataset contains 1,529/382 dialogues in the pre-defined train/test split, respectively. We further split the train set into a train/validation set with 1,147/382 dialogues, respectively. We train all models on the train set, use the validation set for hyperparameter tuning, and show results on the test set.

The dataset is completely anonymized, and as a result there is no demographic information available on individual students or tutors. All students are from the United Kingdom. The dataset is licensed using cc-by-nc-sa 4.0, and we release our annotations under the same license.

Dialogues are 23.42 turns long on average across student and tutor turns, with a minimum length of 10 and a maximum of 109. Dialogues can be initiated by tutors or students, with 82.63% of dialogues initiated by tutors. Student/tutor turns are 4.11/14.84 words long on average, respectively, with both containing a mixture of English, numbers, mathematical symbols, and emojis. We show the distribution of act labels in Figure 5.2 and the distribution of correctness labels in Figure 5.3.

Since solutions are not provided in the dataset, we use GPT-4.1 to annotate each question with the correct answer, a textual solution, and an explanation for each multiple-choice option. We also ask the model to identify if the question is solvable, since several questions are associated with images that were not properly translated to text or contain other data processing issues. We exclude such unsolvable questions from the dataset, resulting in 60 dialogues (3% of the total) being removed.

Each dialogue is associated with a set of "subjects", which we use to form the set of knowledge components (KCs) C for our knowledge acquisition metric. We only use the most granular level of subject definitions (level 3 in the dataset). We also include an additional "Default" KC for each dialogue that the LLM can assign to turns

that may not fall into the given subjects. Not including Default, there are 157 unique subjects in the dataset, with each dialogue being associated with an average of 4.57 subjects.

In rare cases, GPT-4.1 fails to annotate dialogues with acts or correctness. Act annotation failures occur 2/1/1 times in the train/validation/test splits, respectively, and correctness annotation failures occur 1/2/2 times in the train/validation/test splits, respectively. In these cases, we simply do not include turns in failed dialogues when computing metrics that rely on the ground-truth labels, at both train and test time.

5.4.2 Simulated Student Benchmarks

We now detail the methods that we benchmark for the student simulation task, including fine-tuning and prompting approaches.

5.4.2.1 Fine-Tuning

We use both `Llama-3.1-8B-Instruct` and `Llama-3.2-3B-Instruct` [85] to examine the impact of model size, and train all models using LoRA [93]. We perform standard **SFT** on all student turns in the train set, as detailed above in Section 5.3.1. We then further perform **DPO** training as detailed above in Section 5.3.3.

5.4.2.2 Prompting

One of the most common approaches for simulating students is to prompt pre-trained LLMs to behave like real students. Therefore, we test a variety of prompting approaches to see how they compare to fine-tuning. Unless otherwise specified, we prompt GPT-4.1 for each of the following methods. We provide an example persona, summary, and dialogue retrieval in Table 5.11.

5.4.2.2.1 Zero-Shot We simply instruct the LLM to behave like a real student with a few simple guidelines for student behavior, adapting the prompt from [64] with minimal changes.

5.4.2.2.2 OCEAN Persona In prompting, we can provide a *persona* to the LLM, instructing it to behave like a student while following the traits described in the persona [87]. We use OCEAN (i.e., "Big Five") personas [161], which are commonly used in human user simulation [143, 111]. Specifically, we first prompt an LLM to estimate each of the five OCEAN traits as "low", "neutral", or "high" given the full, ground-truth dialogue. We then provide this estimated persona in the simulated student prompt. We note that this approach does leak information from the ground-truth. If prior student dialogues are available, we can use them to estimate the persona; however, they are not available in the dataset we are using.

5.4.2.2.3 Oracle As an upper bound for prompting-based methods, we investigate how well prompted models can simulate student behavior given a summary of the current dialogue. Specifically, we first have an LLM summarize the student's behavior in the dialogue, including OCEAN traits and learning patterns. We then provide this summary in the simulated student prompt, revealing the student's behavior ahead of time.

5.4.2.2.4 In-Context Learning (ICL) We also test ICL where an example dialogue is given in the prompt. Adopting the approach in [125], we use dialogue summaries to retrieve examples. We first encode the Oracle summaries using `Qwen3-Embedding-8B` and then select the dialogue in the training set whose summary is closest to the summary of the current dialogue to be evaluated. Similar to OCEAN, this approach technically leaks information from the ground-truth and prior dialogues from the same student should be used, if available.

5.4.2.2.5 Reasoning Finally, we experiment with a reasoning LLM, GPT-5 mini, with medium reasoning effort. In addition to instructions from the Zero-Shot prompt, we include descriptions of the evaluation criteria that the response will be judged on, letting the model reason about how to generate student-like utterances.

5.4.3 Models and Hyperparameters

We use `meta-llama/Llama-3.1-8B-Instruct` as the base model for all fine-tuned models, except in the experiments that use a smaller student model, where we use `meta-llama/Llama-3.2-3B-Instruct`. We load all models in floating point 16 precision. For LLM prompting, we use the `gpt-4.1-2025-04-14` version of GPT-4.1 and the `gpt-5-mini-2025-08-07` version of GPT-5 mini.

We conduct preliminary hyperparameter exploration using the validation set, optimizing for our automated metrics with the student models. We use the same hyperparameters, except for epochs (shown in Table 5.2), across all fine-tuned models. We train using the AdamW optimizer, with learning rate = $5 \cdot 10^{-5}$ with a linear warmup for 10% of training steps, effective batch size = 64 via gradient accumulation, weight decay = $1 \cdot 10^{-2}$, and gradient norm clipping = 1.0. We set LoRA's rank r = 32, α = 64, and dropout = 0.05, with rank stabilization [104] and adapters on all internal weight matrices. For DPO, we set the learning rate to $5 \cdot 10^{-6}$, $\beta = 0.1$, the number of candidate student utterances per turn to $n = 4$, and the reward threshold to $\epsilon = 0.1$. For DPO, we only train on a random 20% of the dialogues in the train set for one epoch to reduce costs, and find performance to be similar to when training on the full set. In total, we form 4,998/4,703 pairs for 8B/3B DPO training, respectively. For all methods, we exclude all prompts longer than 6,000 characters at train-time to avoid memory issues.

At test-time, for all non-reasoning models, we use greedy decoding and set the maximum number of generated tokens to 400; for reasoning models, we use a tem-

perature of 1.0 and 15,000 maximum generated tokens. For data annotation, we use 4,000 maximum generated tokens.

In Table 5.2, we report the validation loss, task-specific validation performance, and training/testing runtimes for each fine-tuned model used in our work. We do not include model loading in testing time, since it is a constant cost, and do not include training data evaluation in DPO training time since it is relative to the testing time of other models. We run all experiments on NVIDIA L40S and A40 GPUs, with each experiment running on a single GPU.

Model	Epochs	Val. Loss	Val. Perf.	Train Time	Test Time
Student Models					
Student SFT (3B)	3	1.9765	–	12	1
Student DPO (3B)	1	0.6458	–	56	1
Student SFT (8B)	3	1.8761	–	19	1
Student DPO (8B)	1	0.6426	–	81	1
Metric Support Models					
Acts (8B)	2	0.0703	0.9219	150	1
Correctness (8B)	1	0.1165	0.9164	76	1
Knowledge (8B)	3	0.6338	0.6557	128	5
Tutor (8B)	3	1.4802	–	20	2

Table 5.2: Training statistics for all fine-tuned models. Train and test time are reported in minutes. The validation performance (Val. Perf.) metric is accuracy for Acts and Correctness and AUC for Knowledge.

For the acts and correctness models, we train using a simple SFT objective where the output is the text of the corresponding label, and use greedy decoding for inference. We clarify that the correctness model is only used for the analysis in Figure 5.3, and not the correctness metric which uses LLM prompting. We implement our KT model using the LLMKT implementation from [205]. Additionally, we condition the KT model on estimated OCEAN personas, as detailed in Section 5.4.2, which increases AUC significantly from 0.5940 to 0.6557. We note that this modification does not leak information to the student model at test-time since the KT model is only used

for evaluation. We also acknowledge that the performance of the KT model may appear low; however, it is a difficult task since correctness is being predicted without observing the question the tutor asks. Furthermore, the goal of the KT model is to learn a correlation between utterances and future correctness, forming a complement to the correctness metric that compares correctness for the current turn.

5.4.4 Evaluation Metrics

We use seven total metrics to evaluate simulated student dialogue turns, as discussed in Section 5.3.2: **Acts** for dialogue acts, **Corr.** and **Errors** for the correctness and errors in student utterances, **Knowledge** for knowledge acquisition patterns, **Cos. Sim.** (cosine similarity) and **ROUGE-L** for linguistic patterns, and **Tutor Resp.** for the likelihood of inducing tutor responses.

Method	Acts	Corr.	Errors	Knowledge	Cos. Sim.	ROUGE-L	Tutor Resp.
Fine-Tuning Methods							
SFT (Llama 3.2 3B)	0.6645	0.5557	<u>0.0815</u>	0.8726	0.7331	0.3058	0.2033
DPO (Llama 3.2 3B)	<u>0.6762</u>	0.5748	0.0584	0.8745	0.7345	<u>0.3109</u>	0.2037
SFT (Llama 3.1 8B)	0.6671	0.5670	0.0661	<u>0.8766</u>	<u>0.7383</u>	**0.3212**	<u>0.2038</u>
DPO (Llama 3.1 8B)	**0.6840**	0.5761	0.0529	**0.8787**	**0.7390**	**0.3212**	**0.2039**
Prompting Methods							
Zero-Shot (GPT 4.1)	0.4998	0.5926	0.0220	0.8078	0.5460	0.1648	0.1911
OCEAN (GPT 4.1)	0.5268	<u>0.6039</u>	0.0308	0.8135	0.5739	0.1772	0.1942
ICL (GPT 4.1)	0.5085	0.5991	0.0319	0.8138	0.5919	0.1939	0.1914
Reasoning (GPT 5 Mini)	0.5755	0.5870	0.0088	0.8395	0.5992	0.2170	0.1909
Oracle (GPT 4.1)	0.5097	**0.6755**	**0.1872**	0.8063	0.6032	0.2109	0.1942

Table 5.3: Results on turn-level student utterance prediction. Prompting-based methods perform well on correctness, while fine-tuned methods perform well on acts, knowledge, linguistic similarity, and tutor responses. All methods, other than Oracle, perform poorly on error prediction. Best method is **bolded** and second best is <u>underlined</u>.

5.4.5 Human Evaluation

To gauge the validity of our dialogue turn-level student simulation performance metrics, we conduct an IRB-approved human evaluation with three evaluators experi-

enced in math teaching or tutoring, recruited from Upwork[2]. We hired four annotators and excluded the data from one due to consistently low-quality submissions. All annotators were paid $60 USD for 2 hours of work, which is significantly higher than the minimum wage in the United States. Instructions were provided in the form of slides, with separate slides corresponding to each annotation table, including Table 5.1, Table 5.4, and Table 5.5, as well as screenshots of the annotation environment. In total, we collected annotations on 190 turns across 38 dialogues, with 20 turns shared across two evaluators for inter-rater agreement.

For each dialogue, an evaluator examines five consecutive student turns in a dialogue, first for ground-truth responses and then for simulated ones. We include simulated responses from DPO trained on the 8B model, Zero-Shot, and Oracle, which cover a wide range of performance levels across the metrics. We inform the evaluator whether a turn is ground-truth, and randomly shuffle the order of simulated turns to not reveal which method produced which response. We collect annotations after the fifth turn in each dialogue to provide sufficient context. For both ground-truth and simulated turns, we ask evaluators to label the dialogue act and correctness. For simulated turns that they label as incorrect, we also ask them to evaluate if they have the same error as the corresponding ground-truth turn. Finally, for simulated turns, we ask evaluators to rate linguistic similarity to the ground-truth on a 5-point Likert scale. We compute 1) the human-evaluated scores for acts, correctness, errors and linguistic similarity, 2) agreement between these four categories and the corresponding automated metrics, 3) agreement between human labels and LLM-assigned labels for acts and correctness on ground-truth turns, and 4) agreement between evaluators on the shared set of turns.

[2]`https://www.upwork.com/`

Correctness	Description
Yes	Student correctly responds to the previous tutor turn.
No	Student incorrectly responds to the previous tutor turn, or indicates they do not know the answer.
N/A	All other cases, such as when the tutor does not ask a question or only asks a conversational question, or if the student response is purely conversational. A turn is conversational when it does not address a mathematical task posed by the tutor.

Table 5.4: Correctness, *Did the student answer correctly?*

Rating	Similarity	Description
1	Not linguistically similar	Completely different wording, structure, or response type.
2	Slightly linguistically similar	Very limited overlap (e.g., both contain a number or keyword), but overall expression is different.
3	Moderately linguistically similar	Same general type of response, but noticeable differences in wording, structure, or detail.
4	Highly linguistically similar	Very similar wording and structure, with minor differences.
5	Nearly identical linguistically	Almost the same wording, structure, and level of detail.

Table 5.5: Linguistic Similarity, *Ignoring correctness, how linguistically similar is the simulated turn to the actual turn in wording and structure?*

5.5 Results

We now detail our experimental results, including quantitative results through automated and human evaluation, a qualitative analysis of simulated student utterances, and finally an ablation study.

5.5.1 Quantitative Results

Table 5.3 shows quantitative results for turn-level student simulation. We see that fine-tuning methods generally outperform prompting methods, with significant improvements on the Acts, Knowledge, Cos. Sim., ROUGE-L, and Tutor Resp. metrics. They also perform better on Errors, with the exception of Oracle, which contains leaked information on exact errors made by students in its prompt. While prompting methods perform better on the correctness metric, this can be attributed to mostly generating correct responses, the majority class, as seen in Figure 5.3. These trends indicate that while LLM prompting can anticipate some high-level behavior of the student, fine-tuning is required to capture more nuanced details.

Different prompting-based methods have clearly different strengths and weaknesses. As expected, Zero-Shot performs the worst, showing the importance of context in the prompt. Between OCEAN and ICL, OCEAN performs better on Acts, which can be explained by the OCEAN persona containing high-level behavioral traits for the student. ICL performs better on linguistic metrics, which can be explained by the model reflecting student language patterns in the example dialogues. The Reasoning method performs significantly better than other prompting approaches on several metrics, including Acts, Knowledge, and ROUGE-L. This result shows that reasoning clearly helps, although less so than fine-tuning. Finally, while Oracle outperforms all other methods on Correctness and Errors due to information leakage in the summaries, it does not perform very well on other metrics. This result shows that LLM prompting

is highly limited for student simulation; even with a "cheat sheet" in the prompt, the model cannot outperform much smaller, fine-tuned models on most metrics.

Perhaps surprisingly, we find that DPO only slightly outperforms SFT on all metrics, with even slightly worse performance on Errors, which is possibly due to a sparse reward signal: since candidate turns are sampled from the SFT model, which does not perform well on errors either, there are not enough positive samples in the data for RL training. Similarly, we find that the larger 8B model outperforms the smaller 3B model by small but consistent margins on all metrics except errors. We postulate that this result is due to the inherent difficulty of predicting what students will do next, especially in open-ended settings like dialogues. More advanced techniques, such as reasoning, online RL, or data augmentation, may be needed to further improve the performance of RL, which are possible directions for future work.

5.5.2 Human Evaluation

Table 5.6 shows the performance of different simulated student methods using human-evaluated proxies of our evaluation metrics. Mostly, results match the patterns on automated metrics: DPO performs best on Acts and Linguistic, while Oracle performs best on Correctness and Errors. Across Acts, Correctness, and Errors, the values tend to be higher than on automated metrics, though this result is likely due to human evaluation being performed on later dialogue turns; earlier turns are more challenging, as seen in Figure 5.4. DPO also performs slightly better than Zero-Shot on Correctness, whereas on the automated metrics it is slightly worse. The likely cause is that annotators are more likely to label Zero-Shot responses as *na*, possibly due to their verbosity. Overall, these results confirm that fine-tuning leads to more realistic student simulations than prompting, although having an Oracle in the prompt helps. However, we note that even the best-performing methods are poor, although it is currently unclear what the theoretical upper bound for performance on our metrics is

(the upper bound should be less than 1 on each metric due to inherent randomness in student behavior). Therefore, it is important to study more advanced techniques for realistically simulating students and predicting student behavior in future work.

5.5.2.1 Metric and Label Agreement

Table 5.7 shows agreement-based reliability results for automated evaluation, LLM annotation, and human evaluation. For Acts, Correctness, and Errors, we report Cohen's Kappa, and for Linguistic, we report Pearson's correlation coefficient. The agreement between human-assigned scores and our automated metrics is very high across labels, indicating that our automated metrics give a reliable measure of student simulation quality. The Errors agreement is slightly lower because 1) it is only computed on incorrect turns, resulting in a smaller sample size, and 2) labels are highly imbalanced, with most simulated turns not making the same error as the ground-truth turn. There is also substantial agreement on the ground-truth turns between human-assigned labels and LLM-assigned labels for Acts and Correctness, showing that our prompting-based method for this annotation is reliable. Finally, we find that inter-rater agreement is very high for Correctness, Errors, and Linguistic. The moderate agreement for acts is primarily due to a single label, Math Answer, being selected much more frequently than the other labels (71- 83% across annotators).

Method	Acts	Corr.	Errors	Linguistic
DPO	**0.7905**	<u>0.6377</u>	<u>0.0612</u>	**0.5405**
Zero-Shot	0.6143	0.6087	0.0408	0.3155
Oracle	<u>0.6476</u>	**0.7101**	**0.2449**	<u>0.4071</u>

Table 5.6: Human evaluation results, where trends roughly match those on automated metrics.

Pairing	Acts	Corr.	Errors	Linguistic
Hum.-Metric	0.7337	0.6891	0.6127	0.7397
Hum.-Anno.	0.7993	0.7219	–	–
Hum.-Hum.	0.4978	0.7187	0.6154	0.6910

Table 5.7: Agreement between human evaluators and automated metrics, human evaluators and LLM-assigned annotations, and inter-rater agreement.

5.5.3 Qualitative Analysis

We perform a qualitative analysis of model outputs to understand the strengths and weaknesses of each simulated student method. We show example simulated utterances from each method in Table 5.9, along with scores on each metric. We show the distributions of dialogue acts and correctness for each method in Figures 5.2 and 5.3, respectively. We also break down the performance of each method over turns as the dialogue progresses in Figure 5.4.

Generally, there are clear differences between fine-tuning and prompting methods. First, we find that fine-tuned models match the linguistic style of real students much more closely. They produce much shorter outputs than prompting methods (2.28 words on average for DPO 8B vs. 10.89 for ICL and 4.11 for real students), echoing the findings in [193]. Prompting methods also consistently use typical LLM-like language features such as formal grammar and punctuation; for example, when a tutor asks "I'll leave you to enter your answer now if you're happy with this?", the Reasoning method generates "Yes, thanks — I'll enter D.", while SFT generates "yes thank you" and the real student simply writes "yes". Beyond these surface-level features, we find that fine-tuned responses follow act and correctness distributions that are much more similar to real students; prompting methods overestimate the rate of Seek Information and correct responses, and underestimate the rate of conversational acts, like Acknowledge and Off-Topic. While all methods perform poorly early in dialogues due to limited

context, prompting methods suffer much more, while fine-tuning methods perform better by leveraging what they learned from real dialogues through training.

However, there are several limitations to the fine-tuning methods as well. First, they tend to write very short responses, and rarely add in typos or excessive punctuation that are found in real student responses. Moreover, all methods struggle to adapt to individual student behavior. For example, some students give very brief responses, some are very verbose, and some convey a consistent sense of helplessness; however, model response patterns tend to be uniform across students, conveying a lack of diversity. Future work can seek to address these challenges by encouraging diverse outputs in training, or by conditioning on prior dialogues or student personas if they are available, to better capture the nuance in student utterances.

In addition to generating overly short responses, fine-tuned methods also significantly underestimate the Seek Information act, and are less likely than real students to ask questions, identified as common behavioral inconsistencies in LLMs [158]. We hypothesize that these differences may be due to using greedy decoding for generation; since a majority of real student responses are three or less words long (62%), the *most likely* response at any point in time will likely be very short. The under-representation of Seek Information further indicates that the models have not learned the nuances of when to use rare linguistic features or generate longer responses that are associated with the act. While random sampling can produce longer and more diverse responses, we found that it leads to significantly reduced performance in preliminary experiments. These findings indicate that more advanced training or decoding techniques, such as overgenerate-and-rank or MCTS, may be required to obtain longer and more complex responses while maintaining accuracy on realism metrics.

Reward	Acts	Corr.	Errors	Knowledge	Cos. Sim.	ROUGE-L	Tutor Resp.
SFT	0.6795	0.5546	0.0506	0.8723	0.7417	0.3155	0.2102
Average	**0.6962**	<u>0.5699</u>	0.0562	0.8691	0.7433	**0.3181**	0.2104
Acts	<u>0.6949</u>	0.5677	0.0506	0.8695	0.7389	0.3111	0.2101
Corr.	0.6692	**0.5852**	0.0506	0.8652	0.7275	0.3129	0.2081
Errors	0.6795	0.5524	0.0562	0.8708	0.7406	0.3122	0.2105
Knowledge	0.6846	0.5437	**0.0730**	<u>0.8756</u>	**0.7486**	<u>0.3147</u>	**0.2124**
Cos. Sim.	0.6731	0.5371	**0.0730**	0.8713	<u>0.7453</u>	0.2957	<u>0.2116</u>
Tutor Resp.	0.6897	0.5349	<u>0.0618</u>	**0.8763**	0.7447	0.3137	0.2113

Table 5.8: Results of reward function ablation study. Most reward functions result in high performance on the corresponding metric. Best method is **bolded** and second best is <u>underlined</u>.

5.5.4 Ablation Study

In order to understand how each metric is related to overall student simulation quality, we run an ablation where we train DPO with rewards from only one metric at a time. We use `Llama-3.2-3B-Instruct` for this experiment and evaluate on a random 20% split of the test data to reduce costs. We show results in Table 5.8. As expected, we see that training on a metric results in high performance on it. The exception is Errors, since the reward signal is likely too sparse to enable learning. Interestingly, training on some metrics induces very high or low performance on completely different metrics. This result may be attributed to the inherent *correlation* between some metrics. For example, training on Knowledge results in the highest performance on Errors, Cos. Sim., and Tutor Resp., yet leads to low performance on Correctness. On the contrary, training on Correctness results in very high performance on Correctness but the lowest performance for most other metrics. While most relationships between aspects of student behavior are as expected, there are several surprising ones, such as between knowledge acquisition and correctness. This result reflects recent findings that training on certain human behavioral data can have unintended effects on seemingly unrelated traits [51]. Finally, we observe that training on the Average reward generally leads to high performance across metrics, although suffers on Knowledge, Errors, and

Tutor Resp. This result suggests that while averaging is a reasonable approach, more advanced techniques for combining metrics could lead to higher overall performance.

5.5.5 Label Distribution Analysis

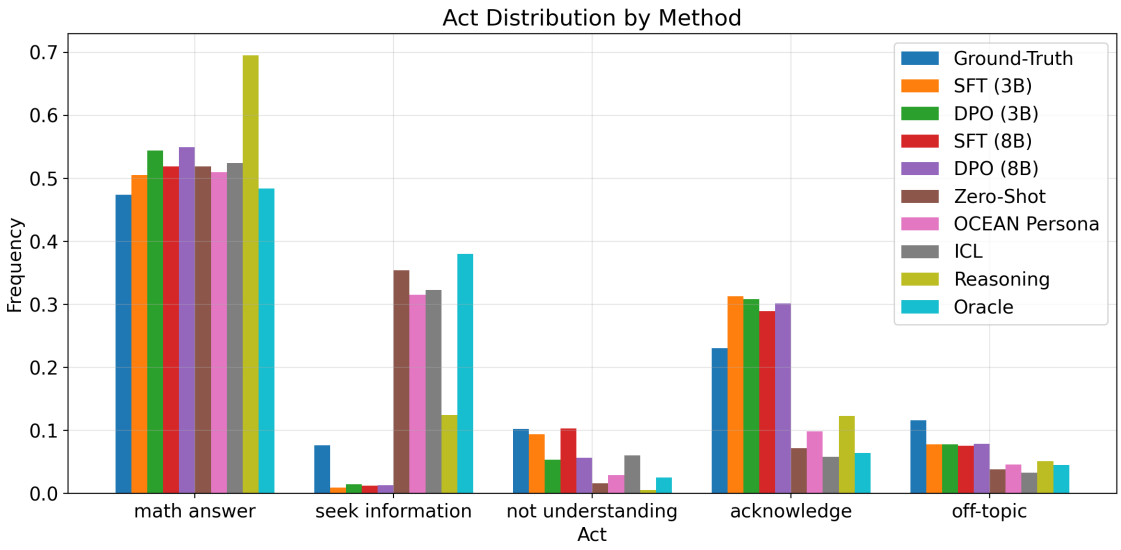


Figure 5.2: Distribution of act labels for real students and simulated student methods.

We examine the distribution of both acts and correctness to better understand the behavior of different student models. Using the full test set, for each student model, we compute the portion of turns that are classified for each possible act and correctness label, using fine-tuned act and correctness classifiers, respectively. We compare these to the distribution of acts and correctness for the ground-truth, real student turns on the test set.

We show the distribution of acts across methods in Figure 5.2. We observe that while most methods have similar frequencies of Math Answer, the distributions for other acts vary greatly. Compared to prompting-based methods, the fine-tuned methods have rates that are much more similar to the ground-truth for Not Understanding, Acknowledge, and Off-Topic. However, fine-tuned methods rarely use the Seek Information act. A possible reason for this is that the fine-tuned responses tend to be

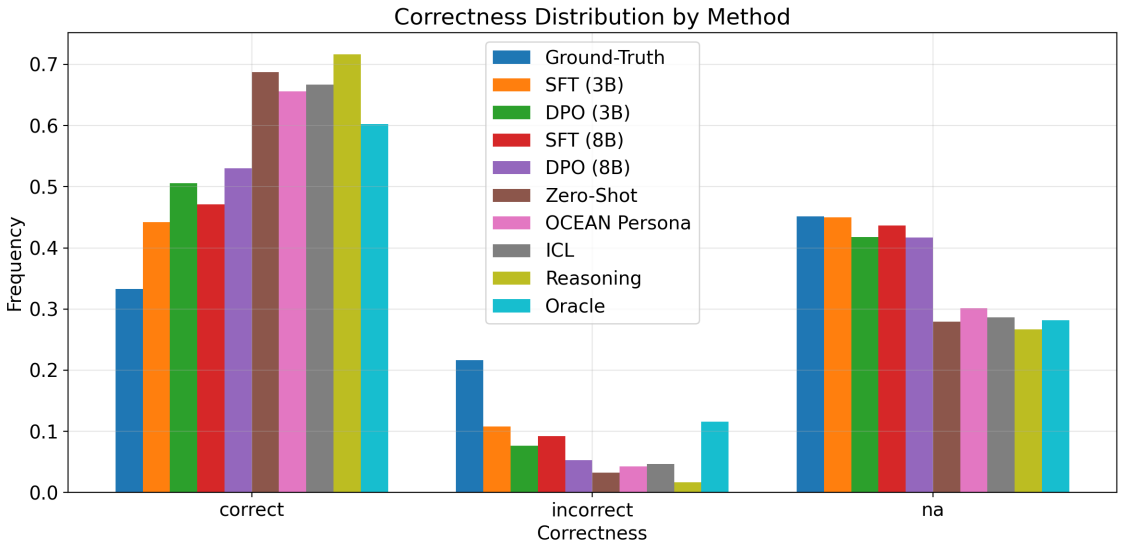


Figure 5.3: Distribution of correctness labels for real students and simulated student methods.

very short, following common linguistic patterns in the data, while Seek Information turns tend to be longer since they involve asking specific questions. On the other hand, prompting-based methods overly use Seek Information, with lower rates for Not Understanding, Acknowledge, and Off-topic. This shows that prompting-based methods are reluctant to generate more conversational turns that are common in student dialogues, while overestimating how often students ask questions. The Reasoning model stands out because it explicitly reasons about what acts to take, but still is less accurate than the fine-tuned models.

We show the distribution of correctness across methods in Figure 5.3. We observe that fine-tuned methods are much closer to the ground-truth distribution than prompting methods, with prompting methods more frequently giving correct answers and less frequently giving answers that do not have a particular correctness (na). This result explains how prompting-based methods perform better on the correctness metric, because they are more likely to predict the majority class. However, both fine-tuned and prompting-based methods overestimate how often students are correct,

demonstrating a need for methods to better anticipate when students will give incorrect answers.

5.5.6 Results by Turn

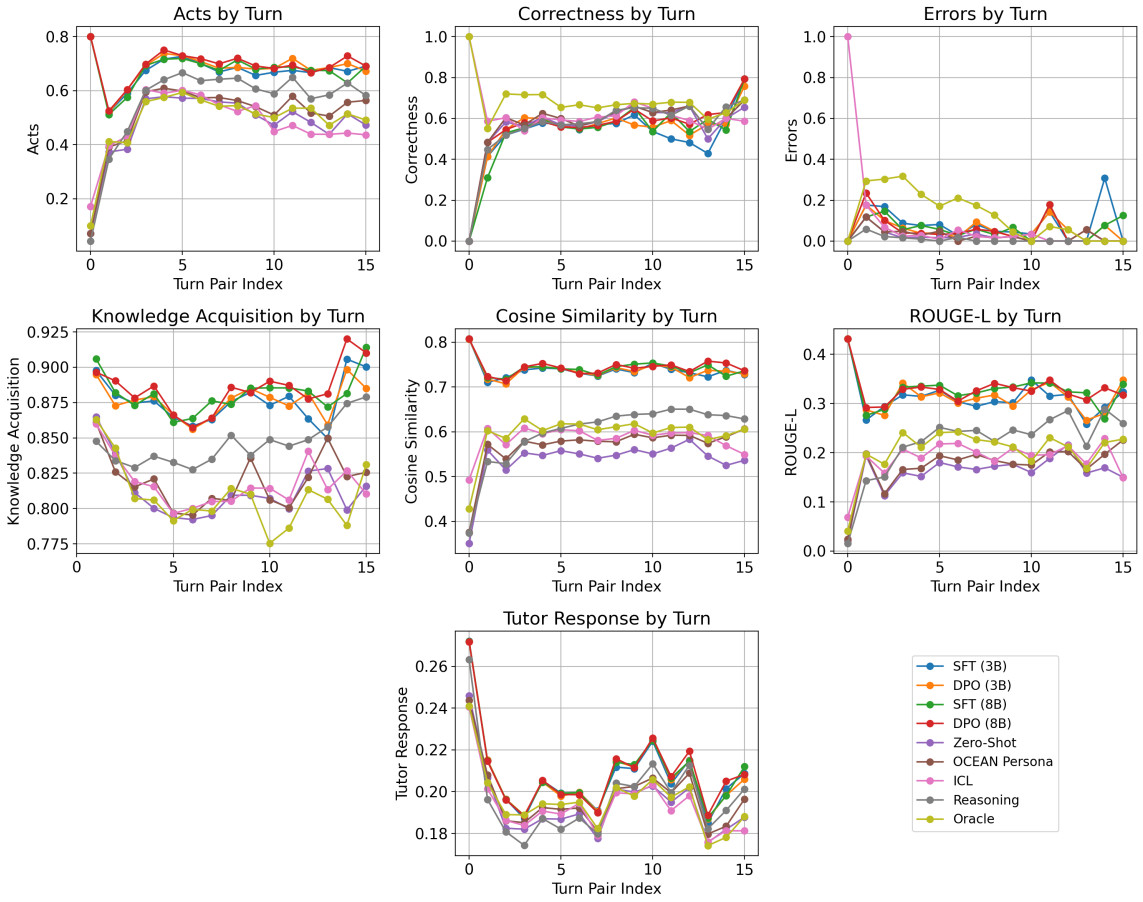


Figure 5.4: Results across metrics and methods broken down for each turn pair index.

To investigate how challenging different aspects of student simulation are at different points in dialogues, we examine how well methods perform on each of our metrics at the turn level. We plot the average at each turn pair index across the test set. To exclude high variance results, we truncate all dialogues after 15 turn pairs; 83% of dialogues are within this length.

We show the results of this investigation in Figure 5.4. We observe that all metrics show variability across the turns of a dialogue. Many metrics, such as Acts, Cosine

Similarity, ROUGE-L, and Tutor Response, are very easy for fine-tuned methods on the first turn due to most dialogues starting with a simple greeting. Prompting methods, on the other hand, perform poorly on the first turn since they have no prior turns in context. Correctness and Errors vary greatly on the first turn since most first turns have *na* as their correctness; in the rare cases where the first turn is correct or incorrect, some methods like Oracle correctly predict this, while others are not able to without context. After the first turn, the metrics are mostly stable, with a few exceptions: Acts are more challenging early on, since it is hard to predict the student's behavior with limited context. On the contrary, Knowledge Acquisition, becomes more difficult between turns 0 and 6, since the problem solving process begins in this range and it is difficult to estimate student knowledge with limited context; afterward, as context increases, so does Knowledge Acquisition performance. Fine-tuned methods tend to dominate prompting methods across turns, although the difference is usually greater early in the dialogues, since the prompting methods perform much better when there is more context. We also observe that the Reasoning method dominates other prompting methods later in the dialogues on Acts, Knowledge Acquisition, Cosine Similarity, and ROUGE-L, showing that it benefits from the added context more than other methods.

5.5.7 Examples

Question
Alex has two fraction cards. $\frac{5}{8}$ $\square$ The product of the cards is $\frac{1}{6}$ What is the other card? A: $\frac{19}{24}$ B: $\frac{5}{48}$ C: $\frac{15}{4}$ D: $\frac{4}{15}$ Correct Answer: D
Dialogue Context

Tutor: Hello again Leo! How can I help?
Student: I'm don't get the fact about missing number questions.
Tutor: Well, we know that 5/8 x ? = 1/6. So we need to rearrange that to get ? =
Student: Would we do the inverse.
Tutor: What would that be?
Student: 5/8 divided by 1/6?

SFT 3B: 8/5

DPO 3B: 8/5

SFT 8B: 5/8

DPO 8B: 5/8 divided by 1/6

Zero-Shot: So, would I divide 1/6 by 5/8?

OCEAN: Um, so would I divide 1/6 by 5/8? I think that's the inverse, right?

ICL: So like, 1/6 divided by 5/8?

Reasoning: Yes — multiply by the reciprocal. x = (1/6) ÷ (5/8) = (1/6)*(8/5) = 8/30 = 4/15

Oracle: So, like, the opposite of multiplying? So we would divide 1/6 by 5/8? Is that right?

Tutor: Nearly! We need to divide the left by 5/8 to get ? on its own, so we must do the same to the right hand side

	Next Student Turn Scores						
Source	Acts (Label)	Corr. (Label)	Errors	Knowledge (Quant.)	Cos. Sim.	ROUGE-L	Tutor Resp.
Ground-Truth	– (MA)	– (inc.)	–	– (4.00)	–	–	–
SFT 3B	1 (MA)	0 (corr.)	0	0.9167 (3.67)	0.5388	0.2500	0.1893
DPO 3B	1 (MA)	0 (corr.)	0	0.9167 (3.67)	0.5388	0.2500	0.1893
SFT 8B	1 (MA)	1 (inc.)	0	0.9167 (3.67)	0.6600	0.5000	0.1818
DPO 8B	1 (MA)	1 (inc.)	1	0.9167 (3.67)	0.9567	1.0000	0.1818
Zero-Shot	0 (SI)	0 (corr.)	0	0.7500 (3.00)	0.8714	0.2667	0.1818
OCEAN	0 (SI)	0 (corr.)	0	0.9167 (3.67)	0.7874	0.1739	0.1798
ICL	1 (MA)	0 (corr.)	0	0.8333 (3.33)	0.9039	0.2857	0.1720
Reasoning	1 (MA)	0 (corr.)	0	1.0000 (4.00)	0.7960	0.3333	0.1531
Oracle	0 (SI)	0 (corr.)	0	0.7500 (3.00)	0.7498	0.1667	0.1778

Table 5.9: Qualitative example for a single turn, showing the ground-truth student response (bordered in green), the simulated responses for that turn from all methods (bordered in yellow), and corresponding scores across all metrics. The ground-truth student response is an incorrect math answer (MA) to the previous tutor turn, where the correct answer would divide in the other direction. Most simulated responses are classified as MA, while some are classified as Seek Information (SI) due to having a more questioning tone. All simulated responses except for SFT 8B and DPO 8B are correct, with SFT 3B and DPO 3B inverting 5/8, most prompting methods inverting the full equation, and Reasoning giving a complete solution to the problem. While SFT 8B and DPO 8B are both incorrect, only DPO 8B has the same error as the ground-truth student response. For knowledge, we show the score and the average delta quantile across KCs. In this example, knowledge is highly correlated with task difficulty, reflecting findings in [205]. Specifically, incorrect or partially correct answers are likely to lead to easier followup tutor questions due to scaffolding, and are therefore more likely to be answered correctly by the student. On the other hand, correct answers are likely to be followed by the tutor posing a new task, which is likely to be harder than a scaffolded question, leading to a lower likelihood of the student answering correctly. The knowledge increases for Reasoning because the question is solved, so no new tasks will be posed. Cos. Sim. and ROUGE-L are highly correlated with textual overlap. Finally, Tutor Resp. is lower for prompting methods since they give full solutions, which would be unlikely to lead to the ground-truth tutor response.

Question
$\frac{A}{10} = \frac{9}{15}$ What is the value of A ? A: 3 B: 4 C: 6 D: 9 Correct Answer: C
Knowledge Components (KCs)
Fractions of an Amount Mental Addition and Subtraction Equivalent Fractions Dividing Fractions Naming Co-ordinates in 2D Default
Dialogue Context

Tutor: Have you tried simplifying 9/15?
Student: yes. 3/5
Tutor: Excellent! Good work!!!! Now how would you get that to a denominator of 10?
Student: what do you mean. 3/5
Tutor: so in 3/5 you have 5 as the denominator
Student: oh
Tutor: this question needs 10
Student: oh you times the numerator and denominator by 2
Tutor: Exactly!!!!! So what do you think it will become?

		Next Student Turn	
Source	Utterance	∇Z	quant(∇Z)
Ground-Truth	c	[0.0117, 0.0117, -0.0156, 0.0039, 0.0078, -0.0078]	[2, 2, 1, 2, 2, 1]
Simulated	6/10	[0.0391, 0.0352, 0.0391, 0.0352, 0.0156, 0.0352]	[3, 3, 3, 3, 3, 3]
Delta Quantile Upper Bounds: [-0.0234, 0.0000, 0.0156, 0.0430, ∞]			
Knowledge Acquisition Similarity			
$1 - \frac{\lvert 2-3\rvert+\lvert 2-3\rvert+\lvert 1-3\rvert+\lvert 2-3\rvert+\lvert 2-3\rvert+\lvert 1-3\rvert}{4\times 6} = 0.6666$			

Table 5.10: Example decomposition of Knowledge Acquisition similarity for a single turn. The simulated turn is from DPO 8B. Early turns in the dialogue with greetings are excluded for brevity. While both answers are correct, the ground-truth simply gives the final answer option, while the simulated turn gives the full fraction associated with the correct answer. The knowledge tracing model estimates that the student's knowledge delta for the ground-truth turn is roughly average for all KCs (2 or 1 between 0 and 4), while the knowledge delta for the simulated turn is above average for all KCs (all 3). This difference may be due to the possibility that answering "c" is simply a lucky guess, whereas answering "6/10" conveys an understanding of the solution, leading to a higher knowledge estimate. This example demonstrates the need for a knowledge-based metric, since two turns can have the same correctness but give different indications of student mastery on concepts.

Question
Arrange the numbers in descending order. $-5, -8, 7, -6$ A: $-8, 7, -6, -5$ B: $7, -8, -6, -5$ C: $-8, -6, -5, 7$ D: $7, -5, -6, -8$ Correct Answer: D
Question Solution (Generated by GPT-4.1)

Solution: To arrange the numbers -5, -8, 7, -6 in descending order, we start with the largest and go to the smallest. 7 is the largest, then -5, then -6, then -8. So the correct order is 7, -5, -6, -8.
A Explanation: This option lists -8 first, which is the smallest number, not the largest. This is the reverse of the correct order (ascending order for the negatives, but with 7 misplaced). A student might mistakenly think that more negative numbers are larger.
B Explanation: This option starts with 7, which is correct, but then puts -8 before -6 and -5. This is incorrect because -8 is the smallest (most negative) number. A student might have placed 7 first but then listed the negatives in the wrong order.
C Explanation: This option lists all the negative numbers first in increasing order, then 7 at the end. This is ascending order, not descending. A student might have confused ascending and descending order.
D Explanation: This is the correct answer: 7, -5, -6, -8. 7 is the largest, followed by -5, then -6, then -8.

Dialogue

Tutor: Hi again! Firstly, do you know what descending order is?
Student: YEARH!
Tutor: Go on, what does that mean?
Student: Biggest to smallest
Tutor: Spot on!! From those number which number would be the biggest?
Student: ✓. Wich numbers
Tutor: We have -5 -8 7 -6
Student: Ohh -5
Tutor: 7 must be the biggest number as it is the only positive number. Try and imagine or even draw out a number line, 7 is the only number greater than 0. So from our answer options we can narrow it down do B or D as they both start with 7. Does that make sense so far?
Student: ✓
Tutor: Excellent
Student: ✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
Tutor: Now we have these number left -5 -8 -6
Student: ✓
Tutor: From these number which one is the biggest? So try and think of a number line, which number is closest to 0?
Student: -5
Tutor: Well done Mariana, that points us to answer D! 7, -5, -6, -8
Student: D✓
Tutor: Bye for now!
Student: Bye thanks for your help!

OCEAN Persona (Generated by GPT-4.1)

Reasoning: The student demonstrates understanding of mathematical concepts when prompted and is able to follow the tutor's guidance. Their responses are brief and sometimes enthusiastic (e.g., 'YEARH!', multiple checkmarks), but do not show much elaboration or curiosity beyond the immediate task, suggesting neutral openness. The student is responsive and follows instructions, indicating a reasonable level of conscientiousness, though there is no evidence of independent organization or planning, so this is also neutral. The student is not very talkative and gives short answers, which suggests low extraversion. The student is polite, cooperative, and expresses gratitude at the end, indicating high agreeableness. There is no evidence of anxiety, frustration, or negative emotions, so neuroticism appears low.
Openness: neutral
Conscientiousness: neutral
Extraversion: low
Agreeableness: high
Neuroticism: low

Oracle Summary (Generated by GPT-4.1)

The student demonstrates a basic understanding of mathematical concepts when prompted, correctly identifying "descending order" as "biggest to smallest" and, with guidance, selecting the correct answer. Initially, the student makes a conceptual error by choosing -5 as the largest number, indicating some confusion about the relative size of negative and positive numbers. However, the student is receptive to feedback, quickly adjusts their thinking when the tutor explains, and successfully applies the number line strategy when prompted. Behaviorally, the student is highly responsive, frequently uses affirmative emojis (✓) and enthusiastic language, and often provides brief, sometimes minimal, answers, suggesting a preference for quick confirmation over detailed explanation. The student is agreeable and polite, expressing gratitude at the end, and shows openness to learning by engaging with the tutor's questions. Linguistically, the student uses informal, enthusiastic responses, sometimes with spelling errors ("YEARH!," "Wich numbers," "Ohh -5"), and tends to favor emoji-heavy, concise replies over elaboration. Overall, the student is eager, agreeable, and responsive, with a tendency toward impulsive answers but a willingness to learn and correct mistakes when guided.

Retrieved ICL Example

Student: Hello!
Tutor: Hiya! How can I help?
Student: May you help me please? Am very stuck!
Tutor: of course!
Student: Thank you
Tutor: are you stuck on writing numbers in desending order?
Student: yes
Tutor: do you know what descending means?
Student: Is it largest to smallest? I don't really know?
Tutor: that's correct!
Student: Oh...
Tutor: so which one is the largest?
Student: 7
Tutor: good, now which is the next biggest?
Student: -5
Tutor: good! now what?
Student: -6
Tutor: good! and finally?
Student: -8
Tutor: good!
Student: oh that was easy! hehe
Tutor: would you like any more help?
Student: i want to say a big thank you for helping me!
Tutor: hahah no problem! You did very well! Have a nice evening!

Table 5.11: Example dialogue, question, solution, and associated context provided to OCEAN, Oracle, and ICL prompting methods. Emojis except checkmark are removed.

5.6 Conclusions and Future Work

In this paper, we introduce the task of simulating student utterances in tutoring dialogues. We propose a suite of metrics that measure realism of a simulated student turn with respect to a reference student turn. We benchmark a wide range of fine-tuned and prompting-based methods, finding that while fine-tuning and RL outperform prompting, there is a long way to go before LLMs can fully resemble real student behavior in dialogues.

There are many avenues for future work. First, since we use simple RL methods, it is possible that more advanced techniques leveraging reasoning or data augmentation would result in significantly better performance. Second, due to data constraints, we

evaluate each dialogue in isolation. However, with available data, future work could include prior student information, such as prior dialogues or knowledge states, for more context in such simulations. Third, since we only explore simulation realism at the turn-level, future work should further extend our metrics to measure realism at the dialogue-level. Fourth, while we have shown that our metrics are reliable *reference-based* measures of student behavior, future work should investigate *reference-free* metrics for settings where ground-truth data is not available. Finally, since we only explore dialogues based on math problems, future work should extend to dialogues in other domains, such as computer science and language learning.

CHAPTER 6

CONCLUSIONS

In this thesis, I presented the problem of aligning LLM-based simulated students with real students, introduced fine-tuning methods for achieving this alignment, and demonstrated how the resulting simulated students can be used to align educational content and systems with student outcomes. I demonstrated that LLMs are capable of modeling student ability and knowledge in domains such as open-ended question answering and tutoring dialogues, but that fine-tuning is necessary for LLMs to faithfully reflect student knowledge and response patterns. I showed that my proposed fine-tuning approaches achieve greater LLM-student alignment than traditional approaches across domains, primarily by integrating student knowledge and behavior into training through reward functions and knowledge-centered objectives. I also demonstrated that by leveraging simulated students, we can achieve greater content alignment, expressed in both question difficulty alignment and tutor behavior that improves student outcomes, compared to baseline approaches that rely on prompting pre-trained LLMs or fine-tuning without simulated students. Finally, I introduced a framework for evaluating the realism of simulated students in dialogues, facilitating future research to develop methods that more faithfully resemble real students, and subsequently improving the alignment of downstream educational AI that uses simulated students in-the-loop.

6.1 Future Work

There are many potential future directions building on this thesis. I present three broad research areas that can benefit from simulated students: personalization, open-ended learning environments, and agentic educational assistants. I detail challenges and goals in each of these areas, how simulated students can help, and areas of improvement for simulated student methods.

6.1.1 Data-Driven Personalization and Adaptation

The "one-size-fits-all" model is common in education, where an entire group of students follow the same pre-defined curriculum. However, there are many reasons why different students should study different materials, practice with different questions, or be assessed in different ways. For instance, students from different cultural backgrounds can have diverging perspectives on certain topics, and therefore may be disadvantaged when only being assessed on items from a single cultural perspective. Students also have a wide variety of interests that can be leveraged when recommending or generating content to keep them engaged. Students with lower levels of motivation may find it difficult to continually practice items, so may benefit more from tutoring and high levels of encouragement. Finally, students with learning disabilities may require that concepts are introduced at different rates.

Therefore, we need to design AI systems that intelligently adapt to each student to address diverse backgrounds and needs, while maintaining fair and high quality practice and assessments. While some personalization is currently supported in current learning platforms, such as difficulty-based item recommendation, highly personalized AI-driven features, such as interest-guided item generation or tutoring grounded in a student's personality, are not widely adopted. The development of such features should be supported by LLM-based student models and simulations, where predictions of how students will respond to items and subsequently learn from them are used as

feedback to the content-generating system. As suggested by experimental evidence in this thesis, such student models will benefit from alignment training. However, new alignment methods will likely be needed to ensure that models are representing a student's knowledge and behavior faithfully. In particular, while methods in this thesis condition on local information, such as a single dialogue session or ability estimate, educational platforms often have access to large amounts of data per student, including historical question answers, clickstreams, and potentially specified preferences. Future alignment methods will need to leverage such rich sources of data to form a more wholistic view of a student and make more accurate predictions of future learning and behavior.

6.1.2 Open-Ended and Creative Learning Environments

The modalities of student modeling have historically been narrow, typically capturing binary correctness of responses, and more recently open-ended text. Additionally, learning platforms typically use multiple-choice and cloze questions since they are easy to grade; while platforms have also begun implementing dialogues more widely, these activities represent a small fraction of those in a typical classroom, especially for younger students. Broader student observations, including auditory utterances and physical interactions, across diverse learning settings can give greater insights on a student's thinking process, behavioral patterns, and affective states. For example, the way that students move around a classroom when playing interactive games can give information on their social skills and engagement in the class. Using multi-modal generative AI and world models, we can construct student models that extend beyond correctness and text, and push into visual, auditory, and physical spaces.

Such modeling will facilitate tutoring in undersupported domains like art and music, and help foster students' 21st-century skills, such as the 4 Cs (creativity, critical thinking, collaboration, and communication). For example, we can build multi-modal

student models to represent music learners, building on recent work that enables real-time human-AI musical interaction [210, 251], to enable adaptive music tutors. Music tutoring agents can support creative learning by generating musical tasks in real-time that aim to improve the student's creative and performative skills. Similar to existing adaptive tutors, these agents can adapt to a student's background, musical preferences, and skill level to boost engagement and learning.

We can support 21st-century skill learning in open-ended environments using a two-stage approach: 1) fine-tuning LLMs to assess these skills using targeted rubrics and guidance from teachers, and 2) integrating AI into various stages in long-term projects and group work. For instance, an AI tutor can interact with a student during project ideation to both encourage creativity and to assess a student's critical thinking for planning out the project. As the project progresses and the student creates materials such as literature reviews or presentation slides, the AI tutor can track how the student's 21st-century skills have developed and make suggestions to foster skills where the student is struggling. This end-to-end process can initially play out with simulated students to gauge effectiveness, where simulation data can be used to refine strategies before being deployed in classrooms.

6.1.3 Agentic Educational Assistants

Recent proprietary LLMs have become powerful generalist agents, leveraging reasoning and tool use to accomplish a large set of tasks, including ones not seen during training. However, these models often underperform in educational tasks, such as creating content that is calibrated to the goals of a particular class, realistically simulating student responses, and predicting student errors. Building upon the previously mentioned goals of personalization and multi-modal modeling in open-ended environments, we can pursue an agentic AI system that can reliably solve a

range of novel tasks in education, follow strong yet configurable pedagogical practices, and adapt to the particular needs of diverse student populations.

Such systems can be designed with simulated students at the core: when executing a task, a multi-agent planning stage can take place, where an expert agent proposes solutions which are then verified using simulated student agents. An example use case is a teacher seeking to create a new course curriculum, where they could describe the desired outcomes of their course and provide relevant historical student data as input. The teacher would collaborate with the system to refine a curriculum that gives a high chance of all students meeting the desired goals, where the system supports its reasoning in an interpretable way by citing applied pedagogical principles, examples from student data, and simulation results. Such a system will not only improve teachers' effectiveness, but will employ data-driven techniques in a dynamic and broad way to deliver fast, reliable, and explainable solutions to challenging problems in a quickly evolving educational landscape.

BIBLIOGRAPHY

[1] Abdelghani, Rania, Wang, Yen-Hsiang, Yuan, Xingdi, Wang, Tong, Lucas, Pauline, Sauzéon, Hélène, and Oudeyer, Pierre-Yves. Gpt-3-driven pedagogical agents to train children's curious question-asking skills. *International Journal of Artificial Intelligence in Education 34*, 2 (2024), 483–518.

[2] Abdelrahman, Ghodai, Wang, Qing, and Nunes, Bernardo. Knowledge tracing: A survey. *ACM Computing Surveys 55*, 11 (2023), 1–37.

[3] Abdelshiheed, Mark, K. Jacobs, Jennifer, and K. D'Mello, Sidney. Aligning tutor discourse supporting rigorous thinking with tutee content mastery for predicting math achievement. In *Artificial Intelligence in Education* (Cham, 2024), Springer Nature Switzerland, pp. 150–164.

[4] Abdulhai, Marwa, Cheng, Ryan, Clay, Donovan, Althoff, Tim, Levine, Sergey, and Jaques, Natasha. Consistently simulating human personas with multi-turn reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems* (2025).

[5] Aleven, Vincent, Mclaren, Bruce, Roll, Ido, and Koedinger, Kenneth. Toward meta-cognitive tutoring: A model of help seeking with a cognitive tutor. *International journal of artificial intelligence in education 16*, 2 (2006), 101–128.

[6] Aleven, Vincent, McLaren, Bruce M, Sewall, Jonathan, and Koedinger, Kenneth R. The cognitive tutor authoring tools (ctat): Preliminary evaluation of efficiency gains. In *International Conference on Intelligent Tutoring Systems* (2006), Springer, pp. 61–70.

[7] Aleven, Vincent, Mclaren, Bruce M, Sewall, Jonathan, and Koedinger, Kenneth R. A new paradigm for intelligent tutoring systems: Example-tracing tutors. *International Journal of Artificial Intelligence in Education 19*, 2 (2009), 105–154.

[8] Aleven, Vincent, Roll, Ido, McLaren, Bruce M, and Koedinger, Kenneth R. Help helps, but only so much: Research on help seeking with intelligent tutoring systems. *International Journal of Artificial Intelligence in Education 26*, 1 (2016), 205–223.

[9] AlKhuzaey, Samah, Grasso, Floriana, Payne, Terry R, and Tamma, Valentina. Text-based question difficulty prediction: A systematic review of automatic approaches. *International Journal of Artificial Intelligence in Education 34*, 3 (2024), 862–914.

[10] Almeda, Ma Victoria, and Baker, Ryan S. Predicting student participation in stem careers: The role of affect and engagement during middle school. *Journal of Educational Data Mining 12*, 2 (2020), 33–47.

[11] Anderson, John R. *Rules of the mind*. Psychology Press, 2014.

[12] Anderson, John R, Corbett, Albert T, Koedinger, Kenneth R, and Pelletier, Ray. Cognitive tutors: Lessons learned. *The journal of the learning sciences 4*, 2 (1995), 167–207.

[13] Anderson, John R, and Jeffries, Robin. Novice lisp errors: Undetected losses of information from working memory. *Human–Computer Interact. 1*, 2 (1985), 107–131.

[14] Anthropic. Claude 3.5 sonnet. `https://www.anthropic.com/news/claude-3-5-sonnet`, 2024. Accessed: 2025-05-19.

[15] Anthropic. Introducing claude for education. Online: `https://www.anthropic.com/news/introducing-claude-for-education`, 2025.

[16] Arroyo, Ivon, Ferguson, Kimberly, Johns, Jeffrey, Dragon, Toby, Meheranian, Hasmik, Fisher, Don, Barto, Andrew, Mahadevan, Sridhar, and Woolf, Beverly Park. Repairing disengagement with non-invasive interventions. In *AIED* (2007), vol. 2007, pp. 195–202.

[17] Ashok Kumar, Nischal, Fernandez, Nigel, Wang, Zichao, and Lan, Andrew. Improving reading comprehension question generation with data augmentation and overgenerate-and-rank. In *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023)* (Toronto, Canada, July 2023), Association for Computational Linguistics, pp. 247–259.

[18] Ashok Kumar, Nischal, and Lan, Andrew. Improving socratic question generation using data augmentation and preference optimization. In *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)* (Mexico City, Mexico, June 2024), Association for Computational Linguistics, pp. 108–118.

[19] Azevedo, Roger, and Hadwin, Allyson F. Scaffolding self-regulated learning and metacognition–implications for the design of computer-based scaffolds. *Instructional science 33*, 5/6 (2005), 367–379.

[20] Badrinath, Anirudhan, Wang, Frederic, and Pardos, Zachary. pybkt: An accessible python library of bayesian knowledge tracing models. In *Proceedings of the 14th International Conference on Educational Data Mining* (2021), pp. 468–474.

[21] Baker, Ryan, Walonoski, Jason, Heffernan, Neil, Roll, Ido, Corbett, Albert, and Koedinger, Kenneth. Why students engage in "gaming the system" behavior in interactive learning environments. *Journal of Interactive Learning Research 19*, 2 (2008), 185–224.

[22] Baker, Ryan SJ d, Corbett, Albert T, and Aleven, Vincent. More accurate student modeling through contextual estimation of slip and guess probabilities in bayesian knowledge tracing. In *International conference on intelligent tutoring systems* (2008), Springer, pp. 406–415.

[23] Baker, Ryan SJd, D'Mello, Sidney K, Rodrigo, Ma Mercedes T, and Graesser, Arthur C. Better to be frustrated than bored: The incidence, persistence, and impact of learners' cognitive–affective states during interactions with three different computer-based learning environments. *International Journal of Human-Computer Studies 68*, 4 (2010), 223–241.

[24] Barnes, Tiffany. The q-matrix method: Mining student response data for knowledge. In *American association for artificial intelligence 2005 educational data mining workshop* (2005), AAAI Press, Pittsburgh, PA, USA, pp. 1–8.

[25] Barnum, Matt. Why sal khan's ai revolution hasn't happened yet, according to sal khan. Online: `https://www.chalkbeat.org/2026/04/09/sal-khan-reflects-on-ai-in-schools-and-khanmigo/`, 2026.

[26] Beck, Joseph. Learning to teach with a reinforcement learning agent. In *AAAI/IAAI* (1998), p. 1185.

[27] Beck, Joseph, and Woolf, Beverly Park. High-level student modeling with machine learning. In *Proceedings of the 5th International Conference on Intelligent Tutoring Systems* (Berlin, Heidelberg, 2000), ITS '00, Springer-Verlag, p. 584–593.

[28] Beck, Joseph E, Chang, Kai-min, Mostow, Jack, and Corbett, Albert. Does help help? introducing the bayesian evaluation and assessment methodology. In *International conference on intelligent tutoring systems* (2008), Springer, pp. 383–394.

[29] Benedetto, Luca. A quantitative study of nlp approaches to question difficulty estimation. In *International Conference on Artificial Intelligence in Education* (2023), Springer, pp. 428–434.

[30] Benedetto, Luca, Aradelli, Giovanni, Cremonesi, Paolo, Cappelli, Andrea, Giussani, Andrea, and Turrin, Roberto. On the application of transformers for estimating the difficulty of multiple-choice questions from text. In *Proceedings of the 16th Workshop on Innovative Use of NLP for Building Educational Applications* (Online, Apr. 2021), Association for Computational Linguistics, pp. 147–157.

[31] Benedetto, Luca, Aradelli, Giovanni, Donvito, Antonia, Lucchetti, Alberto, Cappelli, Andrea, and Buttery, Paula. Using LLMs to simulate students' responses to exam questions. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 11351–11368.

[32] Benedetto, Luca, Cremonesi, Paolo, Caines, Andrew, Buttery, Paula, Cappelli, Andrea, Giussani, Andrea, and Turrin, Roberto. A survey on recent approaches to question difficulty estimation from text. *ACM Computing Surveys 55*, 9 (2023), 1–37.

[33] Bicknell, Klinton, Brust, Claire, and Settles, Burr. How duolingo's ai learns what you need to learn: The language-learning app tries to emulate a great human tutor. *IEEE Spectrum 60*, 3 (2023), 28–33.

[34] Bin, Li, Jun, Lu, Jian-Min, Yao, and Qiao-Ming, Zhu. Automated essay scoring using the knn algorithm. In *2008 International Conference on Computer Science and Software Engineering* (2008), vol. 1, pp. 735–738.

[35] Birnbaum, Allan. Some latent trait models and their use in inferring an examinee's ability. *Statistical theories of mental test scores* (1968).

[36] Brown, John Seely, and Burton, Richard R. Diagnostic models for procedural bugs in basic mathematical skills. *Cogn. sci. 2*, 2 (1978), 155–192.

[37] Brown, Tom, Mann, Benjamin, Ryder, Nick, Subbiah, Melanie, Kaplan, Jared D, Dhariwal, Prafulla, Neelakantan, Arvind, Shyam, Pranav, Sastry, Girish, Askell, Amanda, Agarwal, Sandhini, Herbert-Voss, Ariel, Krueger, Gretchen, Henighan, Tom, Child, Rewon, Ramesh, Aditya, Ziegler, Daniel, Wu, Jeffrey, Winter, Clemens, Hesse, Chris, Chen, Mark, Sigler, Eric, Litwin, Mateusz, Gray, Scott, Chess, Benjamin, Clark, Jack, Berner, Christopher, McCandlish, Sam, Radford, Alec, Sutskever, Ilya, and Amodei, Dario. Language models are few-shot learners. In *Advances in Neural Information Processing Systems* (2020), vol. 33, Curran Associates, Inc., pp. 1877–1901.

[38] Brownlee, Jason. How to implement the frechet inception distance (fid) for evaluating gans, 2019. Accessed: 2025-05-19.

[39] Bui, Ngoc, Nguyen, Hieu Trung, Kumar, Shantanu, Theodore, Julian, Qiu, Weikang, Nguyen, Viet Anh, and Ying, Rex. Mixture-of-personas language models for population simulation, 2025.

[40] Byrd, Matthew, and Srivastava, Shashank. Predicting difficulty and discrimination of natural language questions. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (Dublin, Ireland, May 2022), Association for Computational Linguistics, pp. 119–130.

[41] Cai, Dejun, Zhang, Yuan, and Dai, Bintao. Learning path recommendation based on knowledge tracing model and reinforcement learning. In *2019 IEEE 5th international conference on computer and communications (ICCC)* (2019), IEEE, pp. 1881–1885.

[42] Cai, Zhiqiang, Graesser, Arthur C, Windsor, Leah, Cheng, Qinyu, Shaffer, David W, and Hu, Xiangen. Impact of corpus size and dimensionality of lsa spaces from wikipedia articles on autotutor answer evaluation. *Journal of educational data mining* (2018).

[43] Cao, Jie, Nguyen, Ha, Yavuz, Selim, Yu, Boran, Wang, Shuguang, Bharaj, Pavneet Kaur, and Francis, Dionne Cross. Developing authentic simulated learners for mathematics teacher learning: Insights from three approaches with large language models. *arXiv preprint arXiv:2604.04361* (2026).

[44] Carnegie Learning. Livehint overview. Online: `https://support.carnegielearning.com/help-center/math/livehint/article/livehint-overview/`, 2024.

[45] Castro, Santiago. Fast Krippendorff: Fast computation of Krippendorff's alpha agreement measure. `https://github.com/pln-fing-udelar/fast-krippendorff`, 2017.

[46] Cen, Hao, Koedinger, Kenneth, and Junker, Brian. Learning factors analysis–A general method for cognitive model evaluation and improvement. In *Proc. Int. Conf. Intell. Tutoring Syst.* (2006), pp. 164–175.

[47] Chalmers, R Philip. mirt: A multidimensional item response theory package for the r environment. *Journal of statistical Software 48* (2012), 1–29.

[48] Chang, Hua-Hua, and Ying, Zhiliang. Nonlinear sequential designs for logistic item response theory models with applications to computerized adaptive tests. *The Annals of Statistics* (2009), 1466–1488.

[49] Chang, Maria, Ventura, Matthew, Ahn, Jae-wook, Foltz, Peter, Ma, Tengfei, Dhamecha, Tejas I, Marvaniya, Smit, Watson, Patrick, D'helon, Cassius, Wetzel, Amy, et al. Dialogue-based tutoring at scale: Design and challenges. In *Practitioner and Industrial Track Proceedings of the 13th International Conference of the Learning Sciences* (2018).

[50] Chen, Chih-Ming, and Duh, Ling-Jiun. Personalized web-based tutoring system based on fuzzy item response theory. *Expert systems with applications 34*, 4 (2008), 2298–2315.

[51] Chen, Runjin, Arditi, Andy, Sleight, Henry, Evans, Owain, and Lindsey, Jack. Persona vectors: Monitoring and controlling character traits in language models, 2025.

[52] Choffin, Benoît, Popineau, Fabrice, Bourda, Yolaine, and Vie, Jill-Jênn. DAS3H: Modeling student learning and forgetting for pptimally scheduling distributed practice of skills. In *Proc. Int. Conf. Educ. Data Mining* (2019), pp. 29–38.

[53] Choi, Youngduck, Lee, Youngnam, Cho, Junghyun, Baek, Jineon, Kim, Byungsoo, Cha, Yeongmin, Shin, Dongmin, Bae, Chan, and Heo, Jaewe. Towards an appropriate query, key, and value computation for knowledge tracing. In *Proceedings of the Seventh ACM Conference on Learning @ Scale* (New York, NY, USA, 2020), L@S '20, Association for Computing Machinery, p. 341–344.

[54] Cobbe, Karl, Kosaraju, Vineet, Bavarian, Mohammad, Chen, Mark, Jun, Heewoo, Kaiser, Lukasz, Plappert, Matthias, Tworek, Jerry, Hilton, Jacob, Nakano, Reiichiro, Hesse, Christopher, and Schulman, John. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).

[55] Common Core State Standards Initiative. Mathematics standards. Online: `https://www.thecorestandards.org/Math/`, 2024.

[56] Corbett, Albert, and Anderson, John. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Model. User-adapted Interact. 4*, 4 (Dec. 1994), 253–278.

[57] Croteau, Ethan A, Heffernan, Neil T, and Koedinger, Kenneth R. Why are algebra word problems difficult? using tutorial log files and the power law of learning to select the best fitting cognitive model. In *Intelligent Tutoring Systems: 7th International Conference, ITS 2004, Maceió, Alagoas, Brazil, August 30-September 3, 2004. Proceedings 7* (2004), Springer, pp. 240–250.

[58] Cui, Peng, and Sachan, Mrinmaya. Adaptive and personalized exercise generation for online language learning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2023), pp. 10184–10198.

[59] Daheim, Nico, Macina, Jakub, Kapur, Manu, Gurevych, Iryna, and Sachan, Mrinmaya. Stepwise verification and remediation of student reasoning errors with large language model tutors. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 8386–8411.

[60] DataShop. Dataset: Codeworkout data spring 2019. Online: `https://pslcdatashop.web.cmu.edu/Files?datasetId=3458`, 2021.

[61] Demszky, Dorottya, and Hill, Heather. The ncte transcripts: A dataset of elementary math classroom transcripts. In *Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023)* (2023), pp. 528–538.

[62] Demszky, Dorottya, Liu, Jing, Mancenido, Zid, Cohen, Julie, Hill, Heather, Jurafsky, Dan, and Hashimoto, Tatsunori B. Measuring conversational uptake: A case study on student-teacher interactions. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)* (2021), pp. 1638–1653.

[63] Dettmers, Tim, Pagnoni, Artidoro, Holtzman, Ari, and Zettlemoyer, Luke. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems 36* (2023), 10088–10115.

[64] Dinucu-Jianu, David, Macina, Jakub, Daheim, Nico, Hakimi, Ido, Gurevych, Iryna, and Sachan, Mrinmaya. From problem-solving to teaching problem-solving: Aligning LLMs with pedagogy using reinforcement learning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (Suzhou, China, Nov. 2025), Association for Computational Linguistics, pp. 272–292.

[65] Dou, Yao, Galley, Michel, Peng, Baolin, Kedzie, Chris, Cai, Weixin, Ritter, Alan, Quirk, Chris, Xu, Wei, and Gao, Jianfeng. SimulatorArena: Are user simulators reliable proxies for multi-turn evaluation of AI assistants? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (Suzhou, China, Nov. 2025), Association for Computational Linguistics, pp. 35212–35290.

[66] Duan, Zhangqi, Fernandez, Nigel, Narayanan, Arun Balajiee Lekshmi, Hassany, Mohammad, de Alencar, Rafaella Sampaio, Brusilovsky, Peter, Akram, Bita, and Lan, Andrew. Automated knowledge component generation for interpretable knowledge tracing in coding problems, 2025.

[67] Duenas, George, Jimenez, Sergio, and Mateus Ferro, Geral. UPN-ICC at BEA 2024 shared task: Leveraging LLMs for multiple-choice questions difficulty prediction. In *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)* (Mexico City, Mexico, June 2024), Association for Computational Linguistics, pp. 542–550.

[68] Duolingo. Video call lets you have real life conversations with lily. Online: `https://blog.duolingo.com/video-call/`, 2024.

[69] Eedi. Eedi labs. `https://www.eedi.com/`, 2026. Accessed: 2026-01-05.

[70] Eignor, Daniel R. The standards for educational and psychological testing. *APA handbook of testing and assessment in psychology, Vol. 1. Test theory and testing and assessment in industrial and organizational psychology* (2013).

[71] Face, Hugging. Hugging face - the ai community building the future. `https://huggingface.co/`, 2025. Accessed: 2025-02-18.

[72] Feldman, Molly Q, Cho, Ji Yong, Ong, Monica, Gulwani, Sumit, Popović, Zoran, and Andersen, Erik. Automatic diagnosis of students' misconceptions in K-8 mathematics. In *Proc. CHI Conf. Human Factors Comput. Syst.* (2018), pp. 1–12.

[73] Feng, Mingyu, Heffernan, Neil, and Koedinger, Kenneth. Addressing the assessment challenge with an online system that tutors as it assesses. *User modeling and user-adapted interaction 19*, 3 (2009), 243–266.

[74] Feng, Wanyong, Lee, Jaewook, McNichols, Hunter, Scarlatos, Alexander, Smith, Digory, Woodhead, Simon, Ornelas, Nancy, and Lan, Andrew. Exploring automated distractor generation for math multiple-choice questions via large language models. In *Findings of the Association for Computational Linguistics: NAACL 2024* (Mexico City, Mexico, June 2024), Association for Computational Linguistics, pp. 3067–3082.

[75] Feng, Wanyong, Tran, Peter, Sireci, Stephen, and Lan, Andrew S. Reasoning and sampling-augmented mcq difficulty prediction via llms. In *Artificial Intelligence in Education: 26th International Conference, AIED 2025, Palermo, Italy, July 22–26, 2025, Proceedings, Part IV* (Berlin, Heidelberg, 2025), Springer-Verlag, p. 31–45.

[76] Fernandez, Nigel, Scarlatos, Alexander, Feng, Wanyong, Woodhead, Simon, and Lan, Andrew. DiVERT: Distractor generation with variational errors represented as text for math multiple-choice questions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 9063–9081.

[77] Gertner, Abigail S, and VanLehn, Kurt. Andes: A coached problem solving environment for physics. In *International conference on intelligent tutoring systems* (2000), Springer, pp. 133–142.

[78] Ghosh, Aritra, Heffernan, Neil, and Lan, Andrew S. Context-aware attentive knowledge tracing. In *Proc. ACM SIGKDD* (2020), pp. 2330–2339.

[79] Glickman, Mark E. Example of the glicko-2 system. *Boston University 28* (2012).

[80] Gomaa, Wael H, and Fahmy, Aly A. Short answer grading using string similarity and corpus-based similarity. *International Journal of Advanced Computer Science and Applications (IJACSA) 3*, 11 (2012).

[81] Gombert, Sebastian, Menzel, Lukas, Di Mitri, Daniele, and Drachsler, Hendrik. Predicting item difficulty and item response time with scalar-mixed transformer encoder models and rational network regression heads. In *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)* (Mexico City, Mexico, June 2024), Association for Computational Linguistics, pp. 483–492.

[82] Gonzalez-Brenes, Jose, and Mostow, Jack. What and when do students learn? fully data-driven joint estimation of cognitive and student models. In *Educational Data Mining 2013* (2013).

[83] Google. How generative ai expands curiosity and understanding with learnlm. Online: `https://blog.google/outreach-initiatives/education/google-learnlm-gemini-generative-ai/`, 2024.

[84] Graesser, Arthur C, VanLehn, Kurt, Rosé, Carolyn P, Jordan, Pamela W, and Harter, Derek. Intelligent tutoring systems with conversational dialogue. *AI magazine 22*, 4 (2001), 39–39.

[85] Grattafiori, Aaron, Dubey, Abhimanyu, Jauhri, Abhinav, Pandey, Abhinav, Kadian, Abhishek, Al-Dahle, Ahmad, Letman, Aiesha, Mathur, Akhil, Schelten, Alan, Vaughan, Alex, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[86] Han, Jieun, Yoo, Haneul, Myung, Junho, Kim, Minsun, Lim, Hyunseung, Kim, Yoonsu, Lee, Tak Yeon, Hong, Hwajung, Kim, Juho, Ahn, So-Yeon, et al. Llm-as-a-tutor in efl writing education: Focusing on evaluation of student-llm interaction. In *Proceedings of the 1st Workshop on Customizable NLP: Progress and Challenges in Customizing NLP for a Domain, Application, Group, or Individual (CustomNLP4U)* (2024), pp. 284–293.

[87] He-Yueya, Joy, Goodman, Noah D., and Brunskill, Emma. Evaluating and optimizing educational content with large language model judgments. In *Proceedings of the 17th International Conference on Educational Data Mining* (July 2024), pp. 68–82.

[88] He-Yueya, Joy, Ma, Wanjing Anya, Gandhi, Kanishk, Domingue, Benjamin W, Brunskill, Emma, and Goodman, Noah D. Psychometric alignment: Capturing human knowledge distributions via language models. *arXiv preprint arXiv:2407.15645* (2024).

[89] He-Yueya, Joy, and Singla, Adish. Quizzing policy using reinforcement learning for inferring the student knowledge state. *International Educational Data Mining Society* (2021).

[90] Henry, Parker. How duolingo uses ai to create lessons faster. Online: `https://blog.duolingo.com/large-language-model-duolingo-lessons/`, 2023.

[91] Heusel, Martin, Ramsauer, Hubert, Unterthiner, Thomas, Nessler, Bernhard, and Hochreiter, Sepp. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems 30* (2017).

[92] Hou, Xinying, Forsyth, Carol, Andrews-Todd, Jessica, Rice, James, Cai, Zhiqiang, Jiang, Yang, Zapata-Rivera, Diego, and Graesser, Art. An llm-enhanced multi-agent architecture for conversation-based assessment. In *Artificial Intelligence in Education: 26th International Conference, AIED 2025, Palermo, Italy, July 22–26, 2025, Proceedings, Part II* (Berlin, Heidelberg, 2025), Springer-Verlag, p. 119–134.

[93] Hu, Edward J, yelong shen, Wallis, Phillip, Allen-Zhu, Zeyuan, Li, Yuanzhi, Wang, Shean, Wang, Lu, and Chen, Weizhu. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations* (2022).

[94] Huang, Zhenya, Liu, Qi, Chen, Enhong, Zhao, Hongke, Gao, Mingyong, Wei, Si, Su, Yu, and Hu, Guoping. Question difficulty prediction for reading problems in standard tests. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence* (2017), AAAI'17, AAAI Press, p. 1352–1359.

[95] Huang, Zhenya, Liu, Qi, Zhai, Chengxiang, Yin, Yu, Chen, Enhong, Gao, Weibo, and Hu, Guoping. Exploring multi-objective exercise recommendations in online education systems. In *Proceedings of the 28th ACM international conference on information and knowledge management* (2019), pp. 1261–1270.

[96] Ikram, Fareya, Scarlatos, Alexander, and Lan, Andrew. Exploring LLMs for predicting tutor strategy and student outcomes in dialogues. In *Proceedings of the 20th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2025)* (Vienna, Austria, July 2025), Association for Computational Linguistics, pp. 765–779.

[97] Ipçi, Eylül, Nazaretsky, Tanya, and Käser, Tanja. Leveraging knowledge profiles and generative ai for realistic student response generation. In *Artificial Intelligence in Education. Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium, Blue Sky, and WideAIED* (Cham, 2025), Springer Nature Switzerland, pp. 143–151.

[98] Jia, Qinjin, Young, Mitchell, Xiao, Yunkai, Cui, Jialin, Liu, Chengyuan, Rashid, Parvez, and Gehringer, Edward. Insta-reviewer: A data-driven approach for generating instant feedback on students' project reports. *International Educational Data Mining Society* (2022).

[99] Jiao, Ying, Shridhar, Kumar, Cui, Peng, Zhou, Wangchunshu, and Sachan, Mrinmaya. Automatic educational question generation with difficulty level controls. In *International Conference on Artificial Intelligence in Education* (2023), Springer, pp. 476–488.

[100] Jin, Hyoungwook, Lee, Seonghee, Shin, Hyungyu, and Kim, Juho. Teach ai how to code: Using large language models as teachable agents for programming education. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (2024), pp. 1–28.

[101] Jordan, Pamela, Rosé, Carolyn Penstein, and VanLehn, Kurt. Tools for authoring tutorial dialogue knowledge. In *Proceedings of AI in Education 2001 Conference* (2001), IOS Press Amsterdam.

[102] Jung, Heeseok, Yoo, Jaesang, Yoon, Yohaan, and Jang, Yeonju. Clst: Cold-start mitigation in knowledge tracing by aligning a generative language model as a students' knowledge tracer, 2024.

[103] Kakarla, Sanjit, Thomas, Danielle, Lin, Jionghao, Gupta, Shivang, and Koedinger, Ken. Using large language models to assess tutors' performance in reacting to students making math errors. In *AI for Education: Bridging Innovation and Responsibility at the 38th AAAI Annual Conference on AI* (2024).

[104] Kalajdzievski, Damjan. A rank stabilization scaling factor for fine-tuning with lora. *arXiv preprint arXiv:2312.03732* (2023).

[105] Kapoor, Radhika, Truong, Sang T, Haber, Nick, Ruiz-Primo, Maria Araceli, and Domingue, Benjamin W. Prediction of item difficulty for reading comprehension items by creation of annotated item repository. *arXiv preprint arXiv:2502.20663* (2025).

[106] Karumbaiah, Shamya, Borchers, Conrad, Shou, Tianze, Falhs, Ann-Christin, Liu, Pinyang, Nagashima, Tomohiro, Rummel, Nikol, and Aleven, Vincent. A spatiotemporal analysis of teacher practices in supporting student learning and engagement in an ai-enabled classroom. In *International conference on artificial intelligence in education* (2023), Springer, pp. 450–462.

[107] Käser, Tanja, and Alexandron, Giora. Simulated learners in educational technology: A systematic literature review and a turing-like test. *International Journal of Artificial Intelligence in Education 34*, 2 (2024), 545–585.

[108] Kendall, Maurice George. Rank correlation methods.

[109] Khan, Salman. *The one world schoolhouse: Education reimagined*. Twelve, 2012.

[110] Khan Academy. Supercharge your teaching experience with khanmigo. Online: `https://www.khanmigo.ai/`, 2023.

[111] Kim, Jiho, Choi, Junseong, Chay, Woosog, Kyung, Daeun, Kwon, Yeonsu, Jo, Yohan, and Choi, Edward. Propersim: Developing proactive and personalized ai assistants through user-assistant simulation, 2025.

[112] Kim, Kyuyoung, Seo, Ah Jeong, Liu, Hao, Shin, Jinwoo, and Lee, Kimin. Margin matching preference optimization: Enhanced model alignment with granular feedback. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 13554–13570.

[113] King, Jules, Burleigh, L, Woodhead, Simon, Kon, Panagiota, Baffour, Perpetual, Crossley, Scott, Reade, Walter, and Demkin, Maggie. Eedi - mining misconceptions in mathematics. `https://kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics`, 2024. Kaggle.

[114] Kloft, Marius, Stiehler, Felix, Zheng, Zhilin, and Pinkwart, Niels. Predicting mooc dropout over weeks using machine learning methods. In *Proceedings of the EMNLP 2014 workshop on analysis of large scale social interaction in MOOCs* (2014), pp. 60–65.

[115] Kochmar, Ekaterina, Vu, Dung Do, Belfer, Robert, Gupta, Varun, Serban, Iulian Vlad, and Pineau, Joelle. Automated personalized feedback improves learning gains in an intelligent tutoring system. In *International conference on artificial intelligence in education* (2020), Springer, pp. 140–146.

[116] Koedinger, K., and Aleven, V. An interview reflection on "intelligent tutoring goes to school in the big city". *International Journal of Artificial Intelligence in Education 26* (2016), 13–24.

[117] Krupp, Lars, Steinert, Steffen, Kiefer-Emmanouilidis, Maximilian, Avila, Karina E, Lukowicz, Paul, Kuhn, Jochen, Küchemann, Stefan, and Karolus, Jakob. Challenges and opportunities of moderating usage of large language models in education. In *Proceedings of the International Conference on Mobile and Ubiquitous Multimedia* (2024), pp. 249–254.

[118] Kurdi, Ghader, Leo, Jared, Parsia, Bijan, Sattler, Uli, and Al-Emari, Salam. A systematic review of automatic question generation for educational purposes. *International Journal of Artificial Intelligence in Education 30* (2020), 121–204.

[119] Kyung, Daeun, Chung, Hyunseung, Bae, Seongsu, Kim, Jiho, Sohn, Jae Ho, Kim, Taerim, Kim, Soo Kyung, and Choi, Edward. Patientsim: A persona-driven simulator for realistic doctor-patient interactions. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track* (2025).

[120] Lalor, John P., Wu, Hao, and Yu, Hong. Learning latent parameters without human response patterns: Item response theory with artificial crowds. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (Hong Kong, China, Nov. 2019), Association for Computational Linguistics, pp. 4249–4259.

[121] Lan, Andrew S, and Baraniuk, Richard G. A contextual bandits framework for personalized learning action selection. In *EDM* (2016), pp. 424–429.

[122] Lan, Andrew S, Studer, Christoph, and Baraniuk, Richard G. Time-varying learning and content analytics via sparse factor analysis. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining* (2014), pp. 452–461.

[123] Lan, Andrew S, Waters, Andrew E, Studer, Christoph, and Baraniuk, Richard G. Sparse factor analysis for learning and content analytics. *Journal of Machine Learning Research 15*, 57 (2014), 1959–2008.

[124] Learning, Carnegie. Meet mathia. Online: `https://discover.carnegielearning.com/meet-mathia`, 2026.

[125] Lee, Seanie, Cheng, Jianpeng, Driesen, Joris, Coca, Alexandru, and Johannsen, Anders. Effective and efficient conversation retrieval for dialogue state tracking with implicit text summaries. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (Mexico City, Mexico, June 2024), Association for Computational Linguistics, pp. 96–111.

[126] Lee, Unggi, Bae, Jiyeong, Kim, Dohee, Lee, Sookbun, Park, Jaekwon, Ahn, Taekyung, Lee, Gunho, Stratton, Damji, and Kim, Hyeoncheol. Language model can do knowledge tracing: Simple but effective method to integrate language model and knowledge tracing task, 2024.

[127] Lee Rodgers, Joseph, and Nicewander, W Alan. Thirteen ways to look at the correlation coefficient. *The American Statistician 42*, 1 (1988), 59–66.

[128] Leelawong, Krittaya, and Biswas, Gautam. Designing learning by teaching agents: The betty's brain system. *International journal of artificial intelligence in education 18*, 3 (2008), 181–208.

[129] Lewis, Arthur, and Smith, David. Defining higher order thinking. *Theory into practice 32*, 3 (1993), 131–137.

[130] Li, Hang, Xu, Tianlong, Tang, Jiliang, and Wen, Qingsong. Knowledge tagging system on math questions via llms with flexible demonstration retriever, 2024.

[131] Li, Haoxuan, Yu, Jifan, Cong, Xin, Dang, Yang, Zhang-Li, Daniel, Mi, Lu, Zhan, Yisi, Liu, Huiqin, and Liu, Zhiyuan. Which type of students can llms act? investigating authentic simulation with graph-based human–ai collaborative system. *Investigating Authentic Simulation with Graph-based Human–AI Collaborative System* (2025).

[132] Li, Haoxuan, Yu, Jifan, Ouyang, Yuanxin, Liu, Zhuang, Rong, Wenge, Li, Juanzi, and Xiong, Zhang. Explainable few-shot knowledge tracing. *arXiv preprint arXiv:2405.14391* (2024).

[133] Li, Nan, Cohen, William W, Koedinger, Kenneth R, and Matsuda, Noboru. A machine learning approach for automatic student model discovery. In *Edm* (2011), ERIC, pp. 31–40.

[134] Lieb, Anna, and Goel, Toshali. Student interaction with newtbot: An llm-as-tutor chatbot for secondary physics education. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (2024), pp. 1–8.

[135] Lin, Chin-Yew. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out* (Barcelona, Spain, July 2004), Association for Computational Linguistics, pp. 74–81.

[136] Lin, Xi, Zhen, Hui-Ling, Li, Zhenhua, Zhang, Qing-Fu, and Kwong, Sam. Pareto multi-task learning. *Advances in neural information processing systems 32* (2019).

[137] Liu, Jiayu, Huang, Zhenya, Xiao, Tong, Sha, Jing, Wu, Jinze, Liu, Qi, Wang, Shijin, and Chen, Enhong. SocraticLM: Exploring socratic personalized teaching with large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems* (2024).

[138] Liu, Naiming, Sonkar, Shashank, Wang, Zichao, Woodhead, Simon, and Baraniuk, Richard G. Novice learner and expert tutor: Evaluating math reasoning abilities of large language models with misconceptions. *arXiv preprint arXiv:2310.02439* (2023).

[139] Liu, Naiming, Wang, Zichao, Baraniuk, Richard, and Lan, Andrew. Open-ended knowledge tracing for computer science education. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing* (2022), pp. 3849–3862.

[140] Liu, Qi, Huang, Zhenya, Yin, Yu, Chen, Enhong, Xiong, Hui, Su, Yu, and Hu, Guoping. Ekt: Exercise-aware knowledge tracing for student performance prediction. *IEEE Trans. Knowl. Data Eng. 33*, 1 (2019), 100–115.

[141] Liu, Yun-En, Mandel, Travis, Brunskill, Emma, and Popovic, Zoran. Trading off scientific knowledge and user learning with multi-armed bandits. In *EDM* (2014), pp. 161–168.

[142] Liu, Yunting, Bhandari, Shreya, and Pardos, Zachary A. Leveraging llm respondents for item evaluation: A psychometric analysis. *British Journal of Educational Technology 56*, 3 (2025), 1028–1052.

[143] Liu, Zhengyuan, Yin, Stella Xin, Lin, Geyu, and Chen, Nancy F. Personality-aware student simulation for conversational intelligent tutoring systems. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 626–642.

[144] Liu, Zitao, Liu, Qiongqiong, Chen, Jiahao, Huang, Shuyan, and Luo, Weiqi. simpleKT: A simple but tough-to-beat baseline for knowledge tracing. In *The Eleventh International Conference on Learning Representations* (2023).

[145] Liu, Zitao, Liu, Qiongqiong, Chen, Jiahao, Huang, Shuyan, Tang, Jiliang, and Luo, Weiqi. pykt: A python library to benchmark deep learning based knowledge tracing models. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track* (2022).

[146] Loaiza-Ganem, Gabriel, and Cunningham, John P. The continuous bernoulli: fixing a pervasive error in variational autoencoders. *Advances in Neural Information Processing Systems 32* (2019).

[147] Loginova, Ekaterina, Benedetto, Luca, Benoit, Dries, and Cremonesi, Paolo. Towards the application of calibrated transformers to the unsupervised estimation of question difficulty from text. In *Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021)* (Held Online, Sept. 2021), INCOMA Ltd., pp. 846–855.

[148] Lu, Xinyi, and Wang, Xu. Generative students: Using llm-simulated student profiles to support question item evaluation. In *Proceedings of the Eleventh ACM conference on learning@ Scale* (2024), pp. 16–27.

[149] Lucy, Li, August, Tal, Wang, Rose E, Soldaini, Luca, Allison, Courtney, and Lo, Kyle. MathFish: Evaluating language model math reasoning via grounding in educational curricula. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 5644–5673.

[150] Létourneau, Angélique, Martineau, Marion Deslandes, Charland, Patrick, Karran, John Alexander, Boasen, Jared, and Léger, Pierre-Majorique. A systematic review of ai-driven intelligent tutoring systems (its) in k-12 education. *NPJ Science of Learning 10* (2025).

[151] Ma, Qianou, Shen, Hua, Koedinger, Kenneth, and Wu, Sherry Tongshuang. How to teach programming in the ai era? using llms as a teachable agent for debugging. In *Artificial Intelligence in Education* (Cham, 2024), Springer Nature Switzerland, pp. 265–279.

[152] Macina, Jakub, Daheim, Nico, Chowdhury, Sankalan, Sinha, Tanmay, Kapur, Manu, Gurevych, Iryna, and Sachan, Mrinmaya. MathDial: A dialogue tutoring dataset with rich pedagogical properties grounded in math reasoning problems. In *Findings of the Association for Computational Linguistics: EMNLP 2023* (Singapore, Dec. 2023), Association for Computational Linguistics, pp. 5602–5621.

[153] Maclellan, Christopher J, Harpstead, Erik, Patel, Rony, and Koedinger, Kenneth R. The apprentice learner architecture: Closing the loop between learning theory and educational data. *International Educational Data Mining Society* (2016).

[154] Maeda, Hotaka. Field-testing multiple-choice questions with ai examinees: English grammar items. *Educational and Psychological Measurement 85*, 2 (2025), 221–244.

[155] Maier, Cristina, Baker, Ryan, and Stalzer, Steve. Challenges to applying performance factor analysis to existing learning systems. In *International Conference on Computers in Education* (2021).

[156] Makatchev, Maxim, and VanLehn, Kurt. Combining bayesian networks and formal reasoning for semantic classification of student utterances. In *Proceedings of the 2007 conference on Artificial Intelligence in Education: Building Technology Rich Learning Contexts That Work* (2007), pp. 307–314.

[157] Markel, Julia M, Opferman, Steven G, Landay, James A, and Piech, Chris. Gpteach: Interactive ta training with gpt-based students. In *Proceedings of the tenth acm conference on learning@ scale* (2023), pp. 226–236.

[158] Martynova, Daria, Macina, Jakub, Daheim, Nico, Yalcin, Nilay, Zhang, Xiaoyu, and Sachan, Mrinmaya. Can LLMs effectively simulate human learners? teachers' insights from tutoring LLM students. In *Proceedings of the 20th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2025)* (Vienna, Austria, July 2025), Association for Computational Linguistics, pp. 100–117.

[159] Matsuda, Noboru, Cohen, William W, and Koedinger, Kenneth R. Teaching the teacher: tutoring simstudent leads to more effective cognitive tutor authoring. *International Journal of Artificial Intelligence in Education 25*, 1 (2015), 1–34.

[160] Matsuda, Noboru, Keiser, Victoria, Raizada, Rohan, Tu, Arthur, Stylianides, Gabriel, Cohen, William W, and Koedinger, Kenneth R. Learning by teaching simstudent: Technical accomplishments and an initial use with students. In *International conference on intelligent tutoring systems* (2010), Springer, pp. 317–326.

[161] McCrae, Robert R, and John, Oliver P. An introduction to the five-factor model and its applications. *Journal of personality 60*, 2 (1992), 175–215.

[162] McGraw Hill. ALEKS, 2026.

[163] McNichols, Hunter, Ikram, Fareya, and Lan, Andrew. The studychat dataset: Student dialogues with chatgpt in an artificial intelligence course, 2025.

[164] Mehrens, William A, and Phillips, Susan E. Sensitivity of item difficulties to curricular validity. *Journal of Educational Measurement 24*, 4 (1987), 357–370.

[165] Miller, Pepper, and DiCerbo, Kristen. Llm based math tutoring: Challenges and dataset, Jul 2024.

[166] Mitrović, Antonija. Experiences in implementing constraint-based modeling in sql-tutor. In *International conference on intelligent tutoring systems* (1998), Springer, pp. 414–423.

[167] Molenaar, Inge, Horvers, Anne, and Baker, Ryan S. What can moment-by-moment learning curves tell about students' self-regulated learning? *Learning and Instruction 72* (2021), 101206.

[168] Munshi, Anabil, Biswas, Gautam, Baker, Ryan, Ocumpaugh, Jaclyn, Hutt, Stephen, and Paquette, Luc. Analysing adaptive scaffolds that help students develop self-regulated learning behaviours. *Journal of Computer Assisted Learning 39*, 2 (2023), 351–368.

[169] Muraki, Eiji. A generalized partial credit model: Application of an em algorithm. *Applied psychological measurement 16*, 2 (1992), 159–176.

[170] Muraki, Eiji. Information functions of the generalized partial credit model. *ETS Research Report Series 1993*, 1 (1993), i–12.

[171] Murray, R Charles, VanLehn, Kurt, and Mostow, Jack. Looking ahead to select tutorial actions: A decision-theoretic approach. *International Journal of Artificial Intelligence in Education 14*, 3-4 (2004), 235–278.

[172] Nguyen, Bang, Du, Tingting, Yu, Mengxia, Angrave, Lawrence, and Jiang, Meng. QG-SMS: Enhancing test item analysis via student modeling and simulation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Vienna, Austria, July 2025), Association for Computational Linguistics, pp. 26152–26168.

[173] Nickow, Andre, Oreopoulos, Philip, and Quan, Vincent. The impressive effects of tutoring on prek-12 learning: A systematic review and meta-analysis of the experimental evidence. Working Paper 27476, National Bureau of Economic Research, July 2020.

[174] North, Kai, and Ormerod, Christopher. Cyborg data: Merging human with ai generated training data, 2025.

[175] Nye, Benjamin D, Graesser, Arthur C, and Hu, Xiangen. Autotutor and family: A review of 17 years of natural language tutoring. *International Journal of Artificial Intelligence in Education 24* (2014), 427–469.

[176] Nye, Benjamin D, Mee, Dillon, and Core, Mark G. Generative large language models for dialog-based tutoring: An early consideration of opportunities and concerns. In *LLM@ AIED* (2023), pp. 78–88.

[177] Ohlsson, Stellan. Constraint-based student modelling. *Journal of Interactive Learning Research 3*, 4 (1992), 429.

[178] OpenAI. Hello gpt-4o. Online: `https://openai.com/index/hello-gpt-4o/`, May 2024.

[179] OpenAI. Introducing gpt-4.1 in the api. Online: `https://openai.com/index/gpt-4-1/`, 2025.

[180] OpenAI. Introducing gpt-5. Online: https://openai.com/index/introducing-gpt-5/, 2025.

[181] OpenAI. Introducing study mode. Online: https://openai.com/index/chatgpt-study-mode/, 2025.

[182] OpenAI. Sycophancy in gpt-4o: what happened and what we're doing about it. Online: https://openai.com/index/sycophancy-in-gpt-4o/, 2025.

[183] OpenAI. New tools for understanding ai and learning outcomes. Online: https://openai.com/index/understanding-ai-and-learning-outcomes/, 2026.

[184] Organizers, Challenge. The 2nd csedm data challenge. Online: https://sites.google.com/ncsu.edu/csedm-dc-2021/, 2021.

[185] Pal Chowdhury, Sankalan, Zouhar, Vilém, and Sachan, Mrinmaya. Autotutor meets large language models: A language model tutor with rich pedagogy and guardrails. In *Proceedings of the Eleventh ACM Conference on Learning@ Scale* (2024), pp. 5–15.

[186] Pan, Sitong, Schmucker, Robin, Garcia Bulle Bueno, Bernardo, Llanes, Salome Aguilar, Albo Alarcón, Fernanda, Zhu, Hangxiao, Teo, Adam, and Xia, Meng. Tutorup: What if your students were simulated? training tutors to address engagement challenges in online learning. In *Proceedings of the 2025 CHI conference on human factors in computing systems* (2025), pp. 1–18.

[187] Pandey, Shalini, and Karypis, George. A self attentive model for knowledge tracing. In *Proc. Int. Conf. Educ. Data Mining* (July 2019), pp. 384–389.

[188] Parikh, Nisarg, Scarlatos, Alexander, Fernandez, Nigel, Woodhead, Simon, and Lan, Andrew. LookAlike: Consistent distractor generation in math MCQs. In *Proceedings of the 20th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2025)* (Vienna, Austria, July 2025), Association for Computational Linguistics, pp. 294–311.

[189] Park, Minju, Kim, Sojung, Lee, Seunghyun, Kwon, Soonwoo, and Kim, Kyuseok. Empowering personalized learning through a conversation-based tutoring system with student modeling. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (2024), pp. 1–10.

[190] Pavlik Jr, Philip, Cen, Hao, and Koedinger, Kenneth. Performance factors analysis–A new alternative to knowledge tracing. In *Proc. Int. Conf. Artif. Intell. Educ.* (2009).

[191] Pelánek, Radek. Bayesian knowledge tracing, logistic models, and beyond: an overview of learner modeling techniques. *User modeling and user-adapted interaction 27* (2017), 313–350.

[192] Pelánek, Radek, and Jarusek, Petr. Student modeling based on problem solving times. *International Journal of Artificial Intelligence in Education 25* (2015), 493–519.

[193] Perczel, Janos, Chow, Jin, and Demszky, Dorottya. Teachlm: Post-training llms for education using authentic learning data. *arXiv preprint arXiv:2510.05087* (2025).

[194] Piech, Chris, Bassen, Jonathan, Huang, Jonathan, Ganguli, Surya, Sahami, Mehran, Guibas, Leonidas J, and Sohl-Dickstein, Jascha. Deep knowledge tracing. In *Proc. NeurIPS* (2015), pp. 505–513.

[195] Pillutla, Krishna, Swayamdipta, Swabha, Zellers, Rowan, Thickstun, John, Welleck, Sean, Choi, Yejin, and Harchaoui, Zaid. Mauve: Measuring the gap between neural text and human text using divergence frontiers. *Advances in Neural Information Processing Systems 34* (2021), 4816–4828.

[196] Prihar, Ethan and Others. Comparing different approaches to generating mathematics explanations using large language models. In *International Conference on Artificial Intelligence in Education* (2023), pp. 290–295.

[197] Rafailov, Rafael, Sharma, Archit, Mitchell, Eric, Manning, Christopher D, Ermon, Stefano, and Finn, Chelsea. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems 36* (2024).

[198] Rafferty, Anna N, Brunskill, Emma, Griffiths, Thomas L, and Shafto, Patrick. Faster teaching via pomdp planning. *Cognitive science 40*, 6 (2016), 1290–1332.

[199] Rasch, Georg. *Probabilistic models for some intelligence and attainment tests.* ERIC, 1993.

[200] Reimers, Nils, and Gurevych, Iryna. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* (11 2019), Association for Computational Linguistics.

[201] Reyes, Diego, Jimenez, Abelino, Dartnell, Pablo, Lions, Séverin, and Ríos, Sebastián. Multiple-choice questions difficulty prediction with neural networks. In *International Conference in Methodologies and intelligent Systems for Techhnology Enhanced Learning* (2023), Springer, pp. 11–22.

[202] Ross, Alexis, and Andreas, Jacob. Toward in-context teaching: Adapting examples to students' misconceptions. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Bangkok, Thailand, Aug. 2024), Association for Computational Linguistics, pp. 13283–13310.

[203] Ross, Alexis, and Andreas, Jacob. Learning to make mistakes: Modeling incorrect student thinking and key errors, 2025.

[204] Ross, Alexis, Srivastava, Megha, Blanchard, Jeremiah, and Andreas, Jacob. Modeling student learning with 3.8 million program traces. *arXiv preprint arXiv:2510.05056* (2025).

[205] Scarlatos, Alexander, Baker, Ryan S., and Lan, Andrew. Exploring knowledge tracing in tutor-student dialogues using llms. In *Proceedings of the 15th International Learning Analytics and Knowledge Conference* (New York, NY, USA, 2025), LAK '25, Association for Computing Machinery, p. 249–259.

[206] Scarlatos, Alexander, Fernandez, Nigel, Ormerod, Christopher, Lottridge, Susan, and Lan, Andrew. SMART: Simulated students aligned with item response theory for question difficulty prediction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (Suzhou, China, Nov. 2025), Association for Computational Linguistics, pp. 25082–25105.

[207] Scarlatos, Alexander, Lee, Jaewook, Woodhead, Simon, and Lan, Andrew. Simulated students in tutoring dialogues: Substance or illusion?, 2026.

[208] Scarlatos, Alexander, Liu, Naiming, Lee, Jaewook, Baraniuk, Richard, and Lan, Andrew. Training llm-based tutors to improve student learning outcomes in dialogues. In *Artificial Intelligence in Education* (Cham, 2025), Springer Nature Switzerland, pp. 251–266.

[209] Scarlatos, Alexander, Smith, Digory, Woodhead, Simon, and Lan, Andrew. Improving the validity of automatically generated feedback via reinforcement learning. In *Artificial Intelligence in Education* (Cham, 2024), Springer Nature Switzerland, pp. 280–294.

[210] Scarlatos, Alexander, Wu, Yusong, Simon, Ian, Roberts, Adam, Cooijmans, Tim, Jaques, Natasha, Tarakajian, Cassie, and Huang, Anna. Realjam: Real-time human-ai music jamming with reinforcement learning-tuned transformers. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA 2025)* (2025).

[211] Schmucker, Robin, Xia, Meng, Azaria, Amos, and Mitchell, Tom. Ruffle&riley: Insights from designing and evaluating a large language model-based conversational tutoring system. In *International Conference on Artificial Intelligence in Education* (2024), Springer, pp. 75–90.

[212] Sharma, Paras, and Li, Qichang. Designing simulated students to emulate learner activity data in an open-ended learning environment. In *Proceedings of the 17th International Conference on Educational Data Mining* (Atlanta, Georgia, USA, July 2024), International Educational Data Mining Society, pp. 986–989.

[213] Shen, Jia Tracy, Yamashita, Michiharu, Prihar, Ethan, Heffernan, Neil, Wu, Xintao, McGrew, Sean, and Lee, Dongwon. Classifying math knowledge components via task-adaptive pre-trained bert. In *Artificial Intelligence in Education: 22nd International Conference, AIED 2021, Utrecht, The Netherlands, June 14–18, 2021, Proceedings, Part I 22* (2021), Springer, pp. 408–419.

[214] Shi, Yang, Schmucker, Robin, Chi, Min, Barnes, Tiffany, and Price, Thomas. Kc-finder: Automated knowledge component discovery for programming problems. *International Educational Data Mining Society* (2023).

[215] Shimmei, Machi, and Matsuda, Noboru. Automatic question generation for evidence-based online courseware engineering. In *iTextbooks@ AIED* (2022), pp. 18–25.

[216] Shin, Dongmin, Shim, Yugeun, Yu, Hangyeol, Lee, Seewoo, Kim, Byungsoo, and Choi, Youngduck. Saint+: Integrating temporal features for ednet correctness prediction. In *11th Int. Learn. Analytics Knowl. Conf.* (2021), pp. 490–496.

[217] Shu, Matthew, Balepur, Nishant, Feng, Shi, and Boyd-Graber, Jordan Lee. KARL: Knowledge-aware retrieval and representations aid retention and learning in students. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 14161–14178.

[218] Smith III, John P, DiSessa, Andrea A, and Roschelle, Jeremy. Misconceptions reconceived: A constructivist analysis of knowledge in transition. *j. learn. sci. 3*, 2 (1994), 115–163.

[219] Snoddy, George S. Learning and stability: a psychophysiological analysis of a case of motor learning with clinical applications. *Journal of Applied Psychology 10*, 1 (1926), 1.

[220] Song, Jiajia, Guo, Zhihan, and Lin, Jionghao. Simulating novice students using machine unlearning and relearning in large language models. *arXiv preprint arXiv:2603.26142* (2026).

[221] Sonkar, Shashank, Chen, Xinghe, Liu, Naiming, Baraniuk, Richard G, and Sachan, Mrinmaya. Llm-based cognitive models of students with misconceptions. *arXiv preprint arXiv:2410.12294* (2024).

[222] Sonkar, Shashank, Liu, Naiming, and Baraniuk, Richard. Student data paradox and curious case of single student-tutor model: Regressive side effects of training LLMs for personalized learning. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 15543–15553.

[223] Sonkar, Shashank, Liu, Naiming, Mallick, Debshila, and Baraniuk, Richard. Class: A design framework for building intelligent tutoring systems based on learning science principles. In *Findings of the Association for Computational Linguistics: EMNLP 2023* (2023), pp. 1941–1961.

[224] Sonkar, Shashank, Ni, Kangqi, Chaudhary, Sapana, and Baraniuk, Richard. Pedagogical alignment of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 13641–13650.

[225] Srivastava, Megha, and Goodman, Noah. Question generation for adaptive education. In *Proc. Annual Meeting of the Association for Computational Linguistics and the International Joint Conference on Natural Language Processing (Volume 2: Short Papers)* (2021), pp. 692–701.

[226] Stamper, John, Xiao, Ruiwei, and Hou, Xinying. Enhancing llm-based feedback: Insights from intelligent tutoring systems and the learning sciences. In *International Conference on Artificial Intelligence in Education* (2024), Springer, pp. 32–43.

[227] Steiss, Jacob, Tate, Tamara, Graham, Steve, Cruz, Jazmin, Hebert, Michael, Wang, Jiali, Moon, Youngsun, Tseng, Waverly, Warschauer, Mark, and Olson, Carol Booth. Comparing the quality of human and chatgpt feedback of students' writing. *Learning and Instruction 91* (2024), 101894.

[228] Suresh, Abhijit, Jacobs, Jennifer, Harty, Charis, Perkoff, Margaret, Martin, James H., and Sumner, Tamara. The TalkMoves dataset: K-12 mathematics lesson transcripts annotated for teacher and student discursive moves. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference* (June 2022), pp. 4654–4662.

[229] Suresh, Abhijit, Jacobs, Jennifer, Perkoff, Margaret, Martin, James H, and Sumner, Tamara. Fine-tuning transformers with additional context to classify discursive moves in mathematics classrooms. In *Proc. Workshop on Innovative Use of NLP for Building Educational Applications* (2022).

[230] Tack, Anaïs, Buseyne, Siem, Chen, Changsheng, D'hondt, Robbe, De Vrindt, Michiel, Gharahighehi, Alireza, Metwaly, Sameh, Nakano, Felipe Kenji, and Noreillie, Ann-Sophie. ITEC at BEA 2024 shared task: Predicting difficulty and response time of medical exam questions with statistical, machine learning, and language models. In *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)* (Mexico City, Mexico, June 2024), Association for Computational Linguistics, pp. 512–521.

[231] Team, LearnLM, Wang, Albert, Rysbek, Aliya, Huber, Andrea, Nambiar, Anjali, Kenolty, Anna, Caulfield, Ben, Lilley-Draper, Beth, Groot, Bibi, Veprek, Brian, et al. Ai tutoring can safely and effectively support students: An exploratory rct in uk classrooms. *arXiv preprint arXiv:2512.23633* (2025).

[232] Ueno, Maomi, and Miyazawa, Yoshimitsu. Uniform adaptive testing using maximum clique algorithm. In *Artificial Intelligence in Education: 20th International Conference, AIED 2019, Chicago, IL, USA, June 25-29, 2019, Proceedings, Part I* (Berlin, Heidelberg, 2019), Springer-Verlag, p. 482–493.

[233] Uto, Masaki, Tomikawa, Yuto, and Suzuki, Ayaka. Question difficulty prediction based on virtual test-takers and item response theory. *Workshop on Automatic Evaluation of Learning and Assessment Content* (2024).

[234] Vail, Alexandria Katarina, and Boyer, Kristy Elizabeth. Identifying effective moves in tutoring: On the refinement of dialogue act annotation schemes. In *Intelligent Tutoring Systems* (Cham, 2014), Springer International Publishing, pp. 199–209.

[235] VanLehn, Kurt, Jordan, Pamela W, Rosé, Carolyn P, Bhembe, Dumisizwe, Böttner, Michael, Gaydos, Andy, Makatchev, Maxim, Pappuswamy, Umarani, Ringenberg, Michael, Roque, Antonio, et al. The architecture of why2-atlas: A coach for qualitative physics essay writing. In *Intelligent Tutoring Systems: 6th International Conference, ITS 2002 Biarritz, France and San Sebastian, Spain, June 2–7, 2002 Proceedings 6* (2002), Springer, pp. 158–167.

[236] von Ahn, Luis. Duolingo: learn a language for free while helping to translate the web. In *Proceedings of the 2013 International Conference on Intelligent User Interfaces* (New York, NY, USA, 2013), IUI '13, Association for Computing Machinery, p. 1–2.

[237] Wang, Jian, Dai, Yinpei, Zhang, Yichi, Ma, Ziqiao, Li, Wenjie, and Chai, Joyce. Training turn-by-turn verifiers for dialogue tutoring agents: The curious case of LLMs as your coding tutors. In *Findings of the Association for Computational Linguistics: ACL 2025* (Vienna, Austria, July 2025), Association for Computational Linguistics, pp. 12416–12436.

[238] Wang, Junling, Macina, Jakub, Daheim, Nico, Pal Chowdhury, Sankalan, and Sachan, Mrinmaya. Book2Dial: Generating teacher student interactions from textbooks for cost-effective development of educational chatbots. In *Findings of the Association for Computational Linguistics: ACL 2024* (Bangkok, Thailand, Aug. 2024), Association for Computational Linguistics, pp. 9707–9731.

[239] Wang, Kaiwen, Kidambi, Rahul, Sullivan, Ryan, Agarwal, Alekh, Dann, Christoph, Michi, Andrea, Gelmi, Marco, Li, Yunxuan, Gupta, Raghav, Dubey, Kumar Avinava, Rame, Alexandre, Ferret, Johan, Cideron, Geoffrey, Hou, Le, Yu, Hongkun, Ahmed, Amr, Mehta, Aranyak, Hussenot, Leonard, Bachem, Olivier, and Leurent, Edouard. Conditional language policy: A general framework for steerable multi-objective finetuning. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (Miami, Florida, USA, Nov. 2024), Association for Computational Linguistics, pp. 2153–2186.

[240] Wang, Rose, Zhang, Qingyang, Robinson, Carly, Loeb, Susanna, and Demszky, Dorottya. Bridging the novice-expert gap via models of decision-making: A case study on remediating math mistakes. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (2024), pp. 2174–2199.

[241] Wang, Rose E, Ribeiro, Ana T, Robinson, Carly D, Loeb, Susanna, and Demszky, Dora. Tutor copilot: A human-ai approach for scaling real-time expertise. *arXiv preprint arXiv:2410.03017* (2024).

[242] Warner, Benjamin, Chaffin, Antoine, Clavié, Benjamin, Weller, Orion, Hallström, Oskar, Taghadouini, Said, Gallagher, Alexis, Biswas, Raja, Ladhak, Faisal, Aarsen, Tom, Cooper, Nathan, Adams, Griffin, Howard, Jeremy, and Poli, Iacopo. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024.

[243] Warshauer, Hiroko Kawaguchi. Productive struggle in middle school mathematics classrooms. *Journal of Mathematics Teacher Education 18*, 4 (2015), 375–400.

[244] Weitekamp, Daniel, N. Siddiqui, Momin, and J. MacLellan, Christopher. Tutorgym: A testbed for evaluating ai agents as tutors and students. In *Artificial Intelligence in Education: 26th International Conference, AIED 2025, Palermo, Italy, July 22–26, 2025, Proceedings, Part III* (Berlin, Heidelberg, 2025), Springer-Verlag, p. 361–376.

[245] Weitekamp, Daniel, Ye, Zihuiwen, Rachatasumrit, Napol, Harpstead, Erik, and Koedinger, Kenneth. Investigating differential error types between human and simulated learners. In *International conference on artificial intelligence in education* (2020), Springer, pp. 586–597.

[246] Whitehill, Jacob. Understanding act-r - an outsider's perspective, 2013.

[247] Winne, Philip H, Baker, Ryan SJd, et al. The potentials of educational data mining for researching metacognition, motivation and self-regulated learning. *Journal of Educational Data Mining 5*, 1 (2013), 1–8.

[248] Wolf, Thomas, Debut, Lysandre, Sanh, Victor, Chaumond, Julien, Delangue, Clement, Moi, Anthony, Cistac, Pierric, Rault, Tim, Louf, Remi, Funtowicz, Morgan, Davison, Joe, Shleifer, Sam, von Platen, Patrick, Ma, Clara, Jernite, Yacine, Plu, Julien, Xu, Canwen, Le Scao, Teven, Gugger, Sylvain, Drame, Mariama, Lhoest, Quentin, and Rush, Alexander. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (Online, Oct. 2020), Association for Computational Linguistics, pp. 38–45.

[249] Woolf, Beverly, Burleson, Winslow, Arroyo, Ivon, Dragon, Toby, Cooper, David, and Picard, Rosalind. Affect-aware tutors: recognising and responding to student affect. *International Journal of Learning Technology 4*, 3-4 (2009), 129–164.

[250] Wu, Tao, Chen, Jingyuan, Lin, Wang, Li, Mengze, Zhu, Yumeng, Li, Ang, Kuang, Kun, and Wu, Fei. Embracing imperfection: Simulating students with diverse cognitive levels using LLM-based agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Vienna, Austria, July 2025), Association for Computational Linguistics, pp. 9887–9908.

[251] Wu, Yusong, Cooijmans, Tim, Kastner, Kyle, Roberts, Adam, Simon, Ian, Scarlatos, Alexander, Donahue, Chris, Tarakajian, Cassie, Omidshafiei, Shayegan, Courville, Aaron, Castro, Pablo Samuel, Jaques, Natasha, and Huang, Cheng-Zhi Anna. Adaptive accompaniment with realchords. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (2024).

[252] Xu, Canwen, Rosset, Corby, Chau, Ethan, Corro, Luciano, Mahajan, Shweti, McAuley, Julian, Neville, Jennifer, Awadallah, Ahmed, and Rao, Nikhil. Automatic pair construction for contrastive post-training. In *Findings of the Association for Computational Linguistics: NAACL 2024* (Mexico City, Mexico, June 2024), Association for Computational Linguistics, pp. 149–162.

[253] Xu, Wenda, Zhu, Guanglei, Zhao, Xuandong, Pan, Liangming, Li, Lei, and Wang, William. Pride and prejudice: LLM amplifies self-bias in self-refinement. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Bangkok, Thailand, Aug. 2024), Association for Computational Linguistics, pp. 15474–15492.

[254] Yamkovenko, Bogdan. Khan academy efficacy results, november 2024. Online: `https://blog.khanacademy.org/khan-academy-efficacy-results-november-2024/`, 2024.

[255] Yaneva, Victoria, North, Kai, Baldwin, Peter, Ha, Le An, Rezayi, Saed, Zhou, Yiyun, Ray Choudhury, Sagnik, Harik, Polina, and Clauser, Brian. Findings from the first shared task on automated prediction of difficulty and response time for multiple-choice questions. In *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)* (Mexico City, Mexico, June 2024), Association for Computational Linguistics, pp. 470–482.

[256] Yang, Boyang, Tian, Haoye, Pian, Weiguo, Yu, Haoran, Wang, Haitao, Klein, Jacques, Bissyandé, Tegawendé F, and Jin, Shunfu. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (2024), pp. 882–894.

[257] Yang, Chuanpeng, Zhu, Yao, Lu, Wang, Wang, Yidong, Chen, Qian, Gao, Chenlong, Yan, Bingjie, and Chen, Yiqiang. Survey on knowledge distillation for large language models: Methods, evaluation, and application. *ACM Trans. Intell. Syst. Technol.* (Oct. 2024).

[258] Yang, Kaiqi, Chu, Yucheng, Darwin, Taylor, Han, Ahreum, Li, Hang, Wen, Hongzhi, Copur-Gencturk, Yasemin, Tang, Jiliang, and Liu, Hui. Content knowledge identification with multi-agent large language models (llms). In *Artificial Intelligence in Education* (2024), pp. 284–292.

[259] Yang, Yang, Shen, Jian, Qu, Yanru, Liu, Yunfei, Wang, Kerong, Zhu, Yaoming, Zhang, Weinan, and Yu, Yong. Gikt: A graph-based interaction model for knowledge tracing. In *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases* (2020).

[260] Yu, Yue, Zhuang, Yuchen, Zhang, Jieyu, Meng, Yu, Ratner, Alexander, Krishna, Ranjay, Shen, Jiaming, and Zhang, Chao. Large language model as attributed training data generator: a tale of diversity and bias. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (Red Hook, NY, USA, 2023), NIPS '23, Curran Associates Inc.

[261] Yuan, Zhihao, Xiao, Yunze, Li, Ming, Xuan, Weihao, Tong, Richard, Diab, Mona, and Mitchell, Tom. Towards valid student simulation with large language models. *arXiv preprint arXiv:2601.05473* (2026).

[262] Yue, Xiang, Inan, Huseyin, Li, Xuechen, Kumar, Girish, McAnallen, Julia, Shajari, Hoda, Sun, Huan, Levitan, David, and Sim, Robert. Synthetic text generation with differential privacy: A simple and practical recipe. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Toronto, Canada, July 2023), Association for Computational Linguistics, pp. 1321–1342.

[263] Zawacki-Richter, Olaf, Marín, Victoria I., Bond, Melissa, and Gouverneur, Franziska. Systematic review of research on artificial intelligence applications in higher education – where are the educators? *International Journal of Educational Technology in Higher Education 16* (2019).

[264] Zelikman, Eric, Ma, Wanjing, Tran, Jasmine, Yang, Diyi, Yeatman, Jason, and Haber, Nick. Generating and evaluating tests for k-12 students with language model simulations: A case study on sentence reading efficiency. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (Singapore, Dec. 2023), Association for Computational Linguistics, pp. 2190–2205.

[265] Zent, Matthew, Smith, Digory, and Woodhead, Simon. PIIvot: A lightweight NLP anonymization framework for question-anchored tutoring dialogues. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (Suzhou, China, Nov. 2025), Association for Computational Linguistics, pp. 27467–27476.

[266] Zhan, Bojun, Guo, Teng, Li, Xueyi, Hou, Mingliang, Liang, Qianru, Gao, Boyu, Luo, Weiqi, and Liu, Zitao. Knowledge tracing as language processing: A large-scale autoregressive paradigm. In *Artificial Intelligence in Education* (Cham, 2024), Springer Nature Switzerland, pp. 177–191.

[267] Zhang, B., and Arroyo, I. Improving student engagement with the mathspring intelligent tutoring system: A reinforcement learning approach. In *EDULEARN25 Proceedings* (30 June-2 July, 2025 2025), 17th International Conference on Education and New Learning Technologies, IATED, pp. 9054–9063.

[268] Zhang, Haotian, Shen, Shuanghong, Xu, Bihan, Huang, Zhenya, Wu, Jinze, Sha, Jing, and Wang, Shijin. Item-difficulty-aware learning path recommendation: From a real walking perspective. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (2024), pp. 4167–4178.

[269] Zhang, He, Xie, Jingyi, Wu, Chuhao, Cai, Jie, Kim, ChanMin, and Carroll, John M. The future of learning: Large language models through the lens of students. In *Proceedings of the 25th Annual Conference on Information Technology Education* (2024), pp. 12–18.

[270] Zhang, Jiani, Shi, Xingjian, King, Irwin, and Yeung, Dit-Yan. Dynamic key-value memory networks for knowledge tracing. In *Proc. Int. Conf. World Wide Web* (Apr. 2017), pp. 765–774.

[271] Zhang, Jiayi, Borchers, Conrad, Aleven, Vincent, and Baker, Ryan S. Using large language models to detect self-regulated learning in think-aloud protocols. *International Educational Data Mining Society* (2024).

[272] Zhang, Tianyi, Kishore, Varsha, Wu, Felix, Weinberger, Kilian Q., and Artzi, Yoav. Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations* (2020).

[273] Zhang, Yanzhao, Li, Mingxin, Long, Dingkun, Zhang, Xin, Lin, Huan, Yang, Baosong, Xie, Pengjun, Yang, An, Liu, Dayiheng, Lin, Junyang, Huang, Fei, and Zhou, Jingren. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176* (2025).

[274] Zhang, Zheyuan, Zhang-Li, Daniel, Yu, Jifan, Gong, Linlu, Zhou, Jinchang, Hao, Zhanxin, Jiang, Jianxiao, Cao, Jie, Liu, Huiqin, Liu, Zhiyuan, Hou, Lei, and Li, Juanzi. Simulating classroom education with LLM-empowered agents. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (Albuquerque, New Mexico, Apr. 2025), Association for Computational Linguistics, pp. 10364–10379.

[275] Zhou, Ya, and Tao, Can. Multi-task bert for problem difficulty prediction. In *2020 International Conference on Communications, Information System and Computer Engineering (CISCE)* (2020), pp. 213–216.